Embedded Operating System and Architecture

Stridi Simone

Anno Accademico 2026/2027

Contents

1 Introduction to Embedded Systems and Toolchain **9**

1.1 The Chain of Evidence and the Bare-Metal Toolchain 9
1.2 What is an Embedded Operating System? 10
1.2.1 Key Properties of Embedded Systems 10
1.3 Operating System Architectures 11
1.3.1 Monolithic OS Structure 11
1.3.2 Microkernel OS Structure 11
1.3.3 Flat OS Structure (Bare-Metal) 12
1.3.4 What we will do in the course? 12
1.4 The Target Machine: Cortex-M3 and QEMU 13
1.4.1 Microcontroller (MCU) vs General-Purpose CPU 13
1.4.2 The QEMU target (`mps2-an385`) 13
1.4.3 Peripheral Registers and the `volatile` Keyword 13
Practical Example of Compiler Optimization 14
1.5 Technical Foundations & Toolchain 14
1.5.1 From Source Code to a Running Program 14
1.5.2 Why Cross-Compilation? 14
Checking the Toolchain: Host vs Target 15
1.5.3 Host, Target, and Emulator 15
1.5.4 Reset Code and the Transition to C 16
1.5.5 Preprocessing: Assemble the Translation Unit 16
Try It: Preprocess Only 16
1.5.6 Compilation: C Becomes Target Instructions 17
Compilation and Target Architecture 17
Try It: Stop After Compilation 17
1.5.7 Assembly: Instructions Become Relocatable Bytes 18
Try It: Assemble to an Object File 18
Inspecting Object Files 18
1.5.8 Linking: Creating the Executable 19
The Linker Script 19
Try It: Link the Objects 19
The Linker Script and the Map File 19
1.5.9 The ELF Image and its Map 20
Thumb State and The Entry Point 20
Sections Layout in Week 1 20
File Size vs RAM Usage 21
1.5.10 Debug Information for Host Tools 21
1.5.11 What Make Actually Does 21

Targets Model Desired Outcomes . . . 21
Prerequisites Capture Rebuild Logic . . . 22
1.5.12 Variables and Compiler Flags . . . 22
Build Directories Keep Artifacts Separate . . . 23
Variables and String Manipulation in Make . . . 23
1.5.13 Defining Targets and Rules . . . 23
Diagnosing the "Missing Separator" Error . . . 24
Variable Assignment Types and Debugging . . . 24
Automatic Variables . . . 25
Pattern Rules and Substitution References . . . 25
Phony Targets . . . 25
Reading the Week 1 Makefile as a Dependency Graph . . . 26
A Disciplined First Build Loop . . . 26
1.5.14 QEMU Runs a Board Model . . . 27
1.5.15 QEMU, Semihosting, and the Observed Result . . . 27
Why Avoid `printf`? . . . 27
Running the Demonstration . . . 27
Interpreting the Output . . . 28
The Final Spin Loop and QEMU Execution . . . 28
1.6 A Practical Inspection Routine . . . 28
1.6.1 Systematic Troubleshooting . . . 28
1.6.2 Foundational Significance . . . 29

2 Boot and CPU Configuration 31
2.1 Introduction to Low-Level CPU Control . . . 31
2.1.1 Where You Start and Where You End . . . 31
2.2 Memory Technologies in Embedded Systems . . . 32
2.2.1 SRAM (Static RAM): The Working Memory . . . 32
2.2.2 NOR Flash: Execute-in-Place Memory . . . 33
2.2.3 Contrasting with NAND Flash and Block Devices . . . 33
2.2.4 Applying Constraints to C Code . . . 33
2.2.5 The Cortex-M Memory Map . . . 34
`MEMORY`: Defining the Regions . . . 34
Symbol Assignments for the Stack Pointer . . . 35
`SECTIONS`: Output Sections Built from Input Sections . . . 35
2.2.6 Run Address and Load Address: The `.data` Section . . . 36
`.bss`, Heap, and Stack: RAM With No Stored Image . . . 36
2.2.7 Linker-Defined Symbols Are Addresses, Not Variables . . . 37
2.2.8 Reading the Result: The Map File, `nm`, and `objdump -h` . . . 37
Analyzing the Base Image . . . 38
Pointers, Local, and Static Variables . . . 38
Try It: Testing Global Declarations . . . 38
Pointers in RAM and Little-Endianness . . . 39
Local vs. Static Local Variables . . . 40
2.2.9 Advanced Memory Layout: Custom Sections and Orphans . . . 40
2.2.10 Overflowing a Region . . . 41
2.3 Reset: The Hardware Contract . . . 41
2.3.1 The Vector Table . . . 41

The Thumb Bit Constraint . . . 42
`KEEP` and Garbage Collection . . . 42
Try It: Analyzing the Lockup Error . . . 42
2.3.2 Back to the Base: Why Only One Line Prints . . . 43
2.4 Taking Control of the CPU . . . 43
2.4.1 What is a Debugger? . . . 44
2.4.2 A Remote Debugger: Two Programs, Two Terminals . . . 44
Reconstructing the Debug Target . . . 44
The GDB Commands You Need . . . 45
2.4.3 Attaching at Reset . . . 45
2.4.4 Cortex-M3 Programmer's Model: The Registers . . . 45
The Status Register (xPSR) and CPU Modes . . . 46
2.4.5 ARM Assembly Basics: A Load/Store Architecture . . . 46
Constants, Labels, and Jumps . . . 47
Control Flow: Decisions, Bits, and Interrupts . . . 47
Assembler Directives . . . 48
2.5 Writing the Startup Script (`Reset_Handler`) . . . 48
2.5.1 Steps 1 and 2: Mask Interrupts and Set the Stack . . . 49
2.5.2 Step 3: Copy `.data` from FLASH to RAM . . . 49
Watching the Copy Loop in GDB . . . 50
Try It: Breaking the Stack on Purpose . . . 51
Test 1: Address Outside of RAM (`0x30000000`) . . . 51
Test 2: Unaligned Stack Pointer (`0x20000001`) . . . 51
2.5.3 Step 4: Zero the `.bss` Section . . . 51
2.6 Exceptions and Faults: One Family . . . 52
2.6.1 Completing the Vector Table with Weak Symbols . . . 53
The Effect on Memory Layout . . . 53
2.6.2 CPU Settings Live at Memory Addresses . . . 54
The Read-Modify-Write Pattern and `volatile` . . . 54
2.6.3 Step 5: VTOR, Where the Vector Table Is . . . 54
2.6.4 Step 6: SHCSR, Separate Fault Handlers . . . 55
2.6.5 Step 7: CCR, What Counts as a Fault . . . 55
2.6.6 Step 8: Exception Priorities . . . 56
2.6.7 Step 9 and After: PRIMASK and Handing Over to C . . . 57
2.6.8 Checking the Configuration from GDB . . . 58
2.6.9 Catching a Divide by Zero . . . 58
Triggering the Fault . . . 59
Validating the Configuration Steps . . . 59
2.6.10 What Real Chips Add (`SystemInit`) . . . 59
2.7 Life Without an Operating System . . . 60
2.7.1 Hosted vs. Freestanding C Environments . . . 60
What is Available in Freestanding C? . . . 61
2.7.2 What is Really Under `printf` . . . 61
2.7.3 Why This Course Uses No libc At All . . . 62
2.7.4 The Build Says It: The Flags in the `Makefile` . . . 62
Why Add `libgcc` Back? . . . 63
Try It: The `printf` Experiment . . . 63
2.7.5 Semihosting: How the Program Still Prints . . . 63

Implementing Semihosting in C 64
2.7.6 Semihosting is a Trap 64
2.7.7 Privileged and Unprivileged Code on Cortex-M 65
The CONTROL Register 65
2.7.8 Dropping Privilege: A One-Way Street 65
2.7.9 What Unprivileged Code Cannot Do (The Sandbox Constraints) 66
Try It: The HardFault Escalation 66
Debugging the CONTROL Register 67
2.7.10 SVC: The Door That Stays Open 67
2.7.11 This Is a System Call 67
2.7.12 Adding the System Call 68
2.7.13 Touching the Hardware Without Privilege 68
Conclusion: Privilege and CPU Modes 69
2.8 Conclusion: The Final Result 69
2.9 Appendix: Where to Find the Code 70
2.9.1 The Base Skeleton 70
2.9.2 The Finished Result 70

3 Devices, Interrupts, and System Calls 71
3.1 Introduction: From CPU Configuration to OS Architecture 71
3.1.1 Organizing Code for an Operating System 71
3.2 CMSDK and CMSIS: ARM's Hardware and Software Standards . . . 72
3.2.1 How the Standards Work 72
3.2.2 Our Approach in This Course 73
3.3 The Emulated Board and the I/O Landscape 73
3.3.1 Memory-Mapped I/O 73
3.4 Hardware Architecture: AHB vs. APB Buses 74
The AHB-to-APB Bridge and Performance 74
3.5 The Board's Interrupt Lines 74
3.6 Software Organization: `include/cmsdk.h` 75
3.6.1 Registers as a Struct 76
3.6.2 Why Use Structures? 76
3.6.3 Bitmasks and Hardware Control 76
3.7 Try It: The Board's Memory Map from QEMU 77
Interpreting the QEMU Memory Map Output 77
3.7.1 Try It: Touch a Device That Is Not There 78
3.8 The General I/O Landscape 79
3.8.1 DMA: Moving Data Without the CPU 79
3.9 Memory-Mapped I/O: Loads and Stores with Side Effects 80
3.9.1 Hardware Side Effects 80
3.10 The `volatile` Keyword 81
3.10.1 Try It: `volatile` at `-O2` 81
3.11 Read-Modify-Write Needs Care 82
3.11.1 Safer Hardware Patterns 83
3.12 The Device Protocol and the Driver Concept 83
3.12.1 What is a Device Driver? 83
3.13 Writing the UART Driver 84
3.13.1 The CMSDK UART Registers 84

Configuring the Baud Rate 84
3.14 Implementing the Device Driver 85
3.14.1 How Bigger OSes Generalize a Driver 85
3.14.2 Try It: The First UART Byte 86
3.14.3 Structuring the Driver Files 87
3.14.4 Step-by-Step Implementation 87
1. Initialization 87
2. Polled Transmission 87
3.14.5 Using the Driver as an Application Designer 88
3.14.6 The Reality of Hardware Printing 89
3.14.7 Try It: A Polled Echo 89
3.15 Variadic Functions and `<stdarg.h>` 90
3.15.1 Separating Formatting from Output (`vformat`) 91
3.16 The Kernel Print Function (`kprintf`) 91
3.16.1 Kernel Mode vs. User Mode Printing 91
3.16.2 Implementing `kprintf` 91
3.17 Upgrading the Kernel: Fault Handlers That Print 92
3.17.1 The Status Registers (`HFSR` and `CFSR`) 93
3.17.2 Implementing the New Handlers 93
3.17.3 Try It: Catching a Divide-By-Zero 94
3.17.4 Practical Note: Project Navigation and Exam Expectations . 94
3.18 Unit 2: Calling Convention and Interrupts 95
3.18.1 The Limitations of Polling 95
3.18.2 The Interrupt-Driven Alternative 95
3.18.3 Practical Note: Project Navigation and Exam Expectations . 96
3.19 Unit 2: Calling Convention and Interrupts 96
3.19.1 The Limitations of Polling 96
3.19.2 The Interrupt-Driven Alternative 96
3.20 Polling vs. Interrupts: A Conceptual Overview 97
3.20.1 Hardware Limitations 97
3.21 Function Calls and the ARM Calling Convention 98
3.21.1 Register Roles in AAPCS 98
3.22 Writing Assembly Callable from C 99
3.22.1 Applying the Convention 99
3.23 Managing the CPU's Interrupt State 99
3.23.1 Designing the OS Interrupt API 100
3.24 Implementing `irq_save` and `irq_restore` in Assembly 100
3.24.1 Implementing `irq_save` 100
3.24.2 Implementing `irq_restore` 101
3.24.3 Try It: Testing the API in C 102
3.24.4 The Philosophy of OS Design 103
3.25 ARM Exception Entry and Return Mechanics 103
3.25.1 The Automatic Hardware State Save (Stacking) 103
3.25.2 The `EXC_RETURN` Value 103
3.26 The Heartbeat of the OS: The SysTick Timer 104
3.26.1 The Necessity of the System Tick 104
3.27 Hardware Preemption and the System Tick 104
3.27.1 The SysTick Hardware Registers 105

3.28 Building the SysTick Driver . . . 105
3.28.1 The API Definition (`systick.h`) . . . 105
3.28.2 The Implementation (`systick.c`) . . . 106
3.28.3 The Exception Handler . . . 106
3.28.4 The `volatile` Variable and the Uptime Calculation . . . 107
3.28.5 Try It: Proving the Hardware Interrupt Works . . . 108
3.28.6 A Remark on Emulated Time vs. Real Time . . . 108
3.29 The Nested Vectored Interrupt Controller (NVIC) . . . 109
3.29.1 The Golden Rule of Interrupt Handlers . . . 109
3.30 The Three Gates of Interrupts . . . 110
3.30.1 Shared Interrupt Lines (Q&A) . . . 110
3.31 Expanding the Vector Table for External Devices . . . 110
3.32 Looking Ahead: System Calls and User Mode . . . 111

4 Exercises 113
4.1 week-01-Lab . . . 113
4.1.1 Exercise A2: Construct the Makefile . . . 113
Deep Dive: LDFLAGS, the Map File, and QEMU . . . 113
1. LDFLAGS and Their Importance . . . 113
2. What the output.map File Actually Does . . . 113
3. The QEMU Command and Its Parameters . . . 114
4.1.2 Exercise A3: Hosted Environment and Inline Assembly . . . 115
Deep Dive: Semihosting Initialization . . . 115
Deep Dive: Inline Assembly Mechanics . . . 116
The Complete Implementation . . . 116
4.1.3 Exercise A4: Transfer to the Freestanding Image . . . 117
Deep Dive: Diagnosing the Linker Error . . . 117
ISO C Standard: Hosted vs. Freestanding . . . 118
Empirical Proof: The Size Cost of `libc` . . . 118
The Freestanding Workaround and Final Code . . . 119
4.1.4 Exercise B1: Extending the Makefile and the Dependency Graph 120
Extending Targets: `size` and `disasm` . . . 120
Rebuild Behavior and Timestamp Dependencies . . . 120
Understanding the Disassembly Output (`objdump`) . . . 120
4.1.5 Exercise B2: Comparing Generated vs. Hand-Written Code . 121
Unoptimized Translation (`-O0`) . . . 121
Size-Optimized Translation (`-Os`) . . . 122
4.1.6 Exercise B3: Extended Inline Assembly and the Clobber List . 122
The Implementation . . . 123
Why the Clobber List Needs "memory" . . . 123
4.1.7 Exercise B4: Memory Layout and Symbol Location . . . 123
Symbol Mapping Verification . . . 123
Tool Preference: `nm` vs. `output.map` . . . 124
4.1.8 Exercise B5: Provoking Link-Time Failures . . . 124
Failure 1: Stack and Heap Collision . . . 124
Failure 2: Flash Memory Overflow . . . 125
Statement of the Difference . . . 125
4.1.9 Exercise B6: Recovering the Vector Table from the Image . . . 125

1. The First Word: Initial Main Stack Pointer 125
2. The Second Word: The Reset Handler and the Thumb Bit 125
3. The Remainder of the Table: The Default Handler 126
4. The Evidence-Based Boot Sequence 126

Chapter 1

Introduction to Embedded Systems and Toolchain

1.1 The Chain of Evidence and the Bare-Metal Toolchain

Building a bare-metal firmware relies on a strict **chain of evidence**. Every tool in the build pipeline has a specific, essential job to connect your C source code to the physical hardware.

Here is exactly what each component does in practice:

- **Cross-Compiler (e.g., `arm-none-eabi-gcc`):** Your PC cannot run microcontroller code. The cross-compiler's job is to translate your human-readable C code into the exact binary language that the target CPU understands (for the Cortex-M3, this is the **Thumb-2** instruction set).
- **Linker and Linker Script (`.ld`):** The compiler creates isolated blocks of code, but the CPU needs them placed at specific physical addresses. The linker glues these blocks together, using the **linker script** as a map. It dictates that executable code and constants must be saved in permanent **Flash** memory, while writable variables must be placed in volatile **SRAM**.
- **Hardware Vector Table:** A fixed directory located at the very beginning of the memory (address `0x00000000`). When powered on, the CPU is "blind" and reads the first two entries here to know how to start:
  1. The first word tells the CPU where to place its temporary working memory (the **Main Stack Pointer**).
  2. The second word tells the CPU the exact address of the first instruction to execute (the **Reset Handler**).
- **Startup Code (Reset Handler):** A crucial piece of code that runs *before* your `main()` function. Its job is to "prepare the house" (the RAM) for your application. It performs two critical tasks:
  1. **Data Initialization:** It copies variables that have a starting value (e.g., `int x = 5;`, the `.data` section) from the permanent Flash into the working SRAM.

2. **BSS Clearing:** It fills variables that have no starting value (the `.bss` section) with zeros directly in SRAM.

Only after the RAM is perfectly set up, the startup code safely calls `main()`.

Understanding what each component actually does transforms debugging. If a program crashes before even reaching `main()`, a developer immediately knows to check the linker script or the startup code, rather than the application logic.

1.2 What is an Embedded Operating System?

An operating system (OS) manages hardware resources for applications. General-Purpose OS use a Memory Management Unit (MMU) to isolate applications in virtual memory.

An **Embedded Operating System (EOS)** is completely different: it is a highly specialized, tightly constrained OS built to execute a single, fixed workload on a specific hardware device.

1.2.1 Key Properties of Embedded Systems

- **Resource Constraints:** Built for kilobytes of RAM/Flash and milliwatts of power. Zero overhead allowed.
- **Real-Time Determinism (RTOS):** Deadlines matter as much as logic. A correct calculation delivered late (e.g., in an airbag sensor) is a total system failure. Predictability beats raw speed.
- **Flat Memory Architecture:** No virtual memory. The kernel, drivers, and tasks share one physical address space and are compiled into a single firmware image.
- **Static Configuration:** Since the workload is fixed at compile time, dynamic memory allocation (`malloc`) is rarely used. Tasks, memory pools, and hardware resources are statically allocated.

EMBEDDED VS. GENERAL-PURPOSE

	Embedded	General-purpose
Workload	one fixed task	anything the user installs
Resources	KB of RAM/flash	GB of RAM, TB of disk
Timing	often hard deadlines	best-effort
User model	no interactive user	multi-user, multi-tasking
Cost/power	pennies, milliwatts	dollars, tens of watts

Figure 1.1: Embedded vs General Purpose OS characteristics.

1.3 Operating System Architectures

1.3.1 Monolithic OS Structure

The entire operating system runs as a single, large program in a privileged kernel space.

- **Unified Kernel:** Core services (scheduling, memory, file systems, drivers) live together.
- **Maximum Throughput:** Components share the same memory and communicate directly without message-passing overhead.
- **The Risk:** If one driver crashes, the entire system halts (e.g., Linux, Windows).

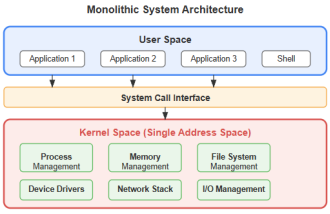


Figure 1.2: Monolithic Operating System Structure.

1.3.2 Microkernel OS Structure

A minimalist design that keeps only essential features (scheduling, basic memory management, IPC) in privileged mode. Everything else runs as isolated user-space apps.

- **Fault Containment (Advantage):** High security and reliability. If a device driver crashes, it restarts independently without bringing down the core OS.
- **Performance Penalty (Disadvantage):** Slower than monolithic systems. Isolated components must constantly talk via Inter-Process Communication (IPC), causing heavy context-switching overhead.

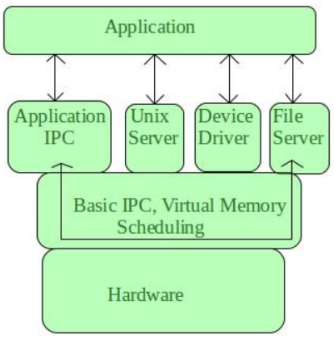


Figure 1.3: Microkernel Operating System Structure.

1.3.3 Flat OS Structure (Bare-Metal)

The simplest architecture, where there are no layers or isolated spaces.

- **Single Linked Image:** The startup code, OS kernel, device drivers, and apps are merged into one executable stored in Flash. No dynamic app loader.
- **Raw Physical Memory:** Everything maps directly to raw physical addresses. No MMU translation.
- **No Hardware Isolation:** No protection boundaries between kernel and user space.

Advantages:

- **High Speed:** Zero latency from OS abstractions or IPC.
- **Simplicity:** Easy to build and deploy for small, dedicated hardware.

Disadvantages & Risks:

- **Zero Isolation:** A single bug (e.g., a bad pointer) in any task can overwrite the entire system memory and cause a hard crash.
- **Blocking Tasks:** A stuck function or infinite loop halts the entire CPU.
- **Poor Scalability:** Unmanageable as code complexity grows.

1.3.4 What we will do in the course?

The starting point (*scratch-os*): The course begins by building *scratch-os*, which utilizes a ***flat firmware architecture***. This exposes the entire internal pipeline, from the initial boot process to task scheduling, making the hardware assumptions completely visible and non-negotiable.

The transition (FreeRTOS): Later, we transition to FreeRTOS, a production-ready RTOS closer to a microkernel philosophy. It features a small dedicated kernel and implements real scheduling policies (though, by default, it does not enforce memory protection). The educational goal is clear: ***using a high-level RTOS API is easy only when you already understand the invisible state transitions happening underneath it***.

The exact sequence of core mechanisms you will build manually:

1. **Reset**: Handling the initial hardware state upon power-up.
2. **Initialize memory**: Setting up the RAM (copying '.data', zeroing '.bss') so C code can safely execute.
3. **Start application**: Transitioning control to the application logic.
4. **React to events & Schedule**: Handling hardware interrupts and deciding which task gets CPU time next.
5. **Synchronize**: Ensuring tasks do not corrupt shared data or peripherals.

1.4 The Target Machine: Cortex-M3 and QEMU

Embedded software is built for a known processor and a fixed memory map. It cannot rely on an OS to create a process, assign an address space, or arrange standard I/O.

1.4.1 Microcontroller (MCU) vs General-Purpose CPU

A **microcontroller (MCU)** integrates onto a single chip the components that a desktop computer splits across a motherboard.

- **Integration**: A desktop CPU is just a processing core requiring external buses for RAM and I/O. An MCU has the CPU, RAM, Flash, and Peripherals (Timers, UART, GPIO) built directly into the same chip.
- **Memory Management**: Desktop CPUs use an MMU to provide isolated virtual address spaces. An MCU relies on a ***single flat physical address map*** shared by code, data, and peripherals (No MMU).
- **Hardware Interaction**: On an MCU, hardware is simply memory you write to directly. There is no abstraction layer between your code and a device register.

1.4.2 The QEMU target (`mps2-an385`)

Instead of physical hardware, the course uses **QEMU**, a software emulator. Specifically, it targets the `mps2-an385` machine, which models an **ARM Cortex-M3** processor. The linker script strictly defines its physical memory map:

- **FLASH (Origin: `0x00000000`, 4 MiB):** Non-volatile. Holds the initial image: vector table, executable instructions (‘.text‘), and read-only constants (‘.rodata‘).
- **SRAM (Origin: `0x20000000`, 4 MiB):** Volatile. Holds mutable state: initialized globals (‘.data‘), zero-initialized globals (‘.bss‘), the heap, and the stack.
- **The Stack:** It starts at the very top of the RAM (`0x20400000`) and ***grows downwards*** to lower addresses.

1.4.3 Peripheral Registers and the `volatile` Keyword

In later weeks, device drivers will interact with peripherals by reading/writing to fixed memory addresses. These memory accesses MUST be declared `volatile` in C.

- **Why `volatile`?**: It tells the compiler that the value at that memory address can change outside the program’s control (e.g., a hardware sensor updating), or that a write triggers a physical effect. Without `volatile`, the compiler’s optimizer might delete the memory access, assuming it’s an unused ordinary variable.
- *Note:* `volatile` does **not** provide atomicity or concurrency protection; it only ensures the access is physically executed by the CPU.

Practical Example of Compiler Optimization Consider a loop waiting for a hardware button press, where the button's state is mapped to a specific memory address:

```
uint32_t *button_status = (uint32_t *) 0x40001000;
while (*button_status == 0) {
    /* Wait for hardware to change the value to 1 */
}
```

Without `volatile`, the compiler's optimizer will assume that since the loop body never modifies `*button_status`, its value will never change. To save time, the compiler will read the memory address *only once* before the loop. If it is 0, it creates an inescapable infinite loop. The program will completely ignore the physical button press[cite: 8].

By declaring it as `volatile uint32_t *button_status`, we force the compiler to physically fetch the value from the RAM address on *every single iteration*, ensuring the hardware's real-time state is always captured[cite: 8].

1.5 Technical Foundations & Toolchain

1.5.1 From Source Code to a Running Program

The goal of this phase is to understand how to compile and run an application step by step, working directly with commands without relying on an IDE (like Visual Studio Code). Often, development environments hide the real compilation process behind a simple "Play" button, but when developing operating systems, it is essential to understand what actually happens under the hood.

The toolchain is a pipeline composed of several small tools, where the output of one becomes the input of the next:

- **C Preprocessor:** manages and expands directives like `#include` and `#define`.
- **Compiler:** takes the preprocessed code and generates assembly code (`.s`).
- **Assembler:** converts the assembly into machine code, producing object files (`.o`).
- **Linker:** concatenates matching sections, resolves symbols to final addresses, and through an `.ld` script, generates the final binary image (the ELF image).

1.5.2 Why Cross-Compilation?

Developing for embedded systems introduces an architectural problem: the computer we are writing the code on (Host) is different from the machine where the code will be executed (Target)[cite: 1]. The host is typically an x86-64 or Apple Silicon machine, whereas our target is an ARM Cortex-M3 processor[cite: 1].

A normal `gcc` compiler installed on the computer emits code optimized for the machine it runs on, which makes it useless for our purpose[cite: 1]. For example, a recent Mac's native compiler would generate code for a 64-bit ARM architecture, while our target board requires code for a 32-bit ARM architecture.

For this reason, a **cross-compilation toolchain** (specifically `arm-none-eabi-*`) is required[cite: 1]. It runs on the host but generates code for the target's ISA (Instruction Set Architecture)[cite: 1].

The `none-eabi` nomenclature is crucial to understand the environment we are working in:

- **none:** indicates the absence of a host operating system on the target (bare-metal development)[cite: 1].
- **eabi:** indicates the use of the standard embedded ARM ABI (Application Binary Interface)[cite: 1].

Working in a "bare-metal" environment means not having the standard libraries typical of desktop operating systems: there will be no `printf` to a console, no `malloc` dynamic allocation supported by an OS, and no `_start` function from a C library unless we explicitly provide it[cite: 1].

Checking the Toolchain: Host vs Target

To concretely verify the difference between compilers, we can simply query the toolchain from the terminal:

- By running `gcc -dumpmachine`, you get the architecture of your host machine (e.g., a 64-bit architecture for a recent desktop system)[cite: 1].
- By running `arm-none-eabi-gcc -dumpmachine`, the output will clearly indicate a 32-bit ARM architecture followed by the `none` environment.

Getting two completely different target strings for the same command typed on the same computer demonstrates the fundamental concept of a cross toolchain[cite: 1]. When writing an operating system from scratch, it is imperative to use this type of compiler, generate the binary file, transfer it to the target machine, and finally run it.

1.5.3 Host, Target, and Emulator

During the development of software for embedded systems, it is essential to keep three key roles distinct in the compilation and testing process[cite: 1]:

- **Host:** This is the PC we work on (e.g., our laptop) and where the text editor, toolchain, and other development tools are executed[cite: 1].
- **Target:** This is the physical hardware destination machine (in our case, a board with an ARM Cortex-M3 processor) for which the executable (the ELF image) is generated[cite: 1].
- **Emulator:** This is a virtual machine that simulates the target's hardware via software directly on our host[cite: 1]. In our case, lacking the physical board, we will use **QEMU** to test the execution of the programs[cite: 1].

1.5.4 Reset Code and the Transition to C

When running a program on a desktop computer, the operating system and the C runtime take care of preparing the memory and starting the program automatically[cite: 10]. In a bare-metal environment, at boot time, no such service exists[cite: 10].

To execute even just the first instruction of our `main()` function, it is necessary to have startup code (`boot/startup.c`) that acts as the bridge between the reset hardware and the application[cite: 10].

This code is responsible for making the C runtime's assumptions true before `main()` runs: it guarantees that every global variable already has its promised value (by initializing the `.data` and `.bss` memory sections) by the time `main()` starts[cite: 10].

The startup source also defines crucial bare-metal safety mechanisms:

- `Default_Handler`: An infinite loop used for unimplemented exception vectors. Stopping in that known loop is much safer and more diagnosable than falling into arbitrary memory after an unexpected hardware exception.

1.5.5 Preprocessing: Assemble the Translation Unit

When we start compiling (e.g., by typing the classic `gcc` command), the process is not monolithic but is divided into very distinct phases. The very first phase is handled by the **C preprocessor**. The preprocessor analyzes the C source code looking for directives (commands starting with the `#` character, such as `#include` or `#define`) and expands these macros before the compiler actually sees the program[cite: 1].

This is purely text substitution, without any machine code generation[cite: 1]. For example, upon encountering an `#include` command, the preprocessor locates the specified file and essentially "copies and pastes" it into the source file, replacing the original directive. The result of this operation is a *translation unit*: a single file of pure C code with all macros and headers already expanded[cite: 1].

Try It: Preprocess Only

We can instruct the toolchain to run *only* the preprocessing phase by using the `-E` flag. The command to run in the terminal is the following[cite: 1]:

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding
-E src/main.c -o build/main.i
```

In this string, we provide several options to describe to the toolchain the characteristics of the machine we actually have available[cite: 1]:

- `-mthumb` and `-mcpu=cortex-m3`: these indicate, respectively, to use the Thumb instruction set and that the target CPU is a Cortex-M3[cite: 1].
- `-nostartfiles` and `-ffreestanding`: these signal that we are working in a bare-metal environment, meaning without standard operating system libraries or default startup files[cite: 1]. These options will be the default for our projects.
- `-E`: commands the compiler to stop immediately after preprocessing[cite: 1].

If we open the generated output file (`main.i`), we will notice fundamental changes compared to the original source:

1. All text comments present in the source have been removed by the preprocessor.

2. The `#include <stdint.h>` directive has disappeared, entirely replaced by the expanded text (the typedef definitions) contained in that header file[cite: 1].

3. The original C code of the `main()` function has remained textually identical at the bottom of the file[cite: 1].

This demonstrates that the actual compiler, in subsequent steps, will never read our original `main.c`, but will work on this "translated" file (`main.i`), which can become huge if there are many files to include.

1.5.6 Compilation: C Becomes Target Instructions

Once preprocessing is finished, the compiler itself comes into play. The role of this step is to check the C syntax and translate the preprocessed code into **assembly** code specific to our target architecture[cite: 1]. All the options we provided earlier (such as `-mcpu`) are needed precisely by the compiler to know exactly which instructions to generate for 32-bit ARM microprocessors.

Compilation and Target Architecture

There are many 32-bit ARM microprocessors available, so it is necessary to specify the exact model being used, which in our case is the Cortex-M3. This specific CPU supports the Thumb instruction set (Thumb-2)[cite: 1]. Therefore, the compiler must be instructed precisely about the target CPU by providing specific options[cite: 1]. Finding the correct configuration involves matching the hardware datasheet specifications with the extensive GCC compiler documentation. Once the correct configuration is established and understood, it can be seamlessly applied to all future projects running on that same target machine.

Try It: Stop After Compilation

To observe the compilation phase without proceeding further in the pipeline, we can run the compiler using the `-S` flag (instead of the `-E` flag used for preprocessing)[cite: 1]. The `-S` option instructs the toolchain to run the preprocessor, compile the code, and stop immediately after producing the assembly version of the C program[cite: 1].

The output is typically stored in a file with a `.s` extension, such as `main.s`. Opening this file reveals that the original high-level C program has been translated into low-level ARM assembly instructions[cite: 1]. These instructions are written using human-readable mnemonics, not raw binary code, proving that the compiler's role at this specific stage is purely translation into assembly language[cite: 1]. While reading these files is not strictly required for this course, it is crucial to know that this intermediate translation step always takes place.

1.5.7 Assembly: Instructions Become Relocatable Bytes

Once the assembly version of the code is generated, the next tool in the chain is the **Assembler**. Its role is similar to a compiler, but it specifically takes the assembly code (`.s`) and translates it into binary machine code[cite: 1].

The output of this phase is an **object file** (identifiable by the `.o` extension). However, it is important to emphasize that the binary code contained in an object file is still not executable firmware[cite: 1]. Because projects are usually composed of multiple source files, the assembler processes each file in complete isolation. As a result, the generated object files contain machine-code bytes, section names, and symbol definitions, but they still lack proper, final memory addresses[cite: 1]. They may also contain unresolved references to code located in other object files (for instance, `startup.o` referring to `main`), which cannot be resolved at this stage[cite: 1].

Try It: Assemble to an Object File

To run the complete compiling chain up to the end of the assembly phase (executing preprocessing, compiling, and assembling sequentially), the `-c` flag must be used[cite: 1]. We can apply this to the different source files in our project:

- `arm-none-eabi-gcc [...] -c boot/startup.c -o build/boot/startup.o`[cite: 1]
- `arm-none-eabi-gcc [...] -c src/main.c -o build/src/main.o`[cite: 1]

By doing this, we generate the object files `startup.o` and `main.o`. Unlike the previous `.i` and `.s` files, these object files represent the first output in the pipeline that is entirely in binary format[cite: 1]. Opening them with a standard text editor will reveal unreadable characters, as they are no longer in a human-readable format.

Inspecting Object Files

To understand what is inside an object file, we can use a tool provided by our toolchain called `arm-none-eabi-nm`[cite: 1]. This tool reads the `.o` files and extracts information in a human-readable way, specifically focusing on "symbols"[cite: 1].

Symbols represent the memory addresses that our program uses, which at this assembly stage are still left open and undefined[cite: 1]. For example, if we run the tool on our object files, we can look for the `main` function:

- In `startup.o`, there is a call to the `main` function, but it is not defined there. The tool will label this symbol with a **U** (Undefined), meaning it is referenced but its address is unknown[cite: 1].
- In `main.o`, the function is actually written and defined. The tool will label this symbol with a **T** (indicating it is defined in the `.text` section)[cite: 1].

This separation is completely normal: the files are compiled in isolation, and it will be the linker's job to connect the "U" to the "T" and assign the final memory addresses[cite: 1].

1.5.8 Linking: Creating the Executable

As established, object files are not runnable firmware[cite: 1]. In ARM architecture, every assembly line corresponds to a 32-bit instruction with a fixed binary encoding. However, whenever an instruction needs to reference a memory address (like calling a function), those bits are left blank and filled with symbols instead.

To convert these isolated objects into a final executable, we need a last step: the **Linker**[cite: 1]. The linker takes all the independent object files, concatenates matching sections, resolves every symbol to a final address, and patches all the references[cite: 1]. The output is an executable file formatted as **ELF** (Executable and Linking Format), which is the standard format originally designed in Unix and used globally (unlike the proprietary `.exe` format used by Windows)[cite: 1].

The Linker Script

When compiling a program on a standard PC, the operating system manages memory automatically, so the linker already knows what to do[cite: 1]. However, in a bare-metal embedded system, there is no operating system to rely on[cite: 1].

Therefore, we must explicitly instruct the linker on how to organize the different pieces of code in memory using a **Linker Script** (usually a `.ld` file)[cite: 1]. This script specifies the physical addresses of the hardware, detailing exactly what goes into the Flash memory (e.g., `.text`, `.rodata`) and what goes into the SRAM (e.g., `.data`, `.bss`, stack)[cite: 1]. Without this script, the toolchain wouldn't know the chip's specific memory map[cite: 1].

Try It: Link the Objects

If we want to run the compiler without stopping at intermediate steps and link the project, the command becomes more complex because we must include the linker script and all the necessary object files[cite: 1].

The command looks like this: `arm-none-eabi-gcc [...] -T scripts/mps2_m3.ld [...] build/boot/startup.o build/src/main.o -o build/scratch-os.axf`[cite: 1]

Here, the `-T` flag is crucial because it tells the compiler which linker script to use (`scripts/mps2_m3.ld`)[cite: 1]. After running this, we obtain our final, linked ELF image (`scratch-os.axf`). If we inspect it again with `arm-none-eabi-nm`, the `main` symbol will no longer be marked as "U" (undefined), but will have a finalized memory address assigned by the linker[cite: 1].

The Linker Script and the Map File

To perform the final link, the linker needs a strict description of physical memory[cite: 14]. To provide this, the link command in the Makefile uses specific flags assigned to the `LDFLAGS` variable:

```
LDFLAGS := -T scripts/mps2_m3.ld
LDFLAGS += -Xlinker -Map=build/output.map
```

- **The Linker Script (`-T`):** The first option chooses the course linker script (`scripts/mps2_m3.ld`)[cite: 14]. The script begins by declaring the `FLASH` and

`RAM` regions, then specifies the output sections that occupy them. Its internal syntax defines how it places the vector table so the processor finds it at reset, how it splits an initialized global's storage between a Flash load address and a RAM run address, and how it reserves the heap and stack. While writing this script from scratch is a Week 2 topic, for now, it is provided as-is to make the build work.

- **The Map File (`-Map`):** The second option emits a map file (`build/output.map`)[cite: 14]. This file is an invaluable explanation of exactly where the linker placed each input contribution and symbol[cite: 14].

For Week 1, reading the generated map file is enough evidence that the script successfully assigned the final addresses and generated a valid executable binary file[cite: 14].

1.5.9 The ELF Image and its Map

The final target file generated by the toolchain is `build/scratch-os.axf`. The `.axf` suffix is commonly used for an ARM executable containing debug information, but the underlying file format is standard **ELF (Executable and Linkable Format)**[cite: 14, 15].

The ELF file organizes the program into several useful views[cite: 15, 16]:

- **Sections:** Serve the linker and debugger (e.g., `.text`, `.rodata`, `.data`, `.bss`)[cite: 15, 16].
- **Program Headers:** Describe memory ranges that a loader (or QEMU) places in memory[cite: 15].
- **Symbols:** Assign human-readable names to final physical addresses.

Thumb State and The Entry Point

The Week 1 build generates a 32-bit, little-endian ARM ELF executable. Running the command `arm-none-eabi-readelf -h -S build/scratch-os.axf` extracts the ELF header and reports the entry point at `0x49`[cite: 15].

However, looking at the map file, the `Reset_Handler` is located at `0x00000048`. This mismatch is not an error: the low bit in the ELF entry address (`0x49` vs `0x48`) explicitly records the **Thumb state**. They describe the exact same execution entry, but the `+1` tells the CPU to execute in Thumb-2 mode.

Sections Layout in Week 1

The generated ELF contains the following allocated sections mapping directly to the physical addresses dictated by the linker script:

```
.isr_vector   address 0x00000000   size 0x40
.text         address 0x00000040   size 0xa0
.data         address 0x20000000   size 0x0
.bss          address 0x20000000   size 0x0
.heap         address 0x20000000   size 0x1000
```

The map file (`build/output.map`) explicitly tracks this derivation[cite: 14]. It records the 64-byte vector table at Flash address zero, followed by the startup code (`startup.o`) at `0x40`. The application code (`main.o`) begins at `0xa4`, and the `"toolchain OK\n"` string literal is placed in `.rodata` at `0xd0`.

File Size vs RAM Usage

In this specific Week 1 snapshot, the `.data` and `.bss` sections have a size of `0x0`. This should **not** be misread as evidence that RAM initialization is optional in bare-metal systems[cite: 10, 16]. It merely indicates that this specific, tiny program has no global variables needing those regions yet[cite: 16].

To accurately verify memory usage, developers run `arm-none-eabi-size build/scratch-os.axf` 17]. For Week 1, it reports 224 bytes of text, 0 bytes of data, and 4096 bytes of `bss`[cite: 17]. Note that this 4096-byte `bss` total includes the `.heap` section, deliberately reserved in the linker layout to occupy RAM[cite: 17].

1.5.10 Debug Information for Host Tools

When an operating system is finalized and released, information about the original source code is typically stripped from the binary to reduce size and protect intellectual property. However, during development and debugging, it is highly beneficial to generate binary files that contain debugging information[cite: 48].

Specific compiler options can embed source-level debug metadata inside the ELF[cite: 48]. This does not mean the target machine understands C; it still only runs machine instructions[cite: 48]. Instead, this metadata allows host-side tools (like QEMU or a debugger) to associate a specific binary instruction address with a corresponding C file line, function, or variable[cite: 48]. This bridge between high-level C code and low-level execution makes investigating crashes possible without altering the basic boot image[cite: 48].

1.5.11 What Make Actually Does

While it is crucial to understand how to compile an embedded program manually, step by step, doing so in practice is extremely inefficient[cite: 49]. Remembering every compiler flag, every linker script option, and listing every single C file becomes impossible for complex projects (for instance, the FreeRTOS kernel consists of hundreds of C files)[cite: 49].

To automate the compilation process, we use tools like **Make**, which is the standard toolchain automation used by Linux and FreeRTOS[cite: 49]. A `Makefile` is essentially a text-based script that defines a set of rules to compile many files[cite: 49].

Targets Model Desired Outcomes

The `Makefile` works by defining **targets** and the actions required for each target[cite: 50]. A target is a named file or action that Make knows how to produce[cite: 50]. For example, if you have a target named `all`, you can simply type `make all` in the terminal, and the tool will execute all the instructions associated with that target[cite: 50]. Some targets, like `clean` or `run`, are usually actions rather than

actual files, and they are declared as `.PHONY` to prevent conflicts with a same-named file[cite: 50].

Make is also powerful because it tracks **prerequisites** (the dependency graph)[cite: 50, 54]. If you specify that the linking step requires all object files to be created first, Make will search for the rule to generate those object files if they do not exist[cite: 50, 54]. It works backward, starting from the desired result and executing all the necessary preceding steps[cite: 50, 54].

Prerequisites Capture Rebuild Logic

Another key feature of Make is its ability to rebuild a target only if a prerequisite is newer than the target itself[cite: 49, 51]. For example, if you change one variable in a massive project (like the Linux kernel, which can take hours to compile), you do not want to recompile everything[cite: 49, 51]. Make identifies that only one `.c` file has changed, recompiles that specific file into an object file, and then simply re-runs the linker to update the final ELF image[cite: 49, 51].

To identify if something has changed, Make uses the last modified timestamp of the files[cite: 51]. However, this can sometimes lead to issues. If you are using version control (like Git) or syncing folders via cloud services (like OneDrive or Dropbox), file timestamps can become corrupted or mistakenly set in the future. In these cases, you might modify a file, but Make will refuse to recompile it because it believes the target is already up-to-date. If you encounter strange build behaviors, checking the timestamps of your files is an excellent troubleshooting step.

1.5.12 Variables and Compiler Flags

The Makefile uses a simple language that allows the definition of variables. Variables are incredibly useful because they centralize configuration: you define them once and use them throughout the script.

For instance, the compiler flags encode crucial information such as the target architecture, warning policies, optimization levels, include paths, and bare-metal assumptions. The Makefile's core flags make the target policy explicit:

```
CFLAGS := -mthumb -mcpu=cortex-m3 -nostartfiles -std=gnu17
CFLAGS += -Wall -Wextra -O0 -g3
```

These are not just personal preferences, but a strict contract for the build:

- `CC`: Chooses the cross-compiler (e.g., `arm-none-eabi-gcc`). The "none" nomenclature explicitly means "No OS," which has practical consequences: there is no host-supplied `_start`, no dynamic loader, no process to return to when `main()` finishes, and no guaranteed `printf` console.

- `-mthumb -mcpu=cortex-m3`: Selects the exact instruction subset implemented by the target.

- `-nostartfiles`: This is particularly important. It prevents the toolchain's usual C startup objects from taking responsibility for the reset. It forces the system to rely on our custom `boot/startup.c` to make that responsibility visible.

- `-O0 -g3`: Disables optimizations (`-O0`) and includes maximum debug symbols (`-g3`). This favors a debuggable, source-corresponding image over an optimized one.
- `LDFLAGS`: Governs memory layout and the final link.

Build Directories Keep Artifacts Separate

In a well-structured `Makefile`, it is common practice to save the generated executable file and intermediate objects in a dedicated directory, often named `build`[cite: 53].

Using a build directory provides several benefits[cite: 53]:

- Source directories remain readable and uncluttered[cite: 53].
- The `clean` target can remove generated files safely without risking the deletion of source code[cite: 53].
- Different configurations can coexist in separate build folders[cite: 53].
- A fresh build tests whether the dependencies are complete[cite: 53].

It is crucial to remember never to manually edit the generated object files or ELF bytes in the build directory in an attempt to change program behavior[cite: 53]. All changes must originate from the source code.

Variables and String Manipulation in Make

Using variables in a `Makefile` is not strictly mandatory; you could write raw commands by skipping variables entirely if your script consists of just one command[cite: 52]. However, it is a highly recommended practice for clean and maintainable code. By saving compiling information (such as the compiler name, the target executable name, or the `build` directory path) into variables, you create a single point of entry[cite: 52, 53]. If, for instance, you decide to switch to another compiler, you only need to change one line, and the rest of the file will still function perfectly[cite: 52]. Variables in Make are essentially strings, and you can perform both variable assignment and variable concatenation[cite: 52].

Make also possesses a powerful language for processing strings and performing replacements, somewhat similar to regular expressions. This is particularly useful when managing source files. Instead of manually writing the compiling command for 100 different C files, you can list all the `.c` files in a `SOURCES` variable[cite: 54]. Then, you can use a short string substitution command to automatically generate the list of object files (`OBJECTS`)[cite: 54]. For example, the command can parse the `SOURCES` string, split it by slashes to remove the original directory (e.g., `boot` or `src`), prepend the `build` directory, and replace the `.c` extension with `.o`[cite: 54]. This automation is why Make is an indispensable tool in large projects[cite: 54].

1.5.13 Defining Targets and Rules

The fundamental unit Make reasons about is the **rule**. A Makefile is composed of targets (what you want to execute) and the exact instructions to execute them[cite: 18].

The syntax for a rule involves a target on the left of a colon, followed by its dependencies (prerequisites), and then the recipe:

```
target: prerequisite1 prerequisite2
    recipe line 1
    recipe line 2
```

- **Target:** Normally a file Make knows how to produce (e.g., an object file or ELF image), or an action (like `clean` or `run`)[cite: 16, 18]. Actions must be explicitly declared as `.PHONY` to prevent Make from looking for a file with that name[cite: 16, 18].
- **Prerequisites:** Files that must already be current before the recipe runs. Make's rebuild decision compares modification times: a target is out of date (and its recipe runs) if it does not exist, or if any prerequisite's timestamp is newer than the target's[cite: 16].
- **Recipe:** The sequence of shell commands that turns the prerequisites into the target.

Crucial Syntax Rule: Every recipe line **MUST** start with a literal `Tab` character, not spaces. This is the single most common syntax error in a first Makefile.

Diagnosing the "Missing Separator" Error

If you indent a recipe line with four spaces instead of a Tab, Make will crash and report a cryptic error like:

```
Makefile:5: *** missing separator.  Stop.
```

The "separator" Make refers to is the Tab character it required to recognize line 5 as a recipe. Finding spaces instead, it tried to parse the line as a variable assignment or a new rule, and failed.

Because four spaces and a Tab look identical in an editor, do not inspect the line by simply reading it. Render its whitespace in the terminal using `cat -et Makefile`. This command marks the end of each line with `$` and each Tab with `^I`, making the missing Tab immediately obvious. To avoid this entirely, configure your editor to never expand Tabs to spaces in Makefiles.

Variable Assignment Types and Debugging

Make variables centralize configuration, but how you assign them matters[cite: 17, 18]:

- `:=` (Simply expanded): The right-hand side is evaluated once, immediately. This is the preferred method.
- `=` (Recursively expanded): The right-hand side is re-evaluated every time the variable is used. This can be slow or cause infinite loops.
- `+=`: Appends to whatever the variable already holds.

By convention, uppercase names like `CC`, `CFLAGS`, and `LDFLAGS` are reserved for tools and flags, while lowercase names are for internal bookkeeping[cite: 17, 18].

Making a Makefile Report its Own State: A Makefile can be wrong without failing (e.g., if a substitution is subtly incorrect). To debug what variables actually expanded to, you can ask the Makefile directly using `$(info ...)` or `$(warning ...)`:

```
$(info OBJECTS = $(OBJECTS))
$(warning CFLAGS is $(CFLAGS))
```

These directives are expanded while the Makefile is being *read*, not built, and they print their values to the console regardless of the target you are building, making them excellent temporary debugging aids.

Automatic Variables

Inside a recipe, a handful of special variables refer back to the rule that triggered it, so the same pattern rule can serve every matching file without repeating filenames[cite: 18]:

- `$@`: The target of this rule[cite: 18].
- `$<`: The first prerequisite[cite: 18].
- `$^`: All prerequisites, space-separated, duplicates removed[cite: 18].
- `$(dir $@)`: The directory part of `$@` (a built-in function)[cite: 18].

The Week 1 pattern rule uses exactly these to automatically create build directories and compile every source file[cite: 18]:

```
$(BUILD_DIR)/%.o: %.c
	@mkdir -p $(dir $@)
	$(CC) $(CFLAGS) -c $< -o $@
```

Pattern Rules and Substitution References

A **pattern rule** (`%.o: %.c`) uses a stem `%` that matches on both sides of the colon. Given a request to build an object file, Make matches it automatically without needing one explicit rule per file[cite: 19].

Additionally, a **substitution reference** like `$(SOURCES:%.c=$(BUILD_DIR)/%.o)` acts as a shorthand to convert the source list into the corresponding object file list[cite: 19].

Phony Targets

Actions like `run` and `clean` do not name actual files that Make should create[cite: 19]. Using `.PHONY` tells Make that a target is not a file, ensuring its recipe always runs when requested and preventing conflicts if a file with the same name exists[cite: 19]. `all` is the conventional default goal[cite: 19].

Reading the Week 1 Makefile as a Dependency Graph

Put together, the checkpoint's complete `Makefile` is structured as a dependency graph[cite: 20]:

```
SOURCES := boot/startup.c src/main.c
OBJECTS := $(SOURCES:%.c=$(BUILD_DIR)/%.o)

.PHONY: all clean run
all: $(BIN)

$(BUILD_DIR)/%.o: %.c
        @mkdir -p $(dir $@)
        $(CC) $(CFLAGS) -c $< -o $@

$(BIN): $(OBJECTS)
        $(CC) $(CFLAGS) $(LDFLAGS) $(OBJECTS) -o $(BIN)

run: $(BIN)
        qemu-system-arm ...

clean:
        rm -rf $(BUILD_DIR)
```

Running `make` walks this graph bottom-up[cite: 20]. This provides the core reproducibility benefit: the graph records how the image is made, and re-running it is always correct whether nothing changed, one file changed, or the build directory does not exist yet[cite: 20].

A Disciplined First Build Loop

The `Makefile` operates on a dependency graph[cite: 54]. For instance, if the target executable (`$(BIN)`) cannot be generated because the object files (`.o`) are missing, Make searches for the rule that dictates how to create them[cite: 54].

This rule typically looks like this: `$(BUILD_DIR)/%.o: %.c`[cite: 54]

This means that for every `.o` file on the left side, the dependency is the corresponding `.c` file on the right side[cite: 54]. If the `.o` file is missing or outdated, this rule is triggered[cite: 54]. The actions associated with this rule usually involve:

1. Creating the `build` folder if it does not already exist.

2. Running the compiler with the `-c` flag, which instructs it to compile and stop before linking[cite: 54].

When you simply type `make`, the tool executes these rules in reverse order: it checks for the final ELF file, realizes it needs the object files, realizes it needs to compile the source files, and then executes the steps sequentially from the bottom up. This provides a highly disciplined build loop: you can run `make` to build the ELF, inspect the terminal for any compiler errors, run the generated ELF, and compare the observed output with what you expect[cite: 55].

1.5.14 QEMU Runs a Board Model

The final crucial target in our `Makefile` is the `run` command[cite: 54]. The rule for `run` depends on `$(BIN)`: the finalized ELF file must be available before execution can begin[cite: 54]. If the file is present, Make executes QEMU (`qemu-system-arm`)[cite: 56].

The emulator requires several parameters to function correctly:

- `-machine mps2-an385`: This tells QEMU which specific hardware board it needs to emulate, ensuring the CPU and memory map match our Cortex-M3 target[cite: 56].
- `-kernel`: This parameter passes the generated ELF file to the emulator[cite: 56].

Other options might include disabling the graphical screen (since we only need the terminal output) and enabling semi-hosting to emulate a debugging cable attached to the board.

When this command runs, QEMU emulates the entire physical process: it acts as if a program has flashed the ELF into memory, the power is turned on, and the board starts executing[cite: 56].

1.5.15 QEMU, Semihosting, and the Observed Result

The `run` target first ensures the ELF exists, then invokes QEMU with specific parameters: `-machine mps2-an385`, `-kernel build/scratch-os.axf`, `-nographic`, and semihosting enabled. QEMU supplies the target-side machine model while presenting its console directly through the host terminal.

Why Avoid `printf`?

The application in `src/main.c` intentionally avoids standard `printf`. The custom reset path does not perform the complete C-library initialization expected by newlib's semihosting-backed stdio.

Instead, `semihost_write0` makes the ABI-level request directly:

1. Places semihosting operation `0x04` (`SYS_WRITE0`) in register `r0`.
2. Places the NUL-terminated message pointer in register `r1`.
3. Executes the Thumb breakpoint instruction `bkpt 0xab`.

With semihosting enabled, QEMU intercepts the breakpoint trap, writes the requested string to the host console, and resumes target execution.

Running the Demonstration

To execute the demonstration from the checkpoint directory:

```
make
make run
```

The observable terminal output is:

```
toolchain OK
```

Interpreting the Output

After printing, `main()` returns to the `Reset_Handler`, which enters its final infinite spin loop[cite: 21]. QEMU continues running because the target continues executing that loop; it must be interrupted manually[cite: 21].

Therefore, the single line `toolchain OK` is **not** an exit notification. It is concrete, empirical evidence that the image successfully reached the intended point in the boot path[cite: 21].

The Final Spin Loop and QEMU Execution

It is important to note what happens after the `main()` function finishes its execution. `main()` itself never returns to an operating system because, in a bare-metal environment, there is no OS to return to[cite: 21].

When it does return, the `Reset_Handler` enters a final `for( ;; ) {}` spin loop, because a bare-metal image has nowhere else to go[cite: 21]. This ensures the processor remains safely locked in a known state instead of wandering into uninitialized memory and executing random garbage data[cite: 21].

Crucial Note on Emulator Behavior: QEMU keeps running that infinite loop; it does not exit on its own. Any explanation saying that QEMU automatically exits after printing the final message is strictly incorrect for this checkpoint.

1.6 A Practical Inspection Routine

When working on this checkpoint, it is essential to inspect the source and generated evidence in the exact order of the pipeline:

1. **Read the** `Makefile` to see the commands and target flags.
2. **Read the linker script** (`mps2_m3.ld`) to establish legal memory regions and section order.
3. **Read** `startup.c` to see how linker symbols become initialization loops and how reset reaches C.
4. **Read** `main.c` to identify the expected observable behavior.
5. **Use map and ELF tools** to verify that the linker implemented the intended layout:

```
arm-none-eabi-readelf -h -S build/scratch-os.axf
arm-none-eabi-size build/scratch-os.axf
```

1.6.1 Systematic Troubleshooting

When faults occur during development, follow a systematic troubleshooting path:

- **If the compiler rejects a command:** Check the cross-toolchain installation and the Makefile flags.
- **If a symbol is unresolved:** Check the object list and matching declarations.

- **If a section overflows:** Inspect the linker-script memory regions and the map file.
- **If QEMU starts but the expected output is absent:** Check the reset path, the call to `semihost_write0`, and the semihosting options.

1.6.2 Foundational Significance

This checkpoint serves as the permanent foundation for all subsequent work: future peripherals will use the same physical-address model, task code will still enter through the same reset path, and every future image will be linked, inspected, and booted through this exact toolchain. Week 1 makes these relationships explicit before later operating system abstractions obscure them.

Chapter 2

Boot and CPU Configuration

2.1 Introduction to Low-Level CPU Control

In the previous week, we built a tiny program for a Cortex-M3, ran it in QEMU, and saw it print one line[cite: 101]. Several pieces of that program were handed to us as-is: the linker script that decided where everything goes in memory, and the startup code that ran before `main()`. We were asked to trust them. This phase is about stopping that trust and building those components from scratch[cite: 101].

By the end of this phase, developers will understand, and will have written, every instruction that the CPU executes between power-on and the first line of `main()`[cite: 101]. The development process covers knowing where each piece of a C program lives in the chip's memory and why, how the CPU finds its first instruction, how to watch the CPU one instruction at a time with a debugger, how to configure the CPU so that errors are reported clearly instead of silently, and what it means to run C with nothing underneath it (no C library, no operating system)[cite: 101]. Finally, the program will be split into a privileged and an unprivileged part, building the smallest possible version of a system call (`svc`)[cite: 101].

The process is organized into three core units[cite: 101]:

1. **Where code and data live:** The kinds of memory a microcontroller has, the Cortex-M memory map, how the linker script turns that map into decisions, and what the CPU does at reset[cite: 101].

2. **Taking control of the CPU:** The debugger, the CPU registers, just enough ARM assembly to read and write boot code, and writing the reset handler step by step (stack, `.data`, `.bss`, vector table, fault configuration, priorities)[cite: 101].

3. **Life without an OS underneath:** What "freestanding" C means, how the program prints without a C library, privileged and unprivileged code, and the `svc` instruction as a system call[cite: 101].

2.1.1 Where You Start and Where You End

All the in-class work happens on top of a starting skeleton, specifically the folder `week-02/base/`[cite: 101]. It is derived from the previous week with one major difference: the original C startup code, which did all the setup work, has been thrown away[cite: 101].

In its place is a new, almost empty assembly file (`base/boot/startup.s`)[cite: 101]. It contains only the two words the CPU absolutely needs at reset (a stack address and the address of the first instruction), and a `Reset_Handler` that does nothing except call `main()` and then loop forever if `main()` ever returns[cite: 101].

The program in `base/src/main.c` attempts to print two lines[cite: 101]:

1. A standard string literal[cite: 101].
2. `data_message`, a global character array initialized with the text `"main: this text lives in .data\n"`[cite: 101].

If we build and run the base skeleton (`make` and `make run`), we see only one line[cite: 101]:

```
main: hello from the base
```

The second line is missing[cite: 102]. Nothing crashes, and no error is printed: the second call simply prints an empty string[cite: 102]. Understanding exactly why this happens is the goal of Unit 1, and fixing it is the first objective in Unit 2[cite: 102].

By the end of the week, the fully implemented program will print four distinct lines, each serving as evidence that a specific piece of the low-level architecture is in place[cite: 102]:

```
main: privileged hello
main: this text lives in .data
SVC_Handler: privileged code printing on behalf of unprivileged code
BusFault_Handler: a bus fault happened
```

Practical Note: QEMU does not exit automatically when the program finishes because the bare-metal image enters an infinite spin loop (there is nowhere to return to)[cite: 102]. It must be manually stopped using `Ctrl-C`[cite: 102].

2.2 Memory Technologies in Embedded Systems

When writing a C program on a standard laptop, developers rarely think about *which* memory their variables reside in. On a microcontroller, this is a critical consideration because the chip contains two very different kinds of memory, and each piece of the program can live in only one of them[cite: 102].

2.2.1 SRAM (Static RAM): The Working Memory

The first kind of memory is **SRAM** (Static RAM)[cite: 102].

- **Direct Bus Connection:** SRAM is connected directly to the CPU's memory bus[cite: 102]. Every byte in it has a specific address, and the CPU reads or writes any byte using ordinary load or store instructions (the same instructions the compiler uses for `x = y;`)[cite: 102].
- **Speed:** It is extremely fast; on a microcontroller, one access typically takes about one clock cycle[cite: 102].

- **Writeability:** You can write to a single byte at any time, as often as you like[cite: 102].

- **Volatility:** Its primary weakness is that it is **volatile**: when power goes off, its content is completely lost[cite: 102]. Furthermore, when power comes back on, the content is not zero; it is **undefined**, consisting of a random pattern of bits left by the physical state of the memory cells[cite: 102].

Because of these characteristics, SRAM is the designated location for everything that *changes* while the program runs, specifically your variables and the stack[cite: 102].

2.2.2 NOR Flash: Execute-in-Place Memory

The second kind of memory is **NOR flash**. NOR flash sits on the memory bus, similar to a ROM: every word has a specific address, allowing the CPU to read any of them quickly and in any order. Because of this, the CPU can fetch instructions directly from the flash and execute them right where they are stored, without needing to copy the program into RAM first. This capability is known as **execute-in-place** (XIP).

NOR flash is **non-volatile**, meaning your program remains intact even after the board is unplugged or powered off. The price for these features is paid when writing data. Writing to flash is not a simple store instruction. It requires a two-step process: first, an entire **sector** (a block of a few kilobytes) must be **erased**, setting every bit within it to 1. Then, individual words can be **programmed**, which only turns bits from 1 to 0. Both operations take microseconds to milliseconds and require a special command sequence sent to the flash controller. Consequently, for a running program, NOR flash is effectively **read-only**.

2.2.3 Contrasting with NAND Flash and Block Devices

It is worth contrasting this with the storage technologies commonly found in standard laptops, such as **NAND flash** (used in SD cards and SSDs) and hard disks.

These storage devices are **not** on the memory bus. They are accessed through a controller, one **block** (or page) at a time (typically 512 bytes to 4 KB). A block has no CPU address. The CPU cannot fetch an instruction from it, nor can it read a single variable directly: the contents must first be copied into RAM and then used from there.

This is exactly what a desktop operating system does when you start a program: the OS **loader** copies the executable from the SSD into RAM before running it. A microcontroller, however, has no loader. Its program already sits in NOR flash, at the exact address it runs from.

2.2.4 Applying Constraints to C Code

Put these facts side by side, and you get the rules for a C program on a microcontroller:

Based on these constraints:

Technology	On the bus?	Execute from it?	Write at run time?	Keeps data at po
SRAM	yes	yes	yes, bytes	no
NOR flash	yes	yes (XIP)	no (erase/program)	yes
NAND / disk	no (blocks)	no	through a driver	yes

- **Code and Constants:** Code stays in flash and runs directly from there. Constants (anything declared `const`, and string literals such as `"hello"`) also stay in flash and are read in place.
- **Uninitialized Variables:** Variables must live in RAM because they change.
- **Initialized Variables:** The interesting case is an **initialized** variable such as `int x = 5;`. The variable lives in RAM, but RAM is random at power-on. Therefore, the value `5` has to come from somewhere that survives power-off (flash). So the `5` is **stored in flash**, and something must **copy it into RAM at boot**, before `main()` runs and reads `x`. This missing copy mechanism is the whole reason the base program prints only one line.

2.2.5 The Cortex-M Memory Map

The CPU sees a single address space of 4 GB (using 32-bit addresses). On every Cortex-M chip, this space is rigidly divided into regions whose *purpose* is fixed by the ARM architecture. Each chip manufacturer then decides how much real memory or how many devices to put in each region.

Address range	Region	On `mps2-an385`
`0x0000_0000`–`0x1FFF_FFFF`	Code	4 MB at `0x0`: the program ("FLASH")
`0x2000_0000`–`0x3FFF_FFFF`	SRAM	4 MB at `0x2000_0000`: "RAM"
`0x4000_0000`–`0x5FFF_FFFF`	Peripherals	UART, timers, ...
`0x6000_0000`–`0xDFFF_FFFF`	External RAM / devices	not used here
`0xE000_0000`–`0xE00F_FFFF`	CPU's private registers	control of the CPU itself (at `0xE000_E00`

The code region starts at address 0, and that is where the program goes. SRAM starts at `0x20000000`. Peripherals such as a serial port live at addresses starting at `0x40000000`: you talk to them by reading and writing those addresses. The last region, starting at `0xE0000000`, contains registers that belong to the CPU core itself.

Emulation Detail: On the `mps2-an385` board (and therefore in QEMU), the 4 MB at address 0 is really fast RAM that the board preloads with your program image. It plays the role of flash. You treat it as read-only, exactly as you would on a real microcontroller with real flash.

`MEMORY`: Defining the Regions

In the `MEMORY` block, each line describes one physical region[cite: 104]: a name you choose (`FLASH`, `RAM`), a start address (`ORIGIN`), a size (`LENGTH`), and access attributes in parentheses (`r` for read, `w` for write, `x` for execute).

The linker actively uses these `LENGTH` values to **check your program**: if what you ask it to place in a region does not fit, the link fails with a clear error instead of producing a broken image that cannot work.

Symbol Assignments for the Stack Pointer

Immediately following the region definitions, the script calculates and assigns the `_estack` symbol:

```
_estack = ORIGIN(RAM) + LENGTH(RAM);      /* = 0x20400000 */
```

This value (`0x20400000`) represents exactly one byte past the last available address in the RAM[cite: 104]. On the Cortex-M architecture, the stack grows **downwards** (toward lower memory addresses)[cite: 105]. By initializing the stack pointer at the very top of the RAM, the system gives it the most room to grow before running into anything else[cite: 105].

`SECTIONS`: Output Sections Built from Input Sections

The `SECTIONS` block is responsible for constructing the **output sections** of the final ELF executable out of the **input sections** found in the object files (`.o`)[cite: 105].

Consider the first two standard output sections:

```
.isr_vector :
{
    . = ALIGN(4);
    KEEP(*(.isr_vector))
} > FLASH

.text :
{
    *(.text*)              /* code from every .o          */
    *(.rodata*)            /* constants, string literals  */
    _etext = .;
} > FLASH
```

Syntax Breakdown:

- **Rule execution order:** The block `.text : { ... }` instructs the linker to create a new output section named `.text`[cite: 105]. Inside the braces, a rule like `*(.text*)` means: "from every input file (the first `*`), extract every section whose name matches `.text*`"[cite: 106].
- **Section Folding:** The rules are applied in order, and this order dictates the layout in memory: all code first, then all read-only data[cite: 106]. Notice that `.rodata` does not get its own output section here; the script **folds** it directly into the `.text` section[cite: 106].
- **The Location Counter (`.`):** The single dot (`.`) is the **location counter**: the address the linker is currently at while it places things[cite: 106]. `. = ALIGN(4)` mathematically rounds the current address up to the next multiple of 4[cite: 106].
- **Defining boundaries:** `_etext = .;` defines a symbol whose value is the current address, effectively marking the end of the executable code[cite: 106].
- **Placement:** The `> FLASH` directive at the end says "put this output section in the FLASH region"[cite: 106]. Because `.isr_vector` is the first output section placed in FLASH, it ends up at address 0; `.text` follows right after it[cite: 106].

2.2.6 Run Address and Load Address: The .data Section

The `.data` section, which holds initialized global variables, requires something new:

```
_sidata = LOADADDR(.data);
.data :
{
    _sdata = .;
    *(.data*)
    _edata = .;
} > RAM AT > FLASH
```

Every output section possesses two distinct memory addresses[cite: 106]:

- **VMA (Virtual Memory Address / Run Address):** Where the program *uses* the section when it runs[cite: 106].
- **LMA (Load Memory Address):** Where the section's raw bytes are *stored* in the image[cite: 106].

For `.text`, the two are the same: code runs where it is stored[cite: 106]. For `.data`, they must differ: the variables are used in RAM, but their initial values have to be stored in flash[cite: 106]. The line `> RAM AT > FLASH` says exactly this: run address in RAM, load address in FLASH[cite: 106].

The script also explicitly records the addresses the boot code will need[cite: 107]: `_sdata` and `_edata` are the start and end of `.data` in RAM (its VMA range), and `_sidata` (`LOADADDR(.data)`) is the flash address where the stored copy of the initial values begins[cite: 107].

The Boot Copy Requirement: The key point is that **nothing moves the bytes from the LMA to the VMA by itself**[cite: 107]. The linker only records the two addresses, and the CPU certainly does not copy anything at reset[cite: 107]. Your boot code must do the copy[cite: 107].

Here is the resulting memory layout mapped by the script:

```
FLASH: [.isr_vector][.text + .rodata][copy of .data] <- _sidata
RAM:   [.data] [.bss] [heap] . . . . . [stack ||] <- _estack
       ^ _sdata ^ _sbss
```

.bss, Heap, and Stack: RAM With No Stored Image

The remaining regions defined in the script exist purely in RAM and require no physical storage in the Flash image[cite: 107]:

```
.bss : { _sbss = .; *(.bss*) *(COMMON) _ebss = .; } > RAM
.heap : { ... . = . + _Min_Heap_Size; _heap_top = .; } > RAM
__StackLimit = _estack - _Min_Stack_Size;
ASSERT(__StackLimit >= _heap_top, "region RAM overflowed: ...")
```

- **The `.bss` section:** This section holds variables that start at zero[cite: 107]. There is no point in storing thousands of zeros in the Flash memory, so `.bss` has **no stored content** at all; the image only records its size[cite: 107]. The boot code must actively fill the range from `_sbss` to `_ebss` with zeros at runtime[cite: 107].

- **The `.heap` section:** The heap does not contain anything at boot[cite: 107]. It only **reserves** space[cite: 107]. The statement `. = . + _Min_Heap_Size` moves the location counter forward by 4 KB, ensuring nothing else can be placed in that memory range[cite: 107].

- **The Stack:** The stack is not a section at all; it is simply the top 4 KB of RAM[cite: 107]. `__StackLimit` is defined as `_estack` minus `_Min_Stack_Size`, marking the lowest address the stack is allowed to reach[cite: 107].

Safety Asserts: To prevent catastrophic memory corruption, the script concludes with an `ASSERT` directive[cite: 107]. This makes the **link fail** if the calculated top of the heap runs into the stack limit[cite: 107]. This brilliant mechanism turns a memory corruption that would appear at run time (which is notoriously hard to debug) into a clear, immediate error at build time[cite: 107].

2.2.7 Linker-Defined Symbols Are Addresses, Not Variables

Symbols defined in the linker script such as `_sdata`, `_edata`, `_sidata`, `_sbss`, `_ebss`, and `_estack` are unusual: they have **no storage**[cite: 107].

There is no physical memory cell called `_sdata` holding a value[cite: 107]. The linker only gives each of them an **address**, and that address itself *is* the information[cite: 107]. This distinction is critical when using them from C:

```
extern uint32_t _sdata, _edata;     /* "there is something at this address"
uint32_t *start = &_sdata;          /* OK: gets the address (0x20000000)
*/
uint32_t  oops  = _sdata;           /* WRONG: reads RAM at that address
*/
```

You declare them as `extern` objects only so that C allows you to take their address with the `&` operator[cite: 107, 108]. Reading `_sdata` directly would read whatever random garbage happens to be in RAM at `0x20000000`[cite: 108].

In assembly, the same concept is written as `ldr r0, =_sdata`, which is the equivalent of `r0 = &_sdata;` in C[cite: 108]. The boot code relies on these symbols strictly as loop boundaries: copying from `_sidata` to `_sdata` up to `_edata`, writing zeros from `_sbss` to `_ebss`, and starting the stack exactly at `_estack`[cite: 108].

2.2.8 Reading the Result: The Map File, `nm`, and `objdump -h`

You now know what the linker script *asks* for. You can verify what the linker actually *did* using three standard toolchain tools[cite: 108]:

1. **The Map File (`build/output.map`):** Generated by passing the `-Map=` option to the linker. Its *Memory Configuration* section repeats your `MEMORY` regions, and the *Linker script and memory map* section lists every output section, every input section inside it (with the `.o` file it came from), and the load address for `.data`[cite: 108].

2. **`arm-none-eabi-nm -n`:** Lists all symbols sorted by address[cite: 108]. The letter before each name identifies its kind[cite: 108]:

- **T/t**: Code (text; lowercase means local to its file)
- **R**: Read-only data
- **D**: Initialized data
- **B**: BSS (uninitialized data)
- **A**: Absolute symbol (belongs to no specific section)

3. `arm-none-eabi-objdump -h`: Prints one line per output section, revealing its exact size, its **VMA** (run address), and its **LMA** (load address)[cite: 108].

Analyzing the Base Image

If you run these tools on the base skeleton, the output perfectly validates our linker script[cite: 108]:

- In the `objdump -h` output, `.isr_vector` sits at address `0x0` with a size of 8 bytes (two words), and `.text` starts right after it at `0x8`[cite: 108].
- There is no `.rodata` line in `objdump` because it was folded into `.text`[cite: 108]. You can confirm this in the map file, where `.rodata` is nested inside `.text` with the string literal from `main.o`[cite: 108].
- The `.data` line shows a VMA of `0x20000000` (start of RAM), but an LMA of `0x60` (a small flash address right after the code)[cite: 108].
- In the `nm` output, `_sidata` exactly matches that `0x60` LMA[cite: 108]. Furthermore, `_sdata` is at `0x20000000`, `_edata` is 32 bytes later, `_sbss` and `_ebss` are perfectly equal (since the base has no uninitialized variables), and `_estack` is at `0x20400000`[cite: 108].

Pointers, Local, and Static Variables

Pointers deserve a moment of care, because a pointer is itself a variable[cite: 109]. In the declaration `const char *p = "hi";`, the characters `"hi"` are a string literal and go to `.rodata`; however, `p`, the variable holding their address, is an initialized writable variable and goes to `.data`[cite: 109].

This table of section rules only applies to declarations made **outside any function**[cite: 109]. A local variable inside a function has no section at all: it lives on the stack while the function runs, and it is gone as soon as the function returns[cite: 109].

Try It: Testing Global Declarations

You can test these placement rules by adding the following **global** variables to `src/main.c`, right after the line that defines `data_message` (before `main`), so outside any function[cite: 109]:

```
const int limit = 100;
int counter = 5;
int total;
int zero = 0;
```

```
const char *greeting = "hi";
char buffer[64];
```

Write down your prediction for each one, then check[cite: 109]:

```
make
arm-none-eabi-nm build/src/main.o
arm-none-eabi-nm -n build/scratch-os.axf
```

You find `counter` and `greeting` marked `D` (initialized data), and `total`, `zero`, and `buffer` marked `B` (bss)[cite: 109]. The variable `limit` is the interesting one: in `main.o` it is marked `R` (read-only data), but in the final image it is marked `T`, as if it were code[cite: 109, 110]. The reason is the linker script: it puts `.rodata` *inside* the `.text` output section, and in the final image `nm` classifies a symbol by the output section it ended up in[cite: 110].

The string `"hi"` has no name, so `nm` does not show it, but the map file shows the `.rodata` input sections of `main.o` inside `.text`[cite: 110]. You can find its bytes directly by dumping the contents of the `.text` output section[cite: 110]:

```
$ arm-none-eabi-objdump -s -j .text build/scratch-os.axf
 ...
 0038 00bffde7 4c000000 00000020 64000000  ....L....... d...
 0048 68690000 6d61696e 3a206865 6c6c6f20  hi..main: hello
```

Each line starts with the address of its first byte, shows 16 bytes, and repeats the same bytes as text on the right. At address `0x44` you find `64 00 00 00`: that is `limit`, whose value is 100 (`0x64`)[cite: 110]. At `0x48` you find `68 69 00`, the characters `h` and `i` followed by the terminating zero: the string `"hi"` starts at address `0x48`, in FLASH[cite: 110].

Pointers in RAM and Little-Endianness

How does the program find it? Through `greeting`, which is a pointer stored in RAM. Dump `.data`[cite: 110]:

```
$ arm-none-eabi-objdump -s -j .data build/scratch-os.axf
 20000000 6d61696e 3a207468 69732074 65787420  main: this text
 20000010 6c697665 7320696e 202e6461 74610a00  lives in .data..
 20000020 05000000 48000000                    ....H...
```

Because you added the variables right after `data_message` in `src/main.c`, the first two lines, from `0x20000000` to `0x2000001F`, are `data_message` (you can read its text on the right), and your variables follow: `counter` at `0x20000020` and `greeting` at `0x20000024`[cite: 110]. The compiler places variables in `.data` in the order you define them.

To read the numbers you need one more fact: the Cortex-M3 is **little-endian**, which means that a 32-bit number is stored with its lowest byte first[cite: 110].

- The four bytes `05 00 00 00` are the number 5 (the value of `counter`)[cite: 110].
- The four bytes `48 00 00 00` are the number `0x00000048`[cite: 110]. That is the value of `greeting`, and it is exactly the FLASH address where `"hi"` starts[cite: 110].

This is all a pointer is: a variable whose value is an address[cite: 110].

Local vs. Static Local Variables

You can see the local-vs-global contrast directly. Inside `main`, add `int local = 7;` and `static int s = 3;`, then check `nm` again[cite: 109].

- `local` never appears at all: it is not a global, it has no fixed address, and it only exists on the stack while `main` is running[cite: 109].

- `s` does appear, as `d s.0`: lowercase `d` means it lives in `.data` but is visible only inside this file, and the compiler appends a `.0` (or `.N` suffix) to tell one function's static local apart from another's if they share a name[cite: 109]. Unlike `local`, a `static` local keeps one fixed address for the whole run of the program, exactly like a global — it is only its *name* that stays private to the function[cite: 109].

2.2.9 Advanced Memory Layout: Custom Sections and Orphans

You are not strictly limited to the default sections the compiler chooses[cite: 110]. GCC provides compiler attributes that allow you to explicitly name the input section of a variable or a function[cite: 110]:

```
int tagged __attribute__((section(".mysection"))) = 42;
```

Where that custom section physically ends up is entirely up to the linker script, **if the script names it**[cite: 110]. A section that does not match any rule in the `SECTIONS` block is called an **orphan**[cite: 110]. The linker does not reject an orphan section; instead, it places it by guessing based on its flags (e.g., whether it is writable or executable) and places it adjacent to sections with similar properties[cite: 110].

While this guess usually allows the program to link successfully, the orphan section is placed completely outside your control and, critically, outside any predefined symbol ranges[cite: 110].

If you add the `tagged` variable to the base project and examine the output using `arm-none-eabi-nm -n`, you will see the linker symbols placed next to your variable[cite: 110]:

```
20000000 D _sdata
20000020 D _edata
20000020 D tagged
```

A new output section `.mysection` appears. The linker saw a writable, initialized section, so it placed it in RAM **right after** `.data`, and gave it its own load address in FLASH.

Now compare its address with `_edata`: `tagged` sits exactly at `_edata`, that is, *after* the end of the range `_sdata..._edata`. A boot loop that copies only `_sdata..._edata` will never copy it, so `tagged` will not hold 42 when `main()` runs. The program links without a single warning and is still wrong. That is the fundamental danger of orphans.

If you declare a `const int` in a section called `.myconst` instead, the linker sees a read-only section and places it safely in FLASH, between `.isr_vector` and `.text`.

2.2.10 Overflowing a Region

The `LENGTH` values in the `MEMORY` block are not decoration: the linker actively enforces them. First, measure how much flash the base image needs:

```
arm-none-eabi-size build/scratch-os.axf
```

The FLASH region has to hold `text + data`: the code and constants, plus the stored copy of `.data`'s initial values. For the base skeleton, this is $96 + 32 = 128$ bytes.

If you edit `scripts/mps2_m3.ld` and set FLASH `LENGTH = 100`, you can test this enforcement. Rebuild with `make -B` (the `Makefile` does not automatically list the linker script as a prerequisite, so `-B` forces everything to be rebuilt). The link will intentionally fail with:

```
region 'FLASH' overflowed by 28 bytes
```

$128 - 100$ is exactly 28. If you set `LENGTH = 128`, the link succeeds because the image fits exactly.

If you then restore FLASH and set RAM `LENGTH = 8K` instead, `_estack` moves down to `0x20002000`. The 4 KB stack reservation starts 4 KB below it, meaning the 4 KB heap no longer fits between `.bss` and the stack. The build failure in this scenario comes directly from the script's own `ASSERT` line, printing its custom message about the heap colliding with the stack.

2.3 Reset: The Hardware Contract

Once the memory layout is finalized and loaded into memory, the machine can be powered on. The next question is how the CPU starts using it.

When the Cortex-M3 comes out of reset, **before any of your code runs**, the hardware executes a strictly predefined sequence. It does exactly two things:

1. It reads the 32-bit word at address `0x0` and puts it directly into **SP**, the stack pointer.

2. It reads the word at address `0x4` and puts it directly into **PC**, the program counter: execution starts at that exact address.

There is no BIOS, no loader, no C runtime. Just those two words. Everything else (copying `.data`, zeroing `.bss`, configuring the CPU, calling `main()`) is **your code's job**, and your code starts at the address stored in that second word.

2.3.1 The Vector Table

Those two initial words are the beginning of the **vector table**: a table of addresses at a fixed place in memory (address `0x0` at reset).

The table exists because of **exceptions**. An exception is an event that makes the CPU stop what it is doing and run a specific piece of code, called its **handler**. Reset is one exception; a fault (such as accessing an unmapped address) is another. Each exception has a specific number, and slot N of the vector table holds the address of the handler for exception N. When exception N happens, the CPU hardware reads

slot N and automatically jumps there. Slot 0 is special: it is not the address of any code, it is strictly the initial value of SP.

In the base skeleton, the table is written in `boot/startup.s`, inside a section explicitly called `.isr_vector`, and currently has only two slots:

```
isr_vector:
    .word _estack              /* 0: initial SP   (C: &_estack)
*/
    .word Reset_Handler        /* 1: reset        (C: Reset_Handler) */
```

`.word X` simply stores the 32-bit value `X`, like one element of an array initializer. The label `isr_vector` is the address of the first word. This connects directly to the linker script: `isr_vector` in `boot/startup.s` lives in section `.isr_vector`, and the script places `.isr_vector` first in FLASH, so the table lands exactly at address 0, exactly where the CPU expects to read it.

The Thumb Bit Constraint

One critical detail you will observe in the debugger: the value stored in word 1 is not exactly the raw address of `Reset_Handler`, but that address with its lowest bit set to 1.

Instructions always sit at even addresses (2-byte or 4-byte aligned), so bit 0 of a code address is otherwise unused. ARM uses bit 0 as a flag to mean "this is Thumb code". Because the Cortex-M3 executes *only* the Thumb instruction set, the hardware strictly requires this bit to be 1.

Notice also that nothing in the program ever *calls* or *reads* `isr_vector` programmatically. Only the hardware reads it. That fact matters for how the linker optimizes code.

`KEEP` and Garbage Collection

The linker possesses an optional optimization feature, enabled by passing the `-Wl,-gc-sections` flag, known as **garbage collection of sections**. Starting from the `ENTRY` symbol, the linker meticulously follows every reference from one section to another and **drops every input section that nothing references**.

While this is exceptionally useful for trimming unused code from a final program, it is dangerous for data that only the hardware reads. Because no software code actively references the vector table, the garbage collector would consider it useless and throw it away.

The `KEEP(...)` directive in the linker script marks input sections as **always kept**, overriding the garbage collector. The rule `KEEP(*(.isr_vector))` in `scripts/mps2_m3.ld` exists for exactly this reason. (Currently, the course `Makefile` does not enable garbage collection, making `KEEP` an insurance policy for the future).

Try It: Analyzing the Lockup Error You can see exactly what this insurance protects by deliberately breaking it:

1. In `scripts/mps2_m3.ld`, replace `KEEP(*(.isr_vector))` with a plain `*(.isr_vector)` and rebuild with `make -B`. `objdump -h` still shows `.isr_vector` at 8 bytes because garbage collection is off.

2. Add the line `LDFLAGS += -Wl,-gc-sections` to the `Makefile` and rebuild with `make -B` again.

Now, the compilation output drastically changes:

- `objdump -h` shows `.isr_vector` with a size of `0`, and `.text` now starts at `0x0`.
- The map file lists `.isr_vector` under *Discarded input sections*.
- `nm -n` shows `Reset_Handler` shifted to address `0x0`.
- Running `make run` fails immediately with `qemu: fatal: Lockup ...`

This lockup makes perfect physical sense. At reset, the CPU strictly reads words 0 and 1 to use as SP and PC. However, because the table was discarded, those words are now the first machine instructions of `Reset_Handler`. The CPU attempts to use instructions as memory addresses, jumps into nonsense, and fatally locks up. Putting `KEEP` back restores the table and fixes the boot process.

2.3.2 Back to the Base: Why Only One Line Prints

With the memory layout and hardware contract fully mapped, we can finally explain the behavior of the base skeleton observed at the beginning of the chapter.

If you run the base code:

make –B
make run

The `main()` function attempts to print two strings:

1. The first is a standard string literal: it lives in `.rodata`, folded inside `.text` in FLASH. The CPU reads it directly from FLASH, and it prints correctly.
2. The second is `data_message`, an initialized global array. As confirmed by `objdump -h`, the `.data` section has its VMA at `0x20000000` in RAM, but its initial bytes are stored at a FLASH LMA.

When `main()` executes, it attempts to read the `data_message` string at its VMA in RAM. However, in the base skeleton, `Reset_Handler` in `base/boot/startup.s` contains only a `bl main` instruction and an infinite loop. **No code moves the bytes from the LMA to the VMA.**

Consequently, the RAM at `0x20000000` still holds whatever it held at power-on, which in the QEMU emulator happens to be all zeros. A string whose first byte is zero is read by C as the empty string. Thus, the second call prints absolutely nothing.

Writing the assembly loop to perform this missing memory copy is where Unit 2 officially begins.

2.4 Taking Control of the CPU

In this unit, you rebuild `Reset_Handler` in `boot/startup.s` yourself, in assembly, one step at a time. To do that, you need two things you have not used before: a debugger to *see* what the CPU does, and enough ARM assembly to read and write the boot code.

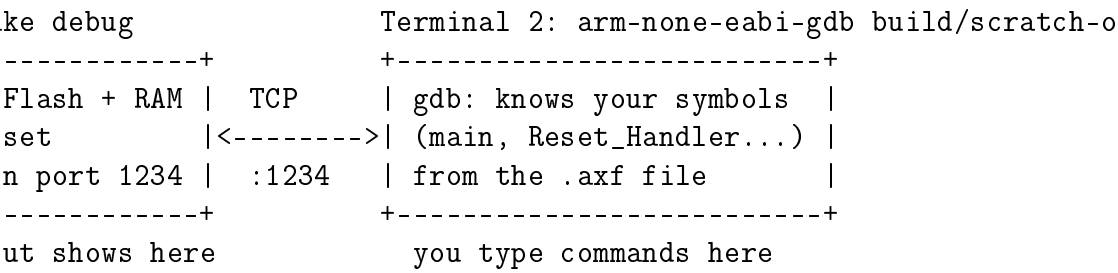
2.4.1 What is a Debugger?

A **debugger** is a program that controls another program while it runs. It can **stop** the program at any instruction you choose (a **breakpoint**) or right at the start, before anything has run. It can **step** it forward one machine instruction, or one C line, at a time. While the program is stopped, it can **inspect** the CPU registers and any memory address, and even change them. And it can **continue** the program until the next breakpoint.

On a desktop, a crashing program at least prints an error message. On a microcontroller, there is often no console at all, and a crash simply looks like "nothing happens". A debugger is frequently the *only* way to see what the CPU is really doing. The standard tool is **gdb**, the GNU debugger, part of the same toolchain family as gcc. The ARM version you use is `arm-none-eabi-gdb`; on Linux, the gdb that understands ARM is usually called `gdb-multiarch` instead.

2.4.2 A Remote Debugger: Two Programs, Two Terminals

In this setup, the program being debugged does not run on the same "machine" as gdb. It runs inside QEMU, which emulates the Cortex-M3 board on your laptop. gdb talks to QEMU over a network connection (a TCP socket on your own machine). So you use two terminals:



In terminal 1, you start QEMU with `make debug`. The `debug` target in the `Makefile` runs the same QEMU command as `make run`, plus two options: `-S`, which freezes the emulated CPU before its very first instruction, and `-gdb tcp::1234`, which starts a gdb server that waits for a connection on TCP port 1234. Anything the program prints still appears in this terminal.

Reconstructing the Debug Target

The `debug` target is therefore nothing more than a shorthand, and you can reconstruct it for any image. Take the QEMU command from the `run` target of the `Makefile` you are working with, and append `-S -gdb tcp::1234`:

```
qemu-system-arm -machine mps2-an385 -monitor null -semihosting \
    -semihosting-config enable=on,target=native \
    -kernel build/scratch-os.axf -serial stdio -nographic \
    -S -gdb tcp::1234
```

This matters after this week: the later checkpoints' Makefiles have a `run` target but no `debug` target, and this command, run from the checkpoint's folder after `make`, is how you debug them.

In terminal 2, you start gdb and give it the ELF file, `arm-none-eabi-gdb build/scratch-os.axf`. gdb reads the ELF only for its **symbols** and debug information, so that it knows the addresses of `main`, `Reset_Handler`, `data_message`, and so on. It does not load the code: QEMU already holds the program. You then connect with `target remote :1234`.

The GDB Commands You Need

This small set of commands is enough for everything this week:

Command	What it does
`target remote :1234`	connect to the gdb server (QEMU) on this machine
`info registers sp pc`	show some registers (`info registers` alone: all of them)
`x/2wx 0`	**e**xamine memory: 2 **w**ords (32 bits each) in he**x**, at address 0
`x/s &data_message`	examine memory as a C **s**tring
`p/x $r1`	**p**rint one register (or a C expression) in hex
`display/i $pc`	after every step, show the next **i**nstruction
`stepi` (`si`)	execute exactly one machine instruction (pressing Enter repeats it)
`break main` (`b`)	stop when execution reaches `main` (or any label)
`continue` (`c`)	run until the next breakpoint; `Ctrl-C` interrupts it
`quit` (`q`)	leave gdb (QEMU keeps running: stop it with `Ctrl-C`)

2.4.3 Attaching at Reset

The first thing to do with the debugger is to catch the CPU *before* its first instruction, and check the reset contract from Unit 1. In the base folder:

```
make debug                                  # terminal 1: QEMU waits
arm-none-eabi-gdb build/scratch-os.axf      # terminal 2
```

and in gdb:

```
(gdb) target remote :1234
(gdb) info registers sp pc
(gdb) x/2wx 0
```

You see `sp` equal to `0x20400000`, `pc` equal to `0x8 <Reset_Handler>`, and at address 0 the two words `0x20400000 0x00000009`. **No instruction has run yet**, yet SP and PC already have values: the CPU itself copied vector word 0 into SP and vector word 1 into PC.

Compare the two: word 1 is `0x9`, PC is `0x8`. This is the Thumb bit from Unit 1. The CPU uses bit 0 to check that the target is Thumb code and then clears it; the instruction really starts at `0x8`. And why `0x8` in the first place? The base's vector table has only two words (8 bytes), and the code is placed right after it in FLASH.

2.4.4 Cortex-M3 Programmer's Model: The Registers

To follow the CPU instruction by instruction, you need to know what it holds. The **programmer's model** is the set of registers your code, and gdb, can see. All of them are 32 bits wide.

Register	Name	Role
R0–R12	general purpose	your values; a C function call passes its first arguments in R0–R3
R13	**SP**	stack pointer: address of the top of the stack
R14	**LR**	link register: where to return after a function call
R15	**PC**	program counter: address of the next instruction
—	**xPSR**	status: flags, Thumb bit, current exception

At this level, there are no variables: only these registers and memory. When your C code says `x = x + 1`, the compiler turns it into "load `x` from memory into a register, add 1 to the register, store the register back". The stack, as you know, grows **downwards**: pushing a value first does `SP = SP - 4`, then writes the value at the new SP.

The Status Register (xPSR) and CPU Modes

xPSR (program status register) packs three critical pieces of information into one register:

- **Flags (Bits 31–28):** The **N Z C V** flags (negative, zero, carry, overflow), set by comparison instructions and used by conditional jumps.
- **Thumb Bit (Bit 24):** The **T** state bit, which must always be 1 on Cortex-M (this is where bit 0 of the vector entry ends up).
- **Exception Number (Bits 8–0):** Holds the number of the exception currently being handled, or 0 if none.

That last field dictates which of the two **modes** the CPU is in:

- **Thread mode:** The CPU runs normal code (`Reset_Handler`, `main()`, and the functions they call); the exception number is 0.
- **Handler mode:** The CPU runs an exception handler, and the field holds that exception's number.

Note: Code in Thread mode can also be privileged or unprivileged; everything in this specific unit runs privileged, and Unit 3 explains the difference.

2.4.5 ARM Assembly Basics: A Load/Store Architecture

The boot code has to be written in assembly because, before it runs, there is no valid stack and no initialized data, so C code cannot yet be trusted. You need only a handful of instructions, and each has a direct C equivalent.

A load/store machine: ARM arithmetic works only on registers. Memory is touched only by **load** instructions (`ldr`, memory to register) and **store** instructions (`str`, register to memory).

The square brackets mean "the memory at this address". The form `[r0], #4` is called **post-increment**: the instruction first uses the address in `r0`, then adds 4 to `r0`. One word is 4 bytes, so a loop using this syntax walks through memory word by word, exactly like `*dst++` in C.

Assembly	C equivalent
`ldr r3, [r0]`	`r3 = *r0;`
`str r3, [r1]`	`*r1 = r3;`
`ldr r3, [r0], #4`	`r3 = *r0; r0 += 4;` (that is, `r3 = *src++;`)
`str r3, [r1], #4`	`*r1 = r3; r1 += 4;` (that is, `*dst++ = r3;`)
`strb r1, [r0, #7]`	`((uint8_t *)r0)[7] = r1;` (one **b**yte, at offset 7)

Constants, Labels, and Jumps

Moving data between registers and controlling program flow requires another set of basic instructions:

Assembly	C equivalent
`ldr r0, =_estack`	`r0 = (uint32_t)&_estack;` (any 32-bit value)
`mov sp, r0`	`sp = r0;` (register to register)
`movs r3, #0`	`r3 = 0;` (small constants only)
`copy_data:`	a **label**: a name for this address, like a C `goto` label
`b copy_data`	`goto copy_data;`
`bl main`	`main();`: **b**ranch with **l**ink, saves the return address in LR

The 32-bit Instruction Encoding Constraint (Literal Pools): The `ldr rX, =value` form needs a word of explanation because you will see it look different in gdb. An instruction is only 16 or 32 bits long, so it mathematically cannot contain an arbitrary 32-bit constant alongside its opcode.

The assembler therefore stores the constant in a small table of constants near the code, called the **literal pool**, and turns your `ldr r0, =_estack` into "load r0 from that table". In gdb, this appears as something like `ldr r0, [pc, #92]` (an address relative to the PC).

The `.ltorg` directive at the end of `Reset_Handler` in `boot/startup.s` tells the assembler where to put this pool: typically after the last instruction, where the CPU will never try to execute it as code.

Control Flow: Decisions, Bits, and Interrupts

Branching and loops in assembly rely entirely on the `xPSR` status register. A decision is a two-step process: compare values to set condition flags, then jump based on those flags.

Assembly	C equivalent
`cmp r1, r2`	compute `r1 - r2`, keep only the flags N Z C V
`bhs done`	`if (r1 >= r2) goto done;` (**h**igher or **s**ame, unsigned)
`orr r1, r1, #0x10`	`r1 |= 0x10;`
`cpsid i`	stop accepting interrupts (`__disable_irq()` in vendor C code)
`cpsie i`	accept interrupts again (`__enable_irq()`)

With `cmp` and a conditional branch, you can build any loop. A C `while` loop becomes "test at the top, jump back at the bottom":

```
loop:   cmp r1, r2          @ while (r1 < r2) {
```

```
        bhs  done           @
        ...                 @       body
        b    loop           @ }
done:
```

The loop continues while `r1 < r2`; as soon as `r1 >= r2`, `bhs` jumps out of the loop.

Assembler Directives

Lines that start with a dot (`.`) produce no instructions: they steer the assembler itself. The directives used in `boot/startup.s` are:

```
    .section .isr_vector,"a",%progbits    @ put what follows in section .i
    .global isr_vector                    @ make the name visible to the l
isr_vector:
    .word _estack                         @ one 32-bit value
    .word Reset_Handler

    .section .text,"ax",%progbits
    .global Reset_Handler
    .thumb_func                           @ next label is Thumb code: its
Reset_Handler:
```

The vector table block translates to the following C equivalent:

```
__attribute__((section(".isr_vector")))
const uint32_t isr_vector[] = { (uint32_t)&_estack, (uint32_t)Reset_Hand
```

- `.section`: Chooses the section, with flags. `"a"` means the section occupies space in the image; `"x"` means it is executable.
- `.global`: Makes a name visible to other files and to the linker. Without it, a label acts like a `static` name in C.
- `.word`: Allocates exactly one 32-bit word and stores the given value inside it.
- `.thumb_func`: Dictates that the next label is Thumb code. This instructs the tools to store `Reset_Handler` with bit 0 set to 1, which is where the `0x9` in the base (and `0x41` in the solution) comes from.

The file also uses `.type` and `.size` to tell `gdb` and `nm` whether a name is code or data and its total size.

2.5 Writing the Startup Script (`Reset_Handler`)

The `Reset_Handler` in `boot/startup.s` has one critical job: to turn a bare CPU into an environment that can safely run C code.

Here is the complete sequence of operations, in the exact order the boot code executes them:

In the base skeleton, `Reset_Handler` has only steps 6 and 7 (and without the unmask). The following sections go through building the remaining steps in order.

Step	Why
1. Mask interrupts	Nothing may interrupt a half-built system
2. Set SP	C functions inherently need a stack
3. Copy `.data` FLASH → RAM	Initialized global variables must get their values
4. Zero `.bss`	C promises uninitialized global variables start at 0
5. Configure the CPU	Set up vector table address, fault handlers, traps, priorities
6. Unmask interrupts, `bl main`	Hand execution over to C
7. Loop forever	If `main` ever returns, there is nowhere to go

2.5.1 Steps 1 and 2: Mask Interrupts and Set the Stack

An **interrupt** is an exception raised by hardware (e.g., a timer expiring or a device receiving data): the CPU drops what it is doing and runs that interrupt's handler. During setup, absolutely nothing is ready to handle one: a handler written in C could attempt to read a variable that has not been initialized yet.

Therefore, the very first instruction, `cpsid i`, tells the CPU to explicitly ignore interrupts for now. (While no device has been told to raise an interrupt yet, writing it makes the rule explicit instead of relying on luck).

Next, the stack pointer (`SP`) must be set:

```
@ C equivalent: sp = &_estack;
ldr r0, =_estack     @ top of RAM, 0x20400000
mov sp, r0
```

Because C has no way to name SP directly, the C equivalent is pseudo-code. One might wonder why this step is strictly needed, since the hardware already loaded SP from word 0 of the vector table at reset. The answer is that `Reset_Handler` is not *always* reached through a real hardware reset: a custom bootloader or a debugger may jump to it directly, with SP holding an arbitrary or invalid value.

Setting SP explicitly makes the code robust and correct in every case, and demonstrates that the stack is nothing more than an address in a register. Notice the two-step ARM pattern: because SP is a special register, the address is first loaded into an ordinary general-purpose register (`r0`), and then explicitly copied into SP using `mov`.

2.5.2 Step 3: Copy `.data` from FLASH to RAM

This is the critical step that fixes the base skeleton's missing print line. The linker script gives you three key addresses:

```
FLASH:  [ .text | .rodata | initial values of .data ]
                             ^ _sidata
RAM:    [ .data                | .bss | ...        stack ]
          ^ _sdata             ^ _edata                  ^ _estack
```

The copy operation is a simple loop, which you can write first in C and then translate line by line into assembly:

Register `r0` plays the role of `src`, `r1` of `dst`, and `r2` holds the end address. The loop is named `copy_data` in `boot/startup.s`, and it exits to `copy_data_done`. It copies whole words, 4 bytes at a time; this is perfectly safe because the linker script intentionally aligns the start and end of `.data` to multiples of 4.

C Equivalent	ARM Assembly

`uint32_t *src = &_sidata;`
`uint32_t *dst = &_sdata;`
	`ldr r0, =_sidata`
	`ldr r1, =_sdata`
	`ldr r2, =_edata`

`while (dst < &_edata)`	`copy_data: cmp r1, r2` `bhs copy_data_done`

`*dst++ = *src++;`	`ldr r3, [r0], #4`
	`str r3, [r1], #4`
	`b copy_data`

	`copy_data_done:`

If you write steps 1–3 into the base skeleton (just before `bl main`) and run it, the missing line finally appears:

```
make run
main: hello from the base
main: this text lives in .data
```

Before the copy, `data_message` sat in RAM as all zeros and printed as an empty string; now it successfully holds its initial value.

Watching the Copy Loop in GDB

It is worth watching your loop run because it makes the abstract ideas of LMA and VMA concrete. Start `make debug` in terminal 1 and gdb in terminal 2, then:

```
(gdb) target remote :1234
(gdb) display/i $pc
(gdb) stepi                          # press Enter to repeat, until "cmp r1,
(gdb) p/x $r0                        # src: a FLASH address
(gdb) p/x $r1                        # dst: 0x20000000, start of RAM
(gdb) x/s &data_message              # "" : nothing copied yet
(gdb) stepi 5                        # one full turn of the loop
(gdb) p/x $r1                        # 0x20000004
(gdb) break copy_data_done
(gdb) continue
(gdb) x/s &data_message              # "main: this text lives in .data\n"
```

As you step, `display/i $pc` shows each instruction just before it runs: `cpsid i`, then the `ldr` from the literal pool, then `mov sp, r0`, then the three `ldr` commands that load the loop bounds.

When you reach `cmp r1, r2`, `r0` holds a small flash address (e.g., `0x88`, depending on how much code precedes it), and `r1` holds `0x20000000`. `x/s &data_message` shows the empty string because nothing has been copied yet.

Five more `stepi` commands execute one full turn of the loop (`cmp`, `bhs`, `ldr`, `str`, `b`), and `r1` moves to `0x20000004` (one word copied). Instead of stepping 8 times more, you put a breakpoint on the label after the loop and continue. `r1` climbs until it equals `r2` (`&_edata`, which is `0x20000020`), and only then does `x/s &data_message` show the full string perfectly mapped in RAM.

Try It: Breaking the Stack on Purpose

Seeing a wrong SP fail is the best way to believe that SP matters. In `boot/startup.s`, inside `Reset_Handler`, find the line you wrote in Step 2 (`ldr r0, =_estack`) and intentionally replace `_estack` with a bad address. (Note that the value the CPU loaded from vector word 0 at reset is still the correct one; it is your `mov sp, r0` that forcefully overwrites it with the bad address).

Test 1: Address Outside of RAM (`0x30000000`) If you force SP to `0x30000000` and run `make run`, nothing is printed at all. QEMU stops with:

```
qemu: fatal: Lockup: can't escalate 3 to HardFault (current priority -1)
```

Here is what happened: the first thing `main` does is push registers onto the stack, writing below `0x30000000`, where there is no physical memory. That access triggers a fault. The CPU tries to report the fault by executing a fault handler, but entering a handler inherently requires writing to the stack, which fails again. With no way to report the fault, the CPU gives up and enters a state called **lockup**.

Test 2: Unaligned Stack Pointer (`0x20000001`) If you force SP to an odd address inside RAM, `make run` prints nothing. To see why, start `make debug`, stop right after your `mov sp, r0`, and check the register: `info registers sp` returns `0x20000000`.

The CPU silently drops the two lowest bits of SP because the stack pointer must always be a multiple of 4. The stack now inadvertently starts at the very bottom of RAM and grows *below* it. At `break main`, SP is already under `0x20000000` (e.g., `0x1ffffff0`), in an address range where QEMU simply ignores writes. There is no explicit crash, just no output at all because the function calls fail silently. (Remember to restore `ldr r0, =_estack` before continuing).

2.5.3 Step 4: Zero the `.bss` Section

The C standard strictly promises that global variables without an explicit initializer (e.g., `int counter;`) will start at `0`[cite: 46]. They physically reside in the `.bss` section, mapped between `_sbss` and `_ebss` in RAM[cite: 46]. Because nothing in flash holds their zeroed value, the boot code must explicitly write these zeros[cite: 46].

This is the exact same looping logic as the data copy, but without a source pointer reading from flash[cite: 46]:

C Equivalent	ARM Assembly

`uint32_t *dst = &_sbss;`	`ldr r1, =_sbss`
	`ldr r2, =_ebss`
	`movs r3, #0`

`while (dst < &_ebss)`	`zero_bss: cmp r1, r2` `bhs zero_bss_done`
`*dst++ = 0;`	`str r3, [r1], #4` `b zero_bss`
	`zero_bss_done:`

This loop is labeled `zero_bss` in `boot/startup.s`[cite: 46]. In the current base skeleton, `.bss` happens to be empty (so `_sbss` equals `_ebss`), meaning the loop exits immediately[cite: 46]. However, the first uninitialized global you add (like `int total;`) will absolutely rely on it[cite: 46].

Warning: Remember that RAM is *not* zeroed at power-on on real hardware[cite: 46]. While the QEMU emulator zeroing RAM for you is a convenience, you must not architecturally count on it[cite: 46].

With steps 1 through 4 completed, `Reset_Handler` replicates everything that the previous C startup code did[cite: 46]. The remaining steps go further: they configure the CPU so that errors are caught, escalated, and reported[cite: 46].

2.6 Exceptions and Faults: One Family

You have already encountered the concept of an **exception**: any hardware or software event that forces the CPU to stop executing the current code and run a specific handler function[cite: 46]. The CPU fetches the address of this handler from the vector table and executes it in Handler mode[cite: 46].

The Cortex-M3 physically defines the following family of exceptions[cite: 46]:

#	**Exception**	**Raised by**
0	*(not an exception)*	This slot holds the initial SP value
1	Reset	Power-on, reset button
2	NMI	Non-maskable interrupt: cannot be ignored
3	**HardFault**	Any fault with nowhere else to go
4	**MemManage**	Breaking memory protection rules
5	**BusFault**	Access to an address where nothing answers, or that refuses access
6	**UsageFault**	Bad use of an instruction: undefined opcode, Thumb bit 0, divide by 0
7–10	*reserved*	Unused: the slot must hold `0`
11	SVCall	The `svc` instruction
12	DebugMonitor	Debug events (unused here)
13	*reserved*	Unused: the slot must hold `0`
14	PendSV	A request raised by software
15	SysTick	The CPU's built-in timer
16+	IRQs	External device interrupts

Slots 7–10 and 13 are strictly reserved[cite: 46]. The four specific faults (3 through 6) are the CPU's direct way of signaling that "the program did something wrong"[cite: 46].

Crucially, at power-on reset, only the `HardFault` exception is actively listening[cite: 46]. The three more specific faults (`MemManage`, `BusFault`, `UsageFault`) are disabled by default[cite: 46]. When a specific fault is disabled, it *escalates* directly to a `HardFault`[cite: 46]. This escalation mechanism explains the "can't escalate 3 to HardFault" lockup message we saw earlier: the CPU was trying to escalate to exception 3 (`HardFault`), but because the handler could not safely push to the stack, it failed[cite: 46].

2.6.1 Completing the Vector Table with Weak Symbols

The base table only has 2 words defined, but the Cortex-M3 expects one word for each exception from 0 to 15 (16 words in total)[cite: 46]. Every slot strictly needs *some* valid address, even for exceptions you have no custom handler for yet, because if that hardware exception triggers, the CPU will blindly jump to whatever address the slot contains[cite: 46]. (The reserved slots must contain `0`)[cite: 46].

Therefore, `isr_vector` in `boot/startup.s` gets 14 additional `.word` entries[cite: 46]. Instead of writing a dozen empty functions for every single exception, the standard engineering practice is to route them all to a generic default loop using weak symbols and aliases in assembly[cite: 47]:

```
    .word HardFault_Handler                 @ slot 3, and so on up to
    .weak      HardFault_Handler            @ may be replaced from C
    .thumb_set HardFault_Handler, Default_Handler
Default_Handler:
    b Default_Handler                       @ for (;;) {}
```

Which translates to the following compiler directive in C[cite: 47]:

```
void HardFault_Handler(void) __attribute__((weak, alias("Default_Handler"
```

How it works:

- `.word HardFault_Handler` places the address of the label into the exact table slot[cite: 47].
- `.thumb_set` dictates to the assembler: "Make `HardFault_Handler` another name for `Default_Handler` (sharing the same address, marked as Thumb code)"[cite: 47].
- `.weak` makes that name a **weak symbol**: a default implementation that easily yields[cite: 47]. If any other file (like `src/main.c`) explicitly defines a normal, non-weak function named `void HardFault_Handler(void)`, the linker silently drops the weak assembly default and routes the hardware exception directly to the developer's custom C function[cite: 47].

`Default_Handler` itself is simply an infinite loop[cite: 47]. An unexpected exception is a critical system bug, and freezing the CPU at a known, predictable place is much easier to diagnose in a debugger than allowing it to run off into random memory addresses[cite: 47].

The Effect on Memory Layout

Completing the vector table immediately alters your memory layout[cite: 47]. The table grows physically from 2 words to 16 words (from 8 bytes to 64 bytes)[cite: 47]. Because the executable code still flows right after it in FLASH, the start of `Reset_Handler` moves down from address `0x8` to `0x40`[cite: 47].

If you attach GDB at reset now, `info registers sp pc` will show `pc 0x40 <Reset_Handler>`, and examining address 0 (`x/2wx 0`) will print `0x0 <isr_vector>: 0x20400000 0x00000041` (proving the Thumb bit is added to the `0x40` address)[cite: 47].

If you examine the `nm` output of the final binary, you will clearly see the effect of the weak symbols: `NMI_Handler`, `PendSV_Handler`, `SysTick_Handler`, and `DebugMon_Handler` are explicitly marked `W` (weak) and all share the exact same memory address as `Default_Handler`[cite: 47]. Meanwhile, any handlers written and compiled from `src/main.c` are firmly marked `T` (Text) at their own unique addresses[cite: 47].

2.6.2 CPU Settings Live at Memory Addresses

The remaining configuration steps change settings inside the CPU core itself. You do not need special assembly instructions for most of them: the Cortex-M architecture makes these settings physically available as 32-bit registers at fixed memory addresses in the system area of the memory map (specifically, from `0xE000E000` to `0xE000EFFF`).

The specific group of registers that controls exceptions is called the **System Control Block (SCB)**. You can change them using ordinary load and store instructions.

C Equivalent	ARM Assembly

`#define SCB_SHCSR (*(volatile uint32_t *)0xE000ED24)`
`SCB_SHCSR |= (1u « 16);`
	`ldr r0, =0xE000ED24 @ addres`
	`ldr r1, [r0] @ read`
	`orr r1, r1, #0x10000 @ modif`
	`str r1, [r0] @ write`

The Read-Modify-Write Pattern and `volatile`

This architectural pattern is known as **read-modify-write**: you read the current value of the register, modify only the specific bits you care about using a bitwise OR, and write the result back. This ensures that all other configuration bits in the register keep their original values.

In the C version, the `volatile` keyword is mandatory. It explicitly tells the compiler that every memory access must truly happen exactly in the order written. Without it, the compiler's optimizer might skip a read (assuming "the value cannot have changed") or remove a write (assuming "nobody reads it"). For hardware registers, both assumptions are completely false.

Debugging tip: You can read these registers from gdb just like any other memory address. For example, `x/wx 0xE000ED00` shows `0x410fc231`, which is the CPUID register identifying the chip as an ARM (`0x41`) Cortex-M3 (`0xc23`).

2.6.3 Step 5: VTOR, Where the Vector Table Is

When an exception happens, the CPU does not technically read slot N strictly at address `4 * N`. It reads it at `VTOR + 4 * N`, where **VTOR** (the Vector Table Offset Register, located at `0xE000ED08`) holds the base address of the table.

Its default value at reset is `0x00000000`, which is exactly why our initial table must reside at address 0. However, the boot code explicitly writes the address of `isr_vector` into it anyway:

C Equivalent	ARM Assembly

`*(volatile uint32_t *)0xE000ED08 = (uint32_t)isr_vector;`	`ldr r0, =0xE000ED08`
	`ldr r1, =isr_vector`
	`str r1, [r0]`

(Note: The table address must be suitably aligned, at least to 128 bytes, because the low bits of VTOR are not used by the hardware).

Why perform this write if our table is already at 0? Because in real production systems, the table is often located elsewhere. A **bootloader** sitting at address 0 may start an application stored higher in the flash memory, whose own vector table is at a different address. The application must then point VTOR to its own table. Alternatively, an OS may copy its table into RAM to replace handlers dynamically while running. Explicitly writing VTOR ensures the boot code remains correct regardless of where the final image is placed.

Where you write it: In `Reset_Handler`, right after the label `zero_bss_done:` (the end of step 4). Because you are setting the entire register to a new absolute address, no read-modify-write is needed. In gdb at `main`, `x/wx 0xE000ED08` will show `0x00000000`: the same as the reset value, confirming the table is at address 0 (verified by `p/x &isr_vector`).

2.6.4 Step 6: SHCSR, Separate Fault Handlers

The **SHCSR** (System Handler Control and State Register) at address `0xE000ED24` enables the three specific hardware faults:

Bit	Name	When set
16	MEMFAULTENA	MemManage faults run `MemManage_Handler`
17	BUSFAULTENA	Bus faults run `BusFault_Handler`
18	USGFAULTENA	Usage faults run `UsageFault_Handler`

At reset, these bits are heavily restricted (set to 0), meaning all faults instantly escalate to a `HardFault`. This gives you a single handler for every kind of error, making it impossible to tell *what* specifically went wrong inside it.

The boot code must set bits 16–18 simultaneously with a bitwise OR (`|= 0x00070000`), ensuring each fault routes cleanly to its own dedicated handler. Because the other bits of SHCSR report critical system status (like which handlers are active or pending), you **must** use the read-modify-write pattern and never simply overwrite the whole register.

Where you write it: Right after step 5. You implement the read-modify-write pattern: `ldr r0, =0xE000ED24`, then `ldr r1, [r0]` (read), `orr r1, r1, #0x00070000` (modify), and finally `str r1, [r0]` (write). In gdb at `main`, checking `x/wx 0xE000ED24` will correctly display `0x00070000`.

2.6.5 Step 7: CCR, What Counts as a Fault

CCR, the Configuration and Control Register physically located at `0xE000ED14`, dictates whether certain suspicious CPU operations are classified as faults at all.

Bit	Name	Meaning	Here
3	UNALIGN_TRP	Trap unaligned word/halfword accesses	left off
4	DIV_0_TRP	Trap integer divide by 0 (as a UsageFault)	**set**
9	STKALIGN	8-byte-align the stack on exception entry	already 1

Without the `DIV_0_TRP` bit set, an integer division by zero on the Cortex-M3 does not inherently trigger a fault: the hardware divide instruction silently returns **0**. The program would continue running with a mathematically incorrect value, and nobody would notice. Setting bit 4 transforms that silent, catastrophic bug into an immediate `UsageFault`.

Unaligned memory accesses, on the other hand, are strictly legal on the Cortex-M3 (though slightly slower). While trapping them can help identify sloppy pointer casts, it can also inadvertently break correct, optimized code produced by the C compiler or standard libraries. Therefore, it is intentionally left off. Bit 9 (`STKALIGN`) is already set to 1 at hardware reset, and the read-modify-write pattern safely preserves it.

Where you write it: Right after step 6. The implementation requires the same four read-modify-write instructions: the address becomes `0xE000ED14` and the bitwise OR utilizes `#(1 « 4)`. The C equivalent is `*(volatile uint32_t *)0xE000ED14 |= (1u « 4);`. Here, read-modify-write is absolutely critical because bit 9 (`STKALIGN`) is already 1 and must remain 1. In gdb at `main`, checking `x/wx 0xE000ED14` will successfully show `0x00000210`.

2.6.6 Step 8: Exception Priorities

When two exceptions compete simultaneously, the more urgent one executes first. An urgent exception can even pre-empt and interrupt a handler that is already currently running for a less urgent one.

On the Cortex-M architecture, **a lower number means a more urgent priority**. Reset (-3), NMI (-2), and HardFault (-1) possess fixed, immutable priorities that are mathematically more urgent than anything a developer can configure. (This finally explains the lockup message encountered earlier: "current priority -1" means the CPU was already executing at the HardFault level, meaning there was physically nowhere higher left to escalate).

Every other exception has a configurable priority, stored as a single byte in the **SHPR** (System Handler Priority Registers):

Register	Byte 0	Byte 1	Byte 2	Byte 3
SHPR1 (`0xE000ED18`)	MemManage	BusFault	UsageFault	—
SHPR2 (`0xE000ED1C`)	—	—	—	SVCall
SHPR3 (`0xE000ED20`)	—	—	PendSV	SysTick

Because each priority is strictly a byte of its own, the boot code writes it directly using a byte store instruction (`strb`). This entirely bypasses the need for the read-modify-write pattern:

The architectural choice made in `Reset_Handler` is structured as follows: faults are the most urgent (`0x00`), SVCall sits in the middle (`0x80`), and PendSV alongside SysTick are the least urgent (`0xE0`). Real physical chips typically implement only

C Equivalent	ARM Assembly

`uint8_t *prio = (uint8_t *)0xE000ED18;`
`movs r1, #0x00`
`prio[1] = 0x00; /* BusFault:  urgent */`
	`ldr r0, =0xE000ED18`
	`strb r1, [r0, #1]`

the **upper** 3 or 4 bits of each priority byte (the lower bits always read as 0), so you must purposefully choose hexadecimal values that differ in their top bits, such as these.

Where you write it: Right after step 7. Think of SHPR1, SHPR2, and SHPR3 collectively as a contiguous array of 12 bytes starting at address `0xE000ED18`.

- `prio[0]`, `prio[1]`, and `prio[2]` map to MemManage, BusFault, and UsageFault (set to `0x00`).
- `prio[7]` maps to SVCall (SHPR2 byte 3, which is $4 + 3$; set to `0x80`).
- `prio[10]` and `prio[11]` map to PendSV and SysTick (SHPR3 bytes 2 and 3; set to `0xE0`).

In assembly, you load the base address `0xE000ED18` into `r0` exactly once. Then, for each distinct value, you execute a `movs r1, #value` followed by one `strb r1, [r0, #index]` instruction per byte. In gdb at `main`, checking `x/3wx 0xE000ED18` will explicitly show `0x00000000 0x80000000 0xe0e00000`.

2.6.7 Step 9 and After: PRIMASK and Handing Over to C

The `cpsid i` and `cpsie i` instruction pair operates directly on **PRIMASK**, a special 1-bit register physically located inside the CPU core (it is not mapped in standard memory). When PRIMASK is set to 1, the CPU aggressively ignores every exception that has a configurable priority; only NMI and HardFault are allowed to pass through.

Instruction	PRIMASK State	C (vendor headers)
`cpsid i`	1: masked	`__disable_irq()`
`cpsie i`	0: accepted	`__enable_irq()`

PRIMASK defaults to `0` at reset. The boot code deliberately sets it at the very beginning (Step 1) and meticulously clears it just before calling `bl main`, ensuring that the vector table, handlers, and exception priorities are entirely configured before any interrupts are permitted.

Interrupts that arrive while the system is masked are not discarded or lost: the hardware registers them as **pending**, forcing them to wait, and immediately executes them as soon as PRIMASK is cleared. You can observe the PRIMASK state in gdb by typing `info registers primask`.

After `cpsie i` comes the `bl main` instruction, seamlessly handing execution over to your C code. Following this, the assembly file implements a `hang` label with a `b hang` instruction. This is the architectural equivalent of an infinite `for (;;) {}` loop: on a bare-metal machine, if `main` ever inadvertently returns, there is physically nowhere to return to, so the CPU must spin safely forever.

Where you write it: `cpsid i` is already present as Step 1. Step 9 is a single `cpsie i` instruction, inserted right after Step 8 and just before `bl main`. In gdb, inspecting `info registers primask` will correctly show `0` at reset, `1` after a single `stepi` (past `cpsid i`), and `0` again once execution reaches `main`.

2.6.8 Checking the Configuration from GDB

Once steps 4 through 9 are written, you can definitively prove that all hardware bits are correctly configured by the time `main` starts. With `make debug` running in your first terminal, execute the following in GDB:

```
(gdb) target remote :1234
(gdb) info registers primask      # 0 at reset; after stepping past cpsid i
(gdb) break main
(gdb) continue
(gdb) x/wx 0xE000ED08             # VTOR
(gdb) x/wx 0xE000ED24             # SHCSR
(gdb) x/wx 0xE000ED14             # CCR
(gdb) x/3wx 0xE000ED18            # SHPR1..3
```

The expected memory values are precisely as follows:

- **VTOR:** `0x00000000` (unchanged, meaning the table is actively at address 0).
- **SHCSR:** `0x00070000`, which exactly corresponds to bits 16, 17, and 18 being active.
- **CCR:** `0x00000210`, which is bit 4 (`0x10`, the division-by-zero trap you added) plus bit 9 (`0x200`, the stack alignment bit already set at reset).
- **SHPR:** `0x00000000 0x80000000 0xe0e00000`.

Reading the SHPR byte by byte: Because the Cortex-M architecture is little-endian, byte 0 is the rightmost pair of hex digits.

- **SHPR1** is all zeros (the three specific faults are set to the highest configurable priority).
- **SHPR2** has `0x80` strictly in byte 3 (corresponding to SVCall).
- **SHPR3** has `0xe0` strictly in bytes 2 and 3 (corresponding to PendSV and SysTick, the lowest priorities).

2.6.9 Catching a Divide by Zero

The configuration performed in the `Reset_Handler` is only useful if it successfully catches real errors in the code. To prove this, we can write custom fault handlers in C and intentionally trigger a fault.

In `src/main.c` (right after `data_message` and before `main`), you can write four functions utilizing the exact names defined in the vector table: `HardFault_Handler`, `MemManage_Handler`, `BusFault_Handler`, and `UsageFault_Handler`. Each function prints its own name using the same `semihost_write0` function that `main` uses, and then enters an infinite loop:

```
void UsageFault_Handler(void)
{
    semihost_write0("UsageFault_Handler: a usage fault happened\n");
    for (;;) { }
}
```

Because these are normal (strong) C definitions, the linker uses them to silently replace the weak assembly defaults established in `boot/startup.s`. They loop forever because a fault represents a critical bug; there is no sensible way to continue execution safely.

Triggering the Fault

To test the handlers, we can force `main` to divide by zero. Crucially, the divisor must be a `volatile` variable holding `0`: without `volatile`, the compiler's optimizer will detect the constant division, compute or remove it entirely at compile time, and no physical divide instruction will ever be executed by the CPU.

Using a preprocessor switch allows us to toggle the demo safely:

```
#define DEMO_DIVIDE_BY_ZERO 1

/* Inside main() */
#if DEMO_DIVIDE_BY_ZERO
    volatile uint32_t numerator = 42;
    volatile uint32_t zero = 0;
    volatile uint32_t result = numerator / zero;
    (void)result; /* Suppresses the 'unused variable' compiler warning */
#endif
```

Running `make run` will successfully halt the program and print:

```
UsageFault_Handler: a usage fault happened
```

Validating the Configuration Steps

The two hardware configuration steps we added each play a distinct role, which can be verified by disabling them one at a time:

- **Without Step 6 (SHCSR bits):** The specific usage fault is disabled. The hardware escalates the error, and you see `HardFault_Handler: a hard fault happened`. The critical information regarding *what* specific kind of fault occurred is permanently lost.
- **Without Step 7 (DIV_0_TRP):** You see no fault message at all. The division silently returns `0`, and `main` continues executing its usual lines as if nothing happened. This is precisely the silent, catastrophic bug the trap was configured to catch.

2.6.10 What Real Chips Add (`SystemInit`)

The QEMU emulator's CPU is ready to run at full speed the moment it leaves reset. However, on a real physical microcontroller, the startup code usually must

perform significantly more initialization before `main` is called. This is typically done in a vendor-supplied function called `SystemInit()`, which is called directly from `Reset_Handler`.

Even though our emulator does not strictly require them, you should understand what a standard `SystemInit()` typically handles:

- **Clock and PLL:** The chip naturally starts on a slow internal oscillator for stability. A Phase-Locked Loop (PLL) circuit must be configured to multiply that frequency up to the chip's maximum operational speed.
- **Flash Wait States:** Physical Flash memory is fundamentally slower than a high-speed CPU. Before ramping up the clock frequency, the code must instruct the flash controller to insert extra "wait cycles" into each memory access; otherwise, the CPU would fetch invalid garbage data.
- **Watchdog Timer:** A watchdog is a hardware timer that automatically resets the chip unless the software periodically "feeds" (resets) it. This acts as a physical safeguard against a program that hangs. On some chips, the watchdog is active immediately upon reset, so the boot code must either explicitly turn it off or begin feeding it.
- **FPU (Floating-Point Unit):** On architectures like the Cortex-M4F, the FPU is disabled at reset to save power. It must be explicitly enabled via the **CPACR** register (`0xE000ED88`); otherwise, executing the very first `float` instruction will trigger a `UsageFault`.
- **C++ Static Constructors:** If writing an OS or application in C++, the startup code must explicitly iterate through the `.init_array` section and call all static object constructors before jumping to `main`.

2.7 Life Without an Operating System

Your program now boots entirely on code you explicitly wrote. This unit examines what is *not* there and the architectural consequences of those missing layers. It culminates with the program's final structural form: `main` intentionally relinquishing its privileges and then requesting privileged work back through a formal system call.

2.7.1 Hosted vs. Freestanding C Environments

The C standard strictly defines two distinct execution environments for any C program.

	Hosted	**Freestanding**
Example	Your laptop program	Our Cortex-M3 image
Under your code	A C library (**libc**) and an OS	Nothing
`printf`, `malloc`, `fopen`, `exit`	Available	Not available
Who starts `main()`	libc startup code, called by the OS	Whoever you decide

On a standard laptop, writing `printf("hi\n")` "just works" because two massive, complex layers sit directly beneath your program doing the heavy lifting: the C

library and the Operating System. Our embedded image operates in a **freestanding** environment. Below your bare-metal code, there is absolutely nothing.

What is Available in Freestanding C?

A freestanding environment still provides a subset of standard headers. However, these are strictly headers that require **no code execution at run time**. They provide only data types, architectural constants, and macros that the compiler resolves entirely at build time.

Header	Provides
`<stdint.h>`	`uint32_t`, `int8_t`, ...
`<stddef.h>`	`size_t`, `NULL`, `offsetof`
`<stdbool.h>`	`bool`, `true`, `false`
`<stdarg.h>`	`va_list`, for variadic functions
`<limits.h>`, `<float.h>`	`INT_MAX`, `FLT_EPSILON`, ...

This is precisely why `src/main.c` only includes `<stdint.h>`. There is no `<stdio.h>`, no `<stdlib.h>`, and no `<string.h>`: this means no printing, no dynamic allocation (`malloc`), no file I/O, no `exit`, and not even `memcpy`. Anything else required by the system, you must write yourself. (This lack of standard library support is the exact reason the assembly boot code had to manually copy the `.data` section with its own explicit loop rather than just calling a convenient `memcpy`).

2.7.2 What is Really Under `printf`

To understand why standard functions fail without an OS, it helps to dissect what `printf` actually does under the hood on a hosted system (like a laptop):

```
printf("x=%d\n", x)           your program
   |  format the text into a buffer          ]   v
| libc
write(1, buf, len)                            ]
   |  system call: the CPU enters the kernel   ]
   v                                           | OS kernel
terminal / file / serial device driver         ]
   |
   v
hardware (screen, UART, disk)
```

`printf` itself only performs **formatting**: it takes a format string like `"x=%d\n"` and the value of `x`, and parses them into a raw sequence of ASCII characters stored in a memory buffer.

However, getting those physical characters out of the machine to a screen is not `printf`'s job. To do that, it calls `write()`, which triggers a **system call**. A system call forces the CPU to enter the Operating System kernel. The kernel's specific device driver then takes over, physically sending the bytes to the terminal, a file, or a serial hardware device.

`printf` is not the only C library feature that relies on hidden, lower-level OS hooks:

libc feature	What it needs underneath
`printf`, `puts`	`write()`: a way to send bytes to a physical device
`malloc`, `free`	a heap memory region, dynamically grown via `sbrk()`
`exit`	an OS process manager to return control to
`main()` being called	startup code (**crt0**): sets up the stack, `.data`, `.bss`, then calls `main`

C libraries designed for embedded systems (such as **newlib**) implement the high-level formatting logic and data structures, but they intentionally leave these bottom-level hardware functions as empty **hooks** (stubs) that the developer must manually implement for their specific physical board.

2.7.3 Why This Course Uses No libc At All

There are three fundamental reasons we deliberately avoid linking any C library at all:

1. **No Underlying Kernel Hooks:** Without an OS beneath us, nothing implements those required hooks. There is no kernel, no `write()` function, and no heap manager; if we linked libc, we would have to write these complex low-level stubs first anyway just to satisfy the linker.

2. **Redundant Startup Logic:** The job of `crt0` (the standard libc startup code that prepares the C environment) is already perfectly handled by the custom `Reset_Handler` we built in `boot/startup.s` (setting up the stack, copying `.data`, clearing `.bss`, and branching to `main`).

3. **Absolute Visibility:** The primary educational goal is to **see every single layer**. Every byte that executes on the board is code you can actively read in the repository. The final compiled image contains exclusively `boot/startup.s`, `src/main.c`, and occasionally a few necessary compiler helper routines. Absolutely nothing is hidden from the developer.

2.7.4 The Build Says It: The Flags in the `Makefile`

The `Makefile` explicitly enforces this freestanding setup using specific compilation and linking flags:

```
CFLAGS := -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding -std=gnu17
LDFLAGS += -nostdlib -lgcc
```

- `-ffreestanding`: This explicitly tells the compiler it is compiling for a freestanding environment. It must not assume that a C library or an OS exists (for example, it cannot assume that a function named `printf` behaves like the standard one).

- `-nostartfiles`: This instructs the linker *not* to link the toolchain's own default startup code (usually `crt0`). Our custom `boot/startup.s` takes its place.

- `-nostdlib`: This goes even further, instructing the linker not to link **any** standard library or startup file (in particular, no `-lc` for libc).

- `-lgcc`: This explicitly adds one specific library back: **libgcc**, which contains the compiler's own internal helper routines.

Why Add `libgcc` Back?

Sometimes, the compiler translates a standard C mathematical operator into a **call** to an internal helper function because the target CPU physically lacks a single hardware instruction for it.

C code	CPU	What GCC emits
`a / b` (32-bit)	Cortex-M0 (no divide instruction)	`bl __aeabi_idiv`
`a / b` (32-bit)	Cortex-M3	`sdiv` (a hardware instruction)
`x / y` (64-bit)	Cortex-M3	`bl __aeabi_uldivmod`

These fallback helper routines live inside `libgcc`. Critically, they need no OS and no libc to function, so keeping them costs nothing. Dropping them would cause perfectly harmless, standard C code (such as a 64-bit division on an M3) to fail at the linking stage.

Try It: The `printf` Experiment

You can tangibly see what "no libc" means by directly asking for `printf`. If you add `#include <stdio.h>` to `src/main.c` and insert a `printf("hi\n");` at the start of `main`, the build will always fail:

- With a bare toolchain (e.g., Homebrew's `arm-none-eabi-gcc`), the compiler fails immediately: `fatal error: stdio.h: No such file or directory`.
- With a toolchain that ships `newlib`, the file compiles, but the linker (due to `-nostdlib`) halts with: `undefined reference to 'printf'`.

The conclusion is identical: nobody below your code provides `printf`.

2.7.5 Semihosting: How the Program Still Prints

If there is no libc and no physical UART driver, how did our bare-metal program successfully print lines to the laptop's terminal? It relies on a hardware debug mechanism called **semihosting**.

Instead of driving real physical hardware, the embedded program asks the **host** (the computer running the emulator or the debugger) to perform the input/output operations on its behalf. It works through this precise sequence:

1. The program places an operation code into `r0`. Here, it uses `0x04`, known as `SYS_WRITE0`, meaning "print this string terminated by a zero byte". It places the memory address of the string into `r1`.
2. It executes `bkpt 0xab`: a software breakpoint instruction tagged with a special identifying number.
3. QEMU (or a physical hardware debugger) intercepts this breakpoint, halts the CPU, reads `r0` and `r1`, fetches the string directly from the target's RAM, and prints it **on your laptop's terminal**.

4. Once finished, the host commands the program to continue execution seamlessly after the `bkpt`.

This mechanism *requires* a debugger or emulator actively listening (which is why `make run` starts QEMU with the `-semihosting` flag). If flashed onto a standalone physical board with no debugger cable attached, the `bkpt` instruction would simply permanently halt the CPU.

Implementing Semihosting in C

The C implementation (found in `src/main.c`) uses Extended Inline Assembly to fulfill this hardware contract:

```
static void semihost_write0(const char *message)
{
    register uint32_t     r0 __asm__("r0") = 0x04;     /* SYS_WRITE0 */
    register const char *r1 __asm__("r1") = message; /* string address */
    __asm__ volatile ("bkpt 0xab" : : "r"(r0), "r"(r1) : "memory");
}
```

- **Register Binding:** The `register ... __asm__("rX")` syntax is a GCC extension that forces the variables directly into the exact CPU registers (`r0` and `r1`) required by the semihosting convention. Standard C normally gives you no control over register allocation.
- **The Instruction:** `__asm__ volatile` allows you to insert the raw `bkpt 0xab` assembly instruction, which has no C equivalent.
- **Memory Barrier:** The `"memory"` clobber at the end warns the compiler that this instruction may interact with RAM (the string itself), actively preventing the optimizer from caching memory reads or delaying writes before this point.

Inline Assembly Formatting Rule: C naturally concatenates adjacent string literals (e.g., `"a" "b"` becomes `"ab"`). If you write multiple assembly instructions inside a single `__asm__` block on separate lines without an explicit newline character (`\n`), the C compiler will run them together into one broken string (`ldr r2, [r0]ldr r3, [r1]`), causing the assembler to fail with `garbage following instruction`. You must explicitly separate instructions using `\n` or `\n\t`.

2.7.6 Semihosting is a Trap

Look closely at what `bkpt 0xab` really does[cite: 78, 79]. The program **stops** doing its own work; control passes to **someone more powerful** (QEMU, outside the emulated CPU); that someone performs the job and **hands control back**[cite: 78, 79]. This is the exact same architectural shape as a system call on Linux: your program cannot drive the terminal itself, so it asks the kernel to do it, and the kernel returns control when the job is done[cite: 78, 79].

That raises a natural question: can the Cortex-M3 itself be split into a powerful part and a restricted part, like the kernel mode and user mode you saw in week 1? It can[cite: 79].

2.7.7 Privileged and Unprivileged Code on Cortex-M

The Cortex-M3 implements the kernel/user mode concept directly in hardware, combined with the two CPU modes[cite: 79]:

CPU mode	**When**	**Privilege**
Handler mode	running an exception handler	always **privileged**
Thread mode	normal code (`main`, ...)	privileged **or** unprivileged

After reset, the CPU defaults to Thread mode and is **privileged**[cite: 80]. That is exactly how `Reset_Handler` was able to freely write to VTOR, SHCSR, and CCR[cite: 80]. **Unprivileged** Thread mode is the equivalent of user mode: the restricted mode for code that a kernel would not trust[cite: 80].

The CONTROL Register

Which privilege level Thread mode uses is selected by the **CONTROL** register, a special register inside the CPU (like PRIMASK, it is not mapped in standard memory)[cite: 81]:

Bit	**Name**	**Meaning**
0	**nPRIV**	0 = privileged, 1 = unprivileged (Thread mode only)
1	SPSEL	selects which stack pointer Thread mode uses (not used here)

Because special registers are not reachable with standard `ldr` and `str` instructions, two dedicated instructions are used to read and write them[cite: 81]:

Instruction	**C equivalent**
`mrs r0, control`	`r0 = CONTROL;` (read a special register)
`msr control, r0`	`CONTROL = r0;` (write a special register)

2.7.8 Dropping Privilege: A One-Way Street

In `src/main.c`, after the first two lines are printed, the program intentionally drops its own privilege using a small block of inline assembly[cite: 81]:

ARM Assembly	**C Equivalent**

`mrs r0, control`
`orr r0, r0, #1`
`msr control, r0`
`isb`
	`CONTROL = CONTROL | 1;`

This is a read-modify-write pattern, this time executed on a special register[cite: 81]. The last instruction, `isb` (Instruction Synchronization Barrier), is critical because modern CPUs fetch instructions ahead of time[cite: 81]. Some of the following instructions may have already been fetched under the old, privileged rules[cite: 81]. `isb` flushes the pipeline, throws those pre-fetched instructions away, and forces the CPU to fetch them again under the strictly enforced unprivileged rules[cite: 81].

The Privilege Trap: This transition is a **one-way street**[cite: 81]. Writing to the CONTROL register is itself a privileged operation: if unprivileged code attempts to execute `msr control, r0` to clear nPRIV, the hardware simply **ignores** the write[cite: 81]. Code cannot maliciously promote itself back to privileged execution[cite: 81, 82]. There are only two ways back: a full hardware **reset**, or **an exception** (because exception handlers automatically run in Handler mode, which is unconditionally privileged)[cite: 82].

2.7.9 What Unprivileged Code Cannot Do (The Sandbox Constraints)

Once restricted, the CPU actively blocks critical operations to create a secure sandbox[cite: 83]:

Attempt	Result
Read or write the **System Control Space** (`0xE000E000`...): SysTick, NVIC, SCB	**BusFault**
`msr control, r0` (raise privilege back)	Ignored
`cpsid i` / `cpsie i` (mask or unmask interrupts)	Ignored
Semihosting `bkpt 0xab`	QEMU refuses it, so it faults

(Note: The NVIC is the hardware block that manages device interrupts; it lives in the same restricted system area as the SCB)[cite: 83].

Ordinary RAM, flash memory, and standard code execution still work normally[cite: 83]. The restricted code simply cannot **reconfigure the machine**, and it cannot silence it by masking interrupts[cite: 83].

Try It: The HardFault Escalation

The semihosting restriction has a direct, observable consequence[cite: 83]. Right after the privilege-drop block in `main`, add a call to `semihost_write0("main: unprivileged hello\n");` and run `make run`[cite: 83]:

```
main: privileged hello
main: this text lives in .data
HardFault_Handler: a hard fault happened
```

Your line never prints[cite: 84]. QEMU strictly refuses semihosting requests from unprivileged code[cite: 84]. The `bkpt` instruction is treated as a security violation and triggers a fault[cite: 84]. Since no dedicated fault handler covers this specific trap in the current setup, it escalates to **HardFault**[cite: 84].

Notice that `HardFault_Handler` *can* print via semihosting, even though the code that caused the fault was unprivileged[cite: 84]. This is because handlers automatically run in Handler mode, regaining full privileges[cite: 84]. (Stop QEMU with `Ctrl-C` and remove the offending line).

Debugging the CONTROL Register

You can watch the `nPRIV` bit flip in the debugger[cite: 80]. With `make debug` in terminal 1, run the following in GDB:

```
(gdb) target remote :1234
(gdb) break main
(gdb) continue
(gdb) p/x $control          # 0x0: privileged
```

Then let the program run past the privilege drop (for example, by setting a breakpoint on `SVC_Handler`—introduced in the next section—and continuing)[cite: 80]. Print `$control` again: it is now `0x1`[cite: 80]. The `nPRIV` bit is set, and it securely stays set[cite: 80].

2.7.10 SVC: The Door That Stays Open

Unprivileged code is strictly blocked from touching the hardware, but it can still **ask** for tasks to be done on its behalf. The `svc #n` ("supervisor call") instruction is deliberately allowed to execute within unprivileged code.

When executed, it raises the **SVCall** exception (Exception 11), whose designated handler resides in slot 11 of the vector table: `SVC_Handler`. As with any exception, the CPU immediately transitions into Handler mode, meaning the handler itself runs fully **privileged**. When the handler function returns, the CPU drops back into Thread mode to execute the instruction right after the `svc`, **still entirely unprivileged**.

```
Thread mode, UNPRIVILEGED                    Handler mode, PRIVILEGED
-------------------------                    ------------------------

main:
  ...
  svc #0  ------ SVCall exception ------> SVC_Handler:
                                            semihost_write0("...")
                                            (can print, can touch
                                             SysTick, NVIC, SCB)
  next instruction <---- exception return ---- return
  ... (still unprivileged)
```

Hardware privilege is **borrowed** only for the strict duration of the handler, and only the specific code *you* compiled into the handler runs with it. The unprivileged code does not get to choose what the handler runs; it can only knock on the door.

The number `#n` inside `svc #n` is encoded directly into the machine instruction. Real OS kernels utilize this number (combined with arguments passed in registers) to determine *which* specific system service is being requested. For our baseline proof of concept, there is only one service, so the program simply executes `svc #0`.

2.7.11 This Is a System Call

Compare this bare-metal sequence directly with what happens on a standard desktop environment:

The architectural mechanism remains identical at every scale: the restricted side cannot safely perform the job itself, so it intentionally traps into code that can, and

	Linux on your laptop	**Our Cortex-M3**
Restricted code	your program, in user mode	`main` after `nPRIV` = 1
Trap instruction	`syscall` (x86-64), `svc` (ARM)	`svc #0`
Privileged side	the kernel	`SVC_Handler`
Service	`write()` puts bytes on the terminal	print via semihosting

that privileged code securely decides what to do. The `write()` system call located at the bottom of the libc `printf` stack on your laptop works exactly this way.

2.7.12 Adding the System Call

The final steps to turn this into working code happen in `src/main.c`.

First, you must write the `SVC_Handler`. It prints a message using `semihost_write0` and then simply returns:

```
void SVC_Handler(void)
{
    semihost_write0("SVC_Handler: privileged code printing on behalf of u
}
```

Unlike the hardware fault handlers you wrote earlier, it does *not* loop forever: an `svc` trap is a deliberate request for service, not a bug. Falling off the end of the C function triggers a clean return from the exception, dropping privileges and jumping back to the instruction after `svc #0`. Its name strongly overrides the weak default defined in `boot/startup.s`, just like the fault handlers did.

(Note: A standard C function can serve flawlessly as an exception handler because the Cortex-M hardware automatically saves and restores the registers a C function might modify upon exception entry and return).

Then, after the privilege drop in `main`, you insert the trap instruction using inline assembly:

```
__asm__ volatile ("svc #0" : : : "memory");
```

Executing `make run` now correctly prints the third line:

```
SVC_Handler: privileged code printing on behalf of unprivileged code
```

Troubleshooting note: If you forget to explicitly write the `SVC_Handler` in C, the `svc` instruction seamlessly lands in the weak `Default_Handler` defined in `boot/startup.s`, and the program hangs silently in its infinite loop (meaning the weak default successfully did its job).

2.7.13 Touching the Hardware Without Privilege

Finally, you must definitively verify that the unprivileged hardware protection is actively enforced.

After the `svc #0` line, while `main` is still unprivileged, attempt to write `0` into the SysTick timer's control register (located at address `0xE000E010` in the restricted System Control Space):

```
volatile uint32_t *systick_ctrl = (volatile uint32_t *) 0xE000E010;
*systick_ctrl = 0;
```

Running `make run` yields:

```
SVC_Handler: privileged code printing on behalf of unprivileged code
BusFault_Handler: a bus fault happened
```

The output proves that the `svc` successfully returned to unprivileged code. Then, the System Control Space hardware actively rejected the unprivileged store instruction, forcing the CPU to raise a **BusFault**. Because `Reset_Handler` enabled bus faults earlier in the SHCSR, the fault routes directly to its own specific handler, and you immediately know exactly what kind of error occurred.

Conclusion: Privilege and CPU Modes

One final observation completes the picture. In gdb, put a breakpoint on `SVC_Handler`, `continue` execution, and print `$control` inside the handler: it is still `0x1`.

The handler is privileged *not* because the `nPRIV` bit magically cleared itself, but because the CPU is currently running in **Handler mode**. Handler mode is architecturally designed to always be privileged, regardless of what the `CONTROL` register says.

2.8 Conclusion: The Final Result

With all the steps in place, the solution's `make run` prints the four lines you saw at the beginning of this chapter:

```
main: privileged hello
main: this text lives in .data
SVC_Handler: privileged code printing on behalf of unprivileged code
BusFault_Handler: a bus fault happened
```

Read them as a definitive summary of the week's architectural work:

- The **first line** shows that `Reset_Handler` successfully reached `main`, operating securely in privileged Thread mode.
- The **second line** shows that the `.data` copy loop works: the linker's LMA/VMA memory split and your assembly boot code perfectly agree.
- The **third line** shows that after intentionally dropping privilege, unprivileged code can still securely get privileged work done through `svc` and `SVC_Handler`: establishing a true system call.
- The **fourth line** shows that unprivileged code attempting to reconfigure the CPU is immediately stopped by a `BusFault`, which is then cleanly reported by its own dedicated handler thanks to the SHCSR configuration.

Note: With `DEMO_DIVIDE_BY_ZERO` in `src/main.c` set to 1, the same image instead prints `UsageFault_Handler: a usage fault happened`, proving the CCR divide-by-zero trap actively works.

Your program now runs completely **freestanding**: there is no libc, no crt0, and no underlying OS (and the `Makefile` flags state this explicitly). It prints through semihosting, trapping into a more powerful host. Crucially, it implements—in just

a few dozen lines of assembly and C—the core idea of every operating system's protection model: **restricted code cannot touch the machine directly, and must ask privileged code through a single, controlled door.**

2.9 Appendix: Where to Find the Code

The project directory (`week-02/`) holds two sibling folders: `base/` and `solution/`. Each has its own `Makefile` and builds completely independently. Unless a path in these notes is explicitly written with a `base/` prefix, files like `boot/startup.s`, `src/main.c`, `scripts/mps2_m3.ld`, and `Makefile` are relative to `week-02/solution/`.

2.9.1 The Base Skeleton

The starting point of the in-class work is `week-02/base/`:

- `base/boot/startup.s` holds only the minimal two-word vector table and a `Reset_Handler` that blindly calls `main`.
- `base/src/main.c` prints the string literal and `data_message`.
- The linker script and `Makefile` are identical to the solution.

2.9.2 The Finished Result

The fully completed bootloader is in `week-02/solution/`:

- `boot/startup.s` contains the full 16-entry vector table, the weak default handlers, and the completely manual `Reset_Handler` (handling the interrupt mask, stack pointer, `copy_data`, `zero_bss`, VTOR, SHCSR, CCR, exception priorities, and finally `bl main`).
- `src/main.c` contains `semihost_write0`, the four strong fault handlers, the `SVC_Handler`, and the `main` logic that drops privilege, calls `svc #0`, and pokes the SysTick timer to test the sandbox.
- `scripts/mps2_m3.ld` and `Makefile` enforce the freestanding memory layout mapped in Units 1 and 3.

Final note: If these lecture notes and the codebase ever disagree, the C and Assembly code is always the absolute reference.

Chapter 3

Devices, Interrupts, and System Calls

3.1 Introduction: From CPU Configuration to OS Architecture

In the first two weeks, the focus was entirely on very low-level management: configuring the bare CPU so that it could start executing code[cite: 91]. We successfully built a freestanding C program (with no `libc` and no underlying OS) that could boot the CPU and print a message to the screen[cite: 91]. However, that printing was done using the semihosting trick—it was not real hardware printing[cite: 91]. Furthermore, our program currently has no way to interact with physical devices, and nothing outside the CPU can interrupt its execution[cite: 91].

The ultimate goal of this course is to write a small operating system. Having started from the bottom by configuring the CPU, the next logical step is to learn how to use the other devices present on our board. This introduces three major additions to our codebase[cite: 91]:

- A driver for the board's UART, allowing us to implement a real `printf` function[cite: 91].
- Hardware interrupts, a timer tick, and an interrupt-driven transmit path[cite: 91].
- System calls, allowing a user application to securely ask the kernel for services instead of touching the hardware directly[cite: 91].

3.1.1 Organizing Code for an Operating System

While controlling a hardware device could technically be written in just a few lines of code, building a real operating system requires a robust software architecture. Starting this week, we will begin spreading our code across different files based on their logical roles in the OS. The codebase will become more complex not because the operations are inherently difficult, but because the code must be structured professionally to ensure maintainability and separation of concerns.

We will tackle device communication in stages:

1. First, we will build a **driver** using **polling** (the simplest way to control an I/O device).

2. Next, we will introduce **interrupts** to manage devices more efficiently.

3. Finally, we will integrate these mechanisms into a proper operating system structure using system calls.

3.2 CMSDK and CMSIS: ARM's Hardware and Software Standards

To understand how we communicate with devices, it is essential to understand ARM's business model. ARM is not a chip manufacturer (like Samsung or Intel); instead, ARM designs the CPU architecture and the Instruction Set Architecture (ISA), and then sells licenses for these designs to manufacturers.

Some companies (like Samsung) may buy the entire CPU design and simply manufacture it. Other companies (like Apple) pay for a license to use the ARM instruction set, but design the internal CPU hardware themselves.

Because every manufacturer could theoretically design their custom I/O devices completely differently, writing portable operating systems would be a nightmare. To solve this, ARM introduced standards to influence producers to build I/O devices that are software-compatible across different boards. These two standards sound similar but mean very different things[cite: 93]:

	CMSDK (Cortex-M System Design Kit)	**CMSIS (Cortex Microcontroller Software Interface Standard)**
What it is	**Hardware:** Ready-made peripheral designs that chip designers can physically place next to the CPU[cite: 93].	**Software:** A standard for the C headers and functions used to describe a chip[cite: 93].
Examples	APB UART, APB timer, dual timer, watchdog, AHB GPIO[cite: 93].	`NVIC_EnableIRQ()`, names like `UART0->DATA`, `UART0TX_IRQn`, `core_cm3.h`[cite: 93].

3.2.1 How the Standards Work

The **CMSDK** standard tells hardware designers exactly how a device (like a UART) must be structured from a programmer's perspective. It dictates exactly how many registers the device must have, what each bit inside those registers means, and the addresses where those registers must be mapped. How the hardware actually works internally does not matter, as long as it respects these specifications.

Because the hardware interface is standardized, ARM can also provide the **CMSIS** standard—a set of ready-to-use C headers and software drivers. If you buy a board with CMSDK-compatible hardware in the industry, you can usually just include the CMSIS headers and everything works out of the box.

3.2.2 Our Approach in This Course

Our emulated board's peripherals are standard CMSDK blocks[cite: 93]. However, to truly understand how an OS interacts with hardware, **we will not use any vendor CMSIS files or libraries**[cite: 93]. Instead, we will rewrite the entire framework manually from scratch. Our hand-written register header (`cmsdk.h`) is designed in a CMSIS style (so its variable names match standard documentation), but the underlying implementation will be entirely our own work[cite: 93].

3.3 The Emulated Board and the I/O Landscape

To write our own low-level OS layers from scratch, we must understand the specific hardware we are working with. We are using QEMU to emulate the **MPS2** ARM development board, specifically loaded with the **AN385** FPGA image[cite: 94]. This configuration features a Cortex-M3 CPU and several standard CMSDK peripherals, all driven by a 25 MHz system clock[cite: 94].

Our emulated board provides a specific set of hardware devices[cite: 94]:

- **Timers:** Simple hardware devices comprising registers connected directly to the system clock[cite: 94]. You can program them with a specific value to count down to zero, or start from zero and count up. Because they are driven by a known clock frequency (25 MHz), they measure real time with extreme precision. Whenever software needs to perform activities with strict timing, timers are the mandatory hardware choice. The board features three of them: `TIMER0`, `TIMER1`, and a dual timer[cite: 94].
- **UART (Universal Asynchronous Receiver-Transmitter):** A legacy but incredibly common interface used to send serial bits over a line[cite: 94]. It is typically used to connect keyboards or terminals to a device. Its greatest advantage is being `asynchronous`, meaning the communicating devices do not need to share a common clock signal. The board provides three UARTs (`UART0`, `UART1`, `UART2`)[cite: 94].
- **GPIO (General-Purpose Input/Output) and FPGAIO:** Simple hardware lines that can be programmed to output a 1 or a 0, or read incoming binary states[cite: 94]. If you need to connect a simple sensor or an actuator (like an LED or a button), you plug it into a GPIO line. On our emulated board, these are represented by `GPIO0` and the `FPGAIO` block[cite: 94].

3.3.1 Memory-Mapped I/O

How is each device physically accessible to the software? Every device consists of a set of hardware registers, and in the ARM Cortex-M architecture, these registers are **mapped directly into memory**.

As seen in the Cortex-M memory map from Week 2, there is a specific range of addresses strictly reserved for peripherals (`0x4000_0000` to `0x5FFF_FFFF`)[cite: 94]. Every single device is assigned to one or more memory addresses within this region[cite: 94]. Software interacts with the device simply by writing to or reading from these exact memory locations:

Address	Device
0x4000_0000	TIMER0 / TIMER1 / dual timer
0x4000_4000	UART0 / UART1 / UART2
0x4000_8000	watchdog
0x4001_0000	GPIO0 (general-purpose pins)
0x4002_8000	FPGAIO: board LEDs and buttons

3.4 Hardware Architecture: AHB vs. APB Buses

When devices are physically connected to the CPU, they sit on hardware buses. A bus is the physical set of wires (address, data, control) that carries every load and store operation from the CPU to a memory module or a peripheral[cite: 95].

In the ARM architecture, there are two distinct types of buses[cite: 95]:

1. **AHB (Advanced High-performance Bus):** This is the main, ultra-fast, pipelined bus, capable of executing one transfer per clock cycle[cite: 95]. When you design a board, devices that require high-speed, high-frequency access are connected here. This includes RAM, flash code memory, and the GPIO controller[cite: 95].

2. **APB (Advanced Peripheral Bus):** This is a slower, simpler, and lower-power bus[cite: 95]. It is designed for peripheral registers that are touched rarely (e.g., configuring a timer's duration or sending a single byte over the UART). Devices like the UARTs, Timers, and the Watchdog reside here[cite: 95].

The AHB-to-APB Bridge and Performance

To allow the CPU to communicate with devices on the slower bus, the board utilizes a small electronic circuit called the **AHB-to-APB bridge**[cite: 95]. This bridge acts like a network router: it takes transactions arriving from the high-speed AHB and, if they are directed to an APB device, forwards them to the low-speed bus[cite: 95].

From a software programming perspective, this complex hardware routing is totally transparent[cite: 95]. When you write software, you simply read or write instructions in memory[cite: 95]. However, from a performance perspective, it has a tangible impact: a single store operation targeting a device on the low-speed APB will take several more clock cycles to complete than a store operation targeting the high-speed AHB[cite: 95]. If you are designing a real-time system where performance is highly critical, you must be aware of the latency introduced by the peripheral's bus placement.

Finally, while these devices execute their operations on the bus, many of them are also capable of generating **interrupts** to proactively notify the CPU that an operation is complete[cite: 96].

3.5 The Board's Interrupt Lines

Devices communicate with the CPU by raising physical hardware signals called Interrupt Request (IRQ) lines[cite: 96]. These physical lines are limited, and in

a standardized architecture like the Cortex-M, devices are hardwired to specific, predefined lines[cite: 96]. When designing a board around a standard architecture, hardware designers are not free to connect devices to any arbitrary interrupt line[cite: 96].

The architecture strictly dictates which device must be connected to which line. For example, device IRQ line N is served by vector-table slot $16 + N$, which sits right after the 16 core slots you configured during the boot process[cite: 96].

The AN385 board provides the following specific IRQ mapping for our devices[cite: 96]:

IRQ	Device	IRQ	Device
0	`UART0` receive (RX)	8	`TIMER0`
1	`UART0` transmit (TX)	9	`TIMER1`
2/3	`UART1` RX / TX	10	dual timer
4/5	`UART2` RX / TX	12	UART overflow (all UARTs combined)

While you do not need to memorize all these lines, when you write software, you absolutely need to know where your hardware is connected. If we want to implement an interrupt-driven UART driver, we must use the exact slot in the vector table (e.g., IRQ 0 and 1 for UART0) to set the interrupt handler[cite: 96].

`Hardware Notes`:

- High-end embedded boards often feature many UARTs and timers; they are extremely important for system design, and having more of them generally makes a board more capable. Our board features three UARTs (`UART0`, `UART1`, `UART2`).

- The watchdog timer has no standard IRQ line; instead, it raises the NMI (Non-Maskable Interrupt, slot 2) exception, which cannot be masked or ignored[cite: 96].

- QEMU raises every line in the table except IRQ 12, because its UART model never reports an overflow condition[cite: 96].

In the initial part of this unit, the CPU will simply poll the devices (asking them in a loop), meaning we will not use these interrupts just yet[cite: 96].

3.6 Software Organization: `include/cmsdk.h`

From the hardware standpoint, we know that every I/O device is associated with a specific range of memory addresses, and the specifications dictate the meaning of each address. From a software perspective, things start to become more complex.

If you wanted to write software in a completely unstructured way, you could simply keep your CPU in privileged mode and directly read or write to raw hexadecimal memory addresses to control everything. However, to organize the software professionally—like a real operating system—it is best practice to define data types that group the information and registers of a device into a `struct`[cite: 97].

3.6.1 Registers as a Struct

The CMSDK specification dictates exactly how many registers a device has[cite: 97]. For instance, a CMSDK-compliant UART features exactly five registers, and every register is 32 bits wide[cite: 97]. We define a data structure with five elements, each being an unsigned 32-bit integer[cite: 97].

This layout is completely provided in the `include/cmsdk.h` file:

```
typedef struct
{
    volatile uint32_t DATA;      /* 0x00: read = received byte, write = t
    volatile uint32_t STATE;     /* 0x04: status flags */
    volatile uint32_t CTRL;      /* 0x08: control flags */
    volatile uint32_t INTSTATUS; /* 0x0C: interrupt status; write 1 to cl
    volatile uint32_t BAUDDIV;   /* 0x10: baud-rate divider */
} CMSDK_UART_TypeDef;

#define CMSDK_UART0 ((CMSDK_UART_TypeDef *) 0x40004000UL)
```

3.6.2 Why Use Structures?

In C, the elements of a `struct` are allocated in continuous memory addresses. If you know the **base address** of a device by checking the board manual (e.g., `0x40004000` for `UART0`), you can instantiate the design by casting that initial address to a pointer of your struct type[cite: 97].

Because the structure exactly mirrors the hardware, addressing the members of the data structure actually accesses the correct, continuous hardware memory addresses[cite: 97]. For example, `CMSDK_UART0->STATE` translates to a 32-bit load from `0x40004004`[cite: 97].

While you could manually declare every single register as a separate pointer, using a `struct` is a highly recommended best practice resulting from years of software maintenance and bug-fixing experience. The SDK design is incredibly general: because our board has three UARTs, we define the `struct` layout only once, and then simply write three `#define` macros specifying the three different base addresses. The more you structure the software like this, the easier it is to maintain.

3.6.3 Bitmasks and Hardware Control

Because every device is ultimately controlled by manipulating specific bits within its registers, relying on raw hexadecimal values for every configuration is incredibly difficult and error-prone. To solve this, another standard good practice is creating **bitmasks**. This allows you to associate a human-readable mnemonic with a specific bit position.

In our `cmsdk.h` file, alongside the data structures, you will find a large number of bitmask definitions. These are typically defined using the left-shift bitwise operator (`«`)[cite: 108]:

- `1 « 0`: Takes a 1 and shifts it 0 places to the left. This creates a mask where exactly bit 0 is equal to 1.

- `1 << 1`: Shifts the 1 one place to the left, meaning exactly bit 1 is equal to 1.
- `1 << 2`: Shifts the 1 two places to the left, meaning exactly bit 2 is equal to 1.

This notation is highly compact. While you could write out the full 32-bit hexadecimal masks (e.g., `0x00000004` for bit 2), reading strings of zeros becomes visually confusing. The compiler generates the exact same optimized machine code either way, but the shift notation allows a human reader to immediately know exactly which bit number is being targeted.

As an operating system designer, this is the very first stage of abstracting your hardware. You define data structures to give names to your memory addresses, and you define bitmasks to control the registers. This ensures your low-level code remains readable and maintainable.

3.7 Try It: The Board's Memory Map from QEMU

How can we verify that these devices and memory addresses actually exist? Because we are using the QEMU emulator, we can directly ask it to print the list of devices it is actively modeling[cite: 98].

QEMU features a **monitor**, which is a command console used to control the emulator itself[cite: 98]. You can start QEMU with no program loaded, paused (`-S`), with the monitor attached to your terminal, and with standard UART output discarded[cite: 98]:

```
qemu-system-arm -M mps2-an385 -display none -S -monitor stdio -serial nu
```

Once inside the QEMU monitor, typing `info mtree -f` prints every single memory address range and specifies what hardware component answers at that address[cite: 98].

Because the raw output is massive (it prints every address range available in QEMU), we can use a compact one-liner to filter the output, keeping only the peripheral addresses (`0x400xxxxx`)[cite: 98]:

```
printf 'info mtree -f\nquit\n' | qemu-system-arm -M mps2-an385 \
-display none -S -monitor stdio -serial null | grep 00000000400 | sort -u
```

Running this command explicitly prints the addresses of the devices that are actually implemented and working in QEMU[cite: 98]. You will clearly see the UART mapped at `0x40004000`, as well as the timers (`cmsdk-apb-timer`), the watchdog (`cmsdk-apb-watchdog`), and the FPGA I/O (`mps2-fpgaio`) mapped to their respective addresses[cite: 99].

Interpreting the QEMU Memory Map Output

When you analyze the output of the `info mtree -f` command, you will notice differences in how the devices are labeled. Specifically, you will see a priority value (`prio`) assigned to each memory region[cite: 99].

- `prio 0` + **Device Name:** If a line shows `prio 0` followed by a specific device name (like `cmsdk-apb-timer` or `uart`), it means QEMU provides a **real, working model** of that device[cite: 99]. If your software performs operations at that memory address, QEMU will fully emulate the hardware's behavior.

- `prio -1000`: If a line shows `prio -1000` (such as the `cmsdk-ahb-gpio`), this indicates a **placeholder** (an "unimplemented device")[cite: 99]. Your software can still legally read or write to that memory location without crashing, but QEMU is not actually emulating the underlying hardware logic[cite: 99].

This distinction is crucial for emulator-based development. Placeholders are beneficial because they allow your software to execute without throwing hardware faults, but they have no functional behavior: writes are silently ignored, and reads typically just return 0[cite: 99].

For example, the GPIO (General-Purpose Input/Output) controller—which is a highly important I/O device on a real board—is merely a placeholder in our QEMU model[cite: 99]. You can write a 1 to a GPIO line to "turn on an LED", but nothing will visually happen in the emulator[cite: 99]. This limitation highlights why, to test complex hardware interactions, developers eventually need to deploy code onto physical hardware.

`Other noted devices`: You might also see `pl022` and `arm_sbcon_i2c` in the list[cite: 99]. These are an SPI controller and an I2C controller, respectively[cite: 99]. They are additional serial interfaces, but they are not fully emulated or used in this specific project[cite: 99].

3.7.1 Try It: Touch a Device That Is Not There

You can actively watch QEMU report these placeholder accesses by running the emulator with the `-d unimp` flag (which logs accesses to unimplemented devices)[cite: 100].

If you start QEMU with tracing enabled and attach GDB[cite: 100]:

```
# Terminal 1
make run-trace

# Terminal 2
arm-none-eabi-gdb build/scratch-os.axf
(gdb) target remote :1234
```

You can poke the system bus directly from GDB[cite: 100]:

```
(gdb) set {unsigned int} 0x40010010 = 1     # Write to GPIO0
(gdb) x/wx 0x40010010                       # Read from GPIO0
(gdb) set {unsigned int} 0x40028000 = 3     # Write to FPGAIO LED register
(gdb) x/wx 0x40028000                       # Read from FPGAIO
```

In Terminal 1 (QEMU), you will see the GPIO write and read operations explicitly logged and dropped[cite: 101]:

```
cmsdk-ahb-gpio: unimplemented device write (size 4, offset 0x010, value 0
cmsdk-ahb-gpio: unimplemented device read (size 4, offset 0x010)
```

In Terminal 2 (GDB), the read results confirm the emulation status[cite: 101]:

- `0x40010010` (GPIO) returns `0x00000000` because it is a placeholder[cite: 101].
- `0x40028000` (FPGAIO) successfully returns `0x00000003` because the LEDs are actively modeled by QEMU and keep the value you wrote[cite: 101].

Lesson: Before trusting a device in QEMU, always verify its status with `info mtree`.

3.8 The General I/O Landscape

To contextualize the devices we are working with, it is helpful to summarize the major categories of I/O interfaces found on modern embedded boards[cite: 102]. You rarely connect a complex peripheral (like a camera) directly to the CPU; instead, you plug it into one of these standardized I/O controllers[cite: 102].

Interface	Typical Use and Mechanics
GPIO	**General-Purpose Input/Output.** These are simple, single-bit physical wires. You can program each line as an `input` (to read the 1/0 state of a sensor or button) or as an `output` (to drive a 1/0 voltage to an actuator, like turning an LED on or off)[cite: 102].
UART	**Universal Asynchronous Receiver-Transmitter.** A serial connection using two distinct wires (TX and RX)[cite: 102]. It is strictly `asynchronous`, meaning there is no shared clock line between the devices; both sides must agree on a transmission speed (baud rate) beforehand[cite: 102]. Typically used for console output or PC communication[cite: 102].
SPI	**Serial Peripheral Interface.** A fast, synchronous serial bus. The controller acts as the `master` and actively drives a shared clock wire[cite: 102]. It uses a chip-select wire to pick which `slave` device on the bus to talk to[cite: 102]. Often used for external flash memory or displays[cite: 102].
I2C	**Inter-Integrated Circuit.** A synchronous serial bus that uses only two wires (clock and data) shared among multiple devices[cite: 102]. It supports multiple masters and multiple slaves, using 7-bit addresses to route data[cite: 102]. Often used for simple sensors[cite: 102].
ADC / DAC	**Analog-to-Digital / Digital-to-Analog Converters.** Interfaces that translate an analog voltage level into a digital number (ADC, e.g., reading a temperature sensor) or convert a digital number into a physical voltage (DAC, e.g., outputting audio)[cite: 102].

In this course, we will focus primarily on driving the UART, with timers acting as an exception since they are internal core devices that do not route through external interfaces.

3.8.1 DMA: Moving Data Without the CPU

Currently, our CPU must manually move every single byte of data: reading a register, storing it to memory, and repeating. For large data transfers (like reading a massive flash sector or an audio buffer), this forces the CPU to be 100% busy for the duration of the transfer[cite: 103].

Modern microcontrollers solve this using a **DMA (Direct Memory Access)** controller[cite: 103]. The DMA is an independent hardware device that copies data by itself[cite: 103]. The software simply programs the DMA with a source address, a destination address, and a byte count, and then starts it[cite: 103]. The DMA uses the bus to move the data while the CPU is free to run other code[cite: 103].

Once finished, the DMA typically signals the CPU via a hardware interrupt[cite: 103].

While incredibly powerful, utilizing DMA comes with costs: the CPU and DMA must share the hardware bus, and the software must be carefully designed to never touch a memory buffer while the DMA is actively filling it[cite: 103]. Because QEMU's AN385 board does not feature a DMA controller, our OS will rely on the CPU to manually move every byte[cite: 103].

3.9 Memory-Mapped I/O: Loads and Stores with Side Effects

Different microprocessor architectures employ different programming styles to control I/O devices. For example, the x86 architecture features specific I/O instructions integrated directly into its assembly instruction set.

The ARM architecture uses a different, more modern approach known as **Memory-Mapped I/O (MMIO)**. The instruction set has no specific I/O commands; instead, peripheral registers are assigned fixed memory addresses[cite: 104]. Talking to a device is achieved using standard `load` and `store` operations[cite: 104].

In C, this looks exactly like a standard variable pointer assignment[cite: 104]:

```
#define UART0_DATA (*(volatile uint32_t *)0x40004000u)
UART0_DATA = 'A'; /* A store to 0x40004000: the UART sends 'A' */
```

3.9.1 Hardware Side Effects

While it looks like a variable assignment in C, a hardware register is fundamentally **not RAM**[cite: 104]. Operations on these memory addresses have immediate physical consequences in the real world (e.g., writing to a register might physically move a robot arm)[cite: 104].

Because of this, MMIO introduces several behaviors that seem counterintuitive when writing standard C code[cite: 104]:

- **Reads have side effects:** Reading a memory location might alter the device's state (e.g., reading the UART data register extracts the byte and clears the "byte waiting" flag)[cite: 104].
- **Writes initiate actions:** Writing a value triggers a hardware action, which may asynchronously change other bits in the register to report the outcome[cite: 104].
- **Values change spontaneously:** If you read the exact same memory address twice in consecutive lines without writing anything, you may get two completely different values (e.g., a hardware timer actively counting down)[cite: 104].
- **Read-back differs from written value:** If you write a value to a register and immediately read it back, you might get a different result because some bits are read-only or hardware-controlled[cite: 104].

3.10 The volatile Keyword

These hardware side effects directly conflict with how standard C compilers optimize code[cite: 105]. A modern compiler assumes it is dealing with ordinary RAM, where a value only changes if the software explicitly writes to it[cite: 105].

Consider a loop waiting for a transmission buffer to empty:

```
while (UART0_STATE & UART_STATE_TXBF)
{
    /* wait until TXBF is 0 */
}
```

Without intervention, the compiler's optimizer analyzes this snippet and notices that `UART0_STATE` is never modified inside the loop body[cite: 105]. It concludes the value "cannot change"[cite: 105]. The compiler will read the value exactly once before the loop; if the condition is true, it optimizes the code into an infinite spin loop, completely removing all subsequent hardware reads[cite: 105]. If the condition is false, it might remove the snippet entirely[cite: 105].

To prevent this catastrophic optimization, the `volatile` keyword is absolutely mandatory when accessing device registers[cite: 105]. It explicitly tells the compiler that the value at this memory address can change without any software action[cite: 105]. It forces the compiler to ensure that **every read and every write in the source code must really happen**, exactly in the order written, and must never be cached or optimized away[cite: 105].

3.10.1 Try It: volatile at -O2

To truly see how the compiler can delete hardware reads, you can examine the generated assembly code[cite: 106]. Consider a C file (`poll.c`) with the following polling loop:

```
#include <stdint.h>
#ifndef QUAL
#define QUAL /* empty: a plain pointer */
#endif

/* UART0_STATE, bit 0 = TXBF (Transmit Buffer Full) */
#define UART0_STATE (*(QUAL uint32_t *)0x40004004u)

void wait_tx_room(void)
{
    while (UART0_STATE & 1)
    {
        /* wait until TXBF is 0 */
    }
}
```

If you compile this twice with optimization level `-O2` (once normally, and once with `-DQUAL=volatile`) and inspect the output using `objdump -d`, you will see a drastic difference[cite: 106, 107]:

- **Without `volatile` (`plain.o`):** The compiler emits a single `ldr` instruction to read the state register `before` the loop[cite: 107]. Inside the loop body, it simply branches back to itself (`b.n`) infinitely without ever issuing another `ldr`[cite: 107]. If the hardware buffer was full on that first read, the program hangs forever[cite: 107].

- **With `volatile` (`vol.o`):** The branch at the bottom of the loop explicitly jumps back to the `ldr` instruction[cite: 107]. The CPU physically reads the hardware register `UART0_STATE` on every single iteration of the loop[cite: 107].

Warning: At optimization level `-O0` (which our Makefile currently uses), the plain version happens to re-read the register anyway[cite: 107]. This means a missing `volatile` bug will hide perfectly until you turn on optimizations for a release build, at which point the system will catastrophically freeze[cite: 107]. Always use `volatile` for hardware registers[cite: 107].

3.11 Read-Modify-Write Needs Care

When configuring a device, you often need to change a single bit while leaving the rest of the register intact. In C, this is commonly written using combined assignment operators, like the bitwise OR assignment:

```
CMSDK_GPIO0->DATAOUT |= (1 << 3); /* set pin 3 */
```

While this looks like a single line of C code, at the hardware level, it compiles into three distinct operations[cite: 108]:

1. **Read:** Load the current value of the register from memory into a CPU register.

2. **Modify:** Perform the bitwise OR operation.

3. **Write:** Store the modified value back to the hardware memory address.

These three operations are **not atomic**[cite: 108]. This introduces several dangerous hardware pitfalls[cite: 108]:

- **Concurrency conflicts:** If an interrupt fires or the hardware autonomously changes a different bit in the register between your read and your write, your subsequent write will forcefully overwrite the hardware's changes with outdated data[cite: 108].

- **Write-1-to-clear (W1C) bits:** Certain status registers clear their flags when you write a 1 to them, leaving them alone if you write a 0[cite: 108]. Using a read-modify-write pattern on a W1C register is disastrous: it reads back all currently set flags (as 1s) and writes them straight back, accidentally clearing every pending interrupt flag[cite: 108].

- **Read $\neq$ Written:** Some registers return different values when read compared to what was written[cite: 108]. Reading and writing back such a register can unintentionally scramble bits you did not mean to touch[cite: 108].

3.11.1 Safer Hardware Patterns

To avoid these pitfalls[cite: 108]:

- **Set / Clear Registers:** Many modern peripherals provide dedicated registers strictly for setting or clearing bits (e.g., `OUTENABLESET`)[cite: 108]. Writing `CMSDK_GPIO0->OUTENABLESET = 1u << 3;` is a single write operation; the hardware ensures that only the bits set to 1 are modified[cite: 108].
- **W1C Registers:** Always use a plain assignment (`=`) containing only the specific bit you want to clear[cite: 108]. Never use `|=`[cite: 108].
- **Unavoidable Read-Modify-Write:** If a read-modify-write is strictly required, ensure that no other code or interrupts can touch the register during the operation (which we will learn to handle gracefully later in the course)[cite: 108].

3.12 The Device Protocol and the Driver Concept

Interacting with a hardware device is not just about flipping bits randomly; it requires following a strict state machine protocol dictated by the device's manual[cite: 109].

The lifecycle typically follows these steps[cite: 109]:

1. **Configure:** Set up operational parameters (e.g., setting the UART baud rate). Hardware designers usually ensure that devices boot in a disabled, uninitialized state to prevent unpredictable behavior at power-on[cite: 109].
2. **Enable:** Switch on the necessary circuits (e.g., turning on the transmitter)[cite: 109].
3. **Check Status:** Verify the device is ready (e.g., checking if the transmit buffer has room)[cite: 109].
4. **Move Data:** Send or receive the actual payload data[cite: 109].
5. **Acknowledge:** Clear any relevant event flags so the hardware can report the next event[cite: 109].

3.12.1 What is a Device Driver?

As operating system designers, our role is to absorb this hardware complexity. The software layers above us (like user applications) should not need to know memory addresses, bitmasks, or hardware protocols.

A **device driver** is the specialized code that intimately knows a specific device's registers and protocol[cite: 111]. It acts as an API boundary, offering the rest of the system simple functions with clear meanings (e.g., "send this byte")[cite: 111]. Callers include the driver's header file (`drivers/uart.h`), but they never directly include `cmsdk.h`[cite: 111]. Only the driver touches the raw hardware registers[cite: 111].

3.13 Writing the UART Driver

We will build a UART driver because the emulated UART is physically connected to the QEMU terminal. By driving this hardware properly, we can implement a real `printf` function, replacing the artificial semihosting debug trick from Week 2.

The driver API will start small and simple[cite: 111]:

```
void uart_init(void);                    /* Set up UART0; called once at boot
void uart_putc_polled(char c);           /* Send one byte; waits if buffer is
int uart_getc_polled(void);              /* Get a byte, or -1 if none is waiti
```

This initial driver will be **polled**, meaning it reads the hardware status register in a loop to determine if it can send or receive data, without using interrupts[cite: 111].

3.13.1 The CMSDK UART Registers

To control the UART and implement our driver, we must intimately understand the meaning of its hardware registers as defined by its specifications[cite: 110].

The CMSDK UART features the following primary registers[cite: 110]:

- `0x00 DATA` **(R/W):** This is the data register. Writing a byte to this address physically instructs the UART to transmit it[cite: 110]. Reading from it extracts the received byte[cite: 110].
- `0x04 STATE` **(R, W1C):** This register reports the current status of the device[cite: 110]. It contains critical bits such as:
  - **Bit 0 (`TXBF`):** Transmit Buffer Full. If this is 1, the device is still sending the last byte; any new write to `DATA` will be lost[cite: 110].
  - **Bit 1 (`RXBF`):** Receive Buffer Full. If this is 1, a byte is waiting to be read[cite: 110].

  Because QEMU pulls characters from the UART and prints them to our terminal much slower than our CPU executes instructions, our software must actively check these bits and wait before pushing new data.
- `0x08 CTRL` **(R/W):** The control register. This allows us to enable or disable the transmitter (`TXEN`) and receiver (`RXEN`), and configure interrupts[cite: 110]. For the first part of this week, we will leave interrupts disabled.
- `0x10 BAUDDIV` **(R/W):** The transmission rate divider[cite: 110].

Configuring the Baud Rate

Because UART is asynchronous (there is no shared clock), both devices must agree on a specific transmission speed, known as the **baud rate** (bits per second)[cite: 102]. If you have ever connected an Arduino to a PC using a program like PuTTY, you had to manually set the baud rate so the two independent devices could understand each other.

To configure this in hardware, you must divide your CPU's clock frequency by your target baud rate. For example, if our board's CPU runs at 25 MHz and we want to transmit at 115,200 baud, we compute[cite: 110]:

$$\text{BAUDDIV} = \frac{25,000,000}{115,200} \approx 217$$

Writing 217 into the `BAUDDIV` register configures the UART correctly[cite: 110]. However, because we are using QEMU, the emulator ignores the baud rate and prints instantly[cite: 110]. Therefore, our driver simply writes `16` (the smallest legal value permitted by the architecture) into this register[cite: 110].

3.14 Implementing the Device Driver

A device driver is essentially a set of high-level functions that implement low-level hardware operations. While complex operating systems like Linux enforce strict standards on how a driver must be written, we are building our own OS and have the freedom to design it ourselves[cite: 112].

Our initial polled driver requires three simple C functions[cite: 111]:

1. An `init` function to initialize the UART[cite: 111].
2. A `write` function to send a character[cite: 111].
3. A `read` function to receive a character[cite: 111].

3.14.1 How Bigger OSes Generalize a Driver

If you want to design an elegant OS architecture, defining plain functions is often not enough. If a system has many different devices of the same category (e.g., character devices like UARTs and keyboards), the OS needs a generic way to talk to them without caring about the underlying hardware[cite: 112].

Operating systems solve this by defining a `struct` composed entirely of function pointers[cite: 112].

```
struct char_driver
{
    void (*init)(void);          /* bring the device up */
    int  (*read)(void);          /* read one byte, or -1 if none */
    void (*write)(char c);       /* send one byte */
};
```

To write a new device driver, the developer implements the low-level functions and then populates an instance of this structure[cite: 112]:

```
static const struct char_driver uart0_driver = {
    uart_init,
    uart_getc_polled,
    uart_putc_polled
};
```

This allows the higher-level kernel code to interact generically with any device[cite: 112]:

```
void say_hi(const struct char_driver *dev)
{
    dev->write('h');
    dev->write('i');
}
```

This is exactly how Linux manages drivers (using `struct file_operations`). For our minimal OS, we will call our UART functions directly, but understanding this architectural pattern is crucial for OS design[cite: 112].

3.14.2 Try It: The First UART Byte

We will now implement the polled driver in `drivers/uart.c`.

First, we implement the initialization function[cite: 113]. Following the hardware protocol, we must configure the speed and explicitly enable the transmitter and receiver circuits[cite: 113]. If we attempt to write data while the UART is disabled (because `CTRL` defaults to 0 at reset), the UART will completely drop the byte, and QEMU will report a guest error[cite: 115].

```
#include "drivers/uart.h"
#include "cmsdk.h"

void uart_init(void)
{
    /* configure: speed (QEMU ignores it, but 16 is the minimum) */
    CMSDK_UART0->BAUDDIV = 16;

    /* enable: transmitter and receiver */
    CMSDK_UART0->CTRL = UART_CTRL_TXEN | UART_CTRL_RXEN;
}
```

Next, we implement the polled transmission function[cite: 113]. Before moving data, we must actively check the hardware status. The function reads the `STATE` register in a `while` loop until the `TXBF` (Transmit Buffer Full) bit clears. Once there is room, it moves the data by writing to the `DATA` register[cite: 113].

```
void uart_putc_polled(char c)
{
    /* check status: wait for room */
    while (CMSDK_UART0->STATE & UART_STATE_TXBF)
    {
        /* spin */
    }

    /* move data: send the byte */
    CMSDK_UART0->DATA = (uint32_t)(unsigned char)c;
}
```

By calling `uart_init()` and `uart_putc_polled()` from `main.c`, we can establish the first real hardware output of our operating system[cite: 114].

3.14.3 Structuring the Driver Files

To properly organize our operating system, every device driver is typically separated into its own header (`.h`) and implementation (`.c`) file.

- `drivers/uart.h`: This header file defines the API boundary. It contains only the prototypes of the functions we want to expose to the rest of the OS (e.g., `uart_init`, `uart_putc_polled`, and `uart_getc_polled`).
- `drivers/uart.c`: This file contains the actual C implementation. Crucially, it must include both its own header (`uart.h`) and the hardware definitions (`cmsdk.h`).

By including `cmsdk.h` strictly inside the `.c` file, we abstract the hardware addresses away from the rest of the system. If the underlying hardware changes, we only need to update the CMSDK layer and the driver's `.c` file; every higher-level application built on top of this driver will continue to compile and work perfectly without modification.

3.14.4 Step-by-Step Implementation

Let us break down the exact C implementation of these operations.

1. Initialization

The first function we implement is `uart_init`. Based on the hardware protocol, we must configure the speed and explicitly enable the device.

```
void uart_init(void)
{
/* Configure the transmission speed (baud rate divider) */
CMSDK_UART0->BAUDDIV = 16;

/* Enable both transmitter and receiver using a bitwise OR */
CMSDK_UART0->CTRL = UART_CTRL_TXEN | UART_CTRL_RXEN;

}
```

We use a logical bitwise OR (`|`) to combine two bitmasks: one that sets the transmission bit to 1, and one that sets the receiving bit to 1. This single assignment enables both data directions simultaneously.

2. Polled Transmission

Next, we implement `uart_putc_polled` to write a character to the UART line. Because we are using polling, we rely on a busy-waiting loop.

```
void uart_putc_polled(char c)
{
/* Check status: Wait until the transmission buffer has an empty slot */
while (CMSDK_UART0->STATE & UART_STATE_TXBF)
{
```

```
/* Busy waiting */
}

/* Move data: Write the character to the data register */
CMSDK_UART0->DATA = (uint32_t)(unsigned char)c;

}
```

If the `TXBF` (Transmit Buffer Full) bit is 1, the UART is not ready to receive new data, so the CPU loops infinitely, waiting. Once the buffer is free, the loop exits, and we perform a store operation. We copy the character into the `DATA` memory address. At this point, the character is handed off to the UART hardware, and the CPU does not care about it anymore.

3.14.5 Using the Driver as an Application Designer

Now that the low-level device driver is written, we can switch roles to become an application software designer. We go into our `src/main.c` program and include the device driver by including `uart.h`. *Notice that `main.c` does not include `cmsdk.h`; it only uses the high-level API.*

```
#include "drivers/uart.h"

int main(void)
{
/* 1. Initialize the driver */
uart_init();

/* 2. Print characters using the hardware */
uart_putc_polled('O');
uart_putc_polled('K');
uart_putc_polled('\n');

for (;;) { }

}
```

When we execute these functions, we are sending characters to an external physical device (the UART). The UART transmits them through a serial line to the receiving device. In our emulation environment, QEMU acts as the receiving computer, capturing whatever is transmitted on that serial line and physically printing it to your screen.

`Troubleshooting Note`: What happens if you comment out the `uart_init()` call in `main`? Because the `CTRL` register defaults to `0` at hardware reset, the transmitter remains strictly off. The UART will drop every byte written to the `DATA` register. Running QEMU with the `-d guest_errors` flag will explicitly report: `CMSDK APB UART: transmit data write with Tx disabled`.

3.14.6 The Reality of Hardware Printing

When you run the code and see `OK` printed on the screen, it is crucial to understand that this is **not** the same mechanism we used last week. Last week, we used a semihosting debug trick that halted the CPU and asked the QEMU emulator to print on our behalf. Now, we are performing real hardware communication: the CPU is writing data bits directly into a hardware register, and the UART peripheral is transmitting those bits over a serial line to QEMU's terminal.

`Architectural note on initialization:` A common question is how many times the UART must be initialized if the project grows to multiple C files. Because the UART is an external physical device, you only initialize it `once` at boot (typically in `main`). Since our operating system currently only has a single process, that single initialization serves the entire system. If we eventually add multiple processes or need to dynamically change the baud rate, we might reinitialize it, but normally, hardware initialization is a one-time setup.

3.14.7 Try It: A Polled Echo

We have implemented transmission, but the UART is a bidirectional interface. We need the opposite operation: receiving data. In the QEMU environment, the UART's output is connected to your terminal screen, and its input is physically connected to your keyboard.

We add a reading function to our device driver (`drivers/uart.c`). Because this is a polled driver, if there is no character waiting, the function must not freeze; it should simply return `-1` immediately:

```
int uart_getc_polled(void)
{
/* Check status: if the Receive Buffer is empty, return -1 */
if ((CMSDK_UART0->STATE & UART_STATE_RXBF) == 0)
{
return -1;
}

/* Move data: reading DATA automatically extracts the byte */
return (int)(CMSDK_UART0->DATA & 0xFF);

}
```

Now, we can update our `main.c` to utilize this new capability. We replace the empty infinite loop with an echo loop:

```
for (;;)
{
int c = uart_getc_polled();
if (c >= 0)
{
uart_putc_polled((char)c);
}
}
```

When you run `make run` and type on your keyboard, you will see the `OK` banner, followed by whatever you type. (With the `-nographic` flag, the terminal itself does not show your keystrokes; every letter you see on the screen was explicitly sent back by your hardware echo loop).

This loop highlights the massive performance penalty of **polling**. The CPU spins in this infinite loop, executing millions of useless checks per second even when you are not pressing any keys. We will solve this inefficiency using interrupts later in this unit.

3.15 Variadic Functions and <stdarg.h>

Currently, our driver can only print single characters. To build a true operating system, we need a `printf`-like function capable of formatting complex strings and numbers (e.g., `%d`, `%s`, `%x`). We will call our kernel-level version `kprintf`.

To build `kprintf`, we face two tasks:

1. String formatting (parsing the `%d` markers and converting integers to ASCII characters).

2. Sending the resulting characters to the UART.

In C, a function that accepts an arbitrary number of arguments (like `printf`) is called a **variadic function**. It is declared using an ellipsis (`...`):

```
void kprintf(const char *fmt, ...);
```

Because we are compiling in a freestanding environment without the standard library (`<stdio.h>`), we must format the string ourselves. Fortunately, the C compiler still provides the `<stdarg.h>` header. This header contains no library executable code; it only provides the compiler built-in macros (`va_list`, `va_start`, `va_arg`, `va_end`) required to iterate sequentially through the unknown extra arguments.

```
#include <stdarg.h>

void kprintf(const char *fmt, ...)
{
va_list ap;               /* a cursor over the extra arguments */
va_start(ap, fmt);        /* start the cursor right after 'fmt' */

/* Formatting logic goes here using va_arg(ap, type) */

va_end(ap);               /* done */

}
```

`Warning:` Variadic functions inherently lack type safety. The function cannot automatically detect how many arguments were passed or what type they are. It strictly trusts the format string; if you specify `%d` but pass a string pointer, the compiler cannot catch the error, resulting in undefined behavior.

3.15.1 Separating Formatting from Output (vformat)

The logic required to parse a format string (like converting the integer 42 into the ASCII characters '4' and '2') is pure C programming[cite: 118]. This formatting logic is completely independent of where the characters are physically sent[cite: 118].

To maintain a clean operating system architecture, we separate the formatting engine from the hardware driver using a function called vformat (implemented in lib/format.c)[cite: 118].

```
void vformat(void (*out)(char c, void *ctx), void *ctx, const char *fmt,
```

Instead of calling the UART directly, vformat accepts a function pointer named out[cite: 118]. It parses the string and calls out(c, ctx) exactly once for every character generated[cite: 118]. This means the exact same formatting code can be reused whether the OS is printing to a serial port, drawing to a screen, or simply writing to a memory buffer[cite: 118].

3.16 The Kernel Print Function (kprintf)

With the formatting engine and the UART driver both ready, we can combine them to create a real print function. We name it kprintf (where the "k" stands for kernel)[cite: 118].

3.16.1 Kernel Mode vs. User Mode Printing

Naming this function kprintf highlights a fundamental operating system concept. In a real OS (like Linux), there is a strict separation between how the kernel prints messages and how a user application prints messages:

- **User-Level printf:** A user application runs in a restricted user mode. It does not know that a UART even exists, nor does it have the hardware privileges to touch it. When a user program calls printf, it triggers a system call, asking the OS to print on its behalf. The OS then securely routes that request to the appropriate device driver.
- **Kernel-Level kprintf:** The OS kernel itself runs in privileged (supervisor) mode. It inherently knows about the hardware and the I/O devices. Therefore, kprintf bypasses system calls entirely and talks directly to the hardware driver.

Any operating system that implements hardware protection features both a user-side print function and a kernel-side print function. We are currently inside the operating system layer, so we implement kprintf first. (The more complex user-level system call mechanism will be implemented later).

3.16.2 Implementing kprintf

In kernel/kprintf.c, we write a small wrapper function that connects vformat to our polled UART driver[cite: 122].

First, we define the callback function that vformat expects. This simply takes a character and hands it to the driver[cite: 122]:

```
#include "kernel/kprintf.h"
#include "lib/format.h"
#include "drivers/uart.h"

/* vformat's output callback: sends one char to the UART */
static void kprintf_out(char c, void *ctx)
{
    (void)ctx; /* Context is unused here */
    uart_putc_polled(c);
}
```

Then, we implement the variadic `kprintf` function itself. It initializes the argument list using the `<stdarg.h>` macros, passes our `kprintf_out` callback to `vformat`, and cleans up when finished[cite: 122]:

```
void kprintf(const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);

    /* Pass the formatting work to vformat, targeting the UART */
    vformat(kprintf_out, 0, fmt, ap);

    va_end(ap);
}
```

Because `kprintf` relies on our polled UART driver, it operates entirely without hardware interrupts[cite: 118]. This is a massive architectural advantage: it means `kprintf` can be safely executed anywhere in the kernel, even inside a catastrophic `HardFault` handler where interrupts are strictly disabled[cite: 118].

Implementation note: To follow along with the codebase, you can copy the fully implemented `lib/format.c` and `kernel/kprintf.c` files directly from the solution folder into your working directory.

3.17 Upgrading the Kernel: Fault Handlers That Print

Now that we have successfully built a working device driver and a robust `kprintf` function, we can put them to use inside the kernel.

Recall from Week 2 that we configured the CPU's vector table with specific exception handlers (like `HardFault_Handler` and `UsageFault_Handler`). In that initial implementation, these handlers were completely silent: they consisted of nothing more than an infinite `while(1)` loop. If a hardware fault occurred, the system simply locked up, leaving the developer completely blind to the cause.

Because our new `kprintf` function relies strictly on a polled driver, it does not require interrupts to function[cite: 118]. This means it can safely execute even inside a catastrophic hardware fault handler where normal system interrupts are masked[cite: 125]. We can now replace the silent loops with handlers that actively print the exact reason for the crash[cite: 123].

3.17.1 The Status Registers (HFSR and CFSR)

When an exception arises, the Cortex-M hardware automatically records the specific reason for the crash into dedicated system registers[cite: 124]. You do not need to memorize these addresses by heart; instead, we access them cleanly through the data structures we mapped earlier (via the SCB pointer in our CMSDK headers).

The two critical registers we want to read are[cite: 124]:

- **CFSR (Configurable Fault Status Register):** Located at 0xE000ED28, this register contains one bit for every specific cause of a MemManage, BusFault, or UsageFault[cite: 124].
- **HFSR (Hard Fault Status Register):** Located at 0xE000ED2C, this indicates exactly why a HardFault was escalated[cite: 124].

3.17.2 Implementing the New Handlers

In kernel/faults.c, we first define a generic halt helper function that implements the infinite loop[cite: 123]. Then, we override the weak default handlers by defining strong C functions that print the contents of these status registers before halting the CPU[cite: 123]:

```
#include "cmsdk.h"
#include "kernel/kprintf.h"

/* Stop here forever. */
static void halt(void)
{
    for (;;) { }
}

void HardFault_Handler(void)
{
    kprintf("HardFault: HFSR=0x%x\n", SCB->HFSR);
    halt();
}

void MemManage_Handler(void)
{
    kprintf("MemManage: CFSR=0x%x\n", SCB->CFSR);
    halt();
}

void BusFault_Handler(void)
{
    kprintf("BusFault: CFSR=0x%x\n", SCB->CFSR);
    halt();
}

void UsageFault_Handler(void)
```

```
{
    kprintf("UsageFault: CFSR=0x%x\n", SCB->CFSR);
    halt();
}
```

3.17.3 Try It: Catching a Divide-By-Zero

We can immediately test if our new fault handlers work by intentionally crashing the OS[cite: 124].

In Week 2 (`Step 7`), we configured the CPU's `CCR` (Configuration and Control Register) to actively trap division by zero by setting the `DIV_0_TRP` bit. To test this, add the following code into `main.c` right after your banner print[cite: 124]:

```
volatile unsigned zero = 0;    /* Use volatile to force a runtime read */
kprintf("%u\n", 10u / zero);   /* This should trigger a hardware fault */
```

If you compile and run the project (`make run`), the program will halt and output the following to your terminal[cite: 125]:

```
UsageFault: CFSR=0x2000000
```

This output perfectly validates our entire architecture. The value `0x2000000` indicates that bit 25 of the `CFSR` is set[cite: 125]. If you check the hardware manual (or the `SCB_CFSR_DIVBYZERO` macro in `cmsdk.h`), bit 25 explicitly means "Divide by Zero"[cite: 125].

The sequence of events was:

1. The CPU detected the illegal division and instantly fired a `UsageFault`.
2. The CPU jumped to our custom `UsageFault_Handler`.
3. The handler successfully used the polled `kprintf` to push the error string byte-by-byte through the UART hardware to QEMU's terminal.
4. The handler called `halt()`, safely freezing the system.

After confirming the test works, remove the intentional divide-by-zero bug from `main.c`[cite: 125]. Every message generated by the OS—including critical hardware faults—is now officially routed through your custom-built UART driver[cite: 125].

3.17.4 Practical Note: Project Navigation and Exam Expectations

As we build these low-level layers, the codebase is becoming increasingly complex. Files are spread across different directories (`drivers/`, `kernel/`, `lib/`, `src/`), each serving a distinct architectural role.

From an academic and practical standpoint, it is critical to understand how to navigate this project structure. During an exam (or in a real-world debugging scenario), you will not be asked to write a full operating system from scratch in 90 minutes. Instead, you will typically be given an existing, structured project and asked to implement a specific function, fix a bug, or integrate a new hardware feature.

Mastering how to read the existing code, locate the correct files, and understand the flow of execution is just as important as understanding the hardware registers themselves.

`Troubleshooting tip`: When copying code or migrating functions between files, always ensure your `#include` directives are correct and that you do not forget initialization steps. For instance, if you forget to call `uart_init()` in `main` before using `kprintf`, the hardware remains disabled and nothing will print, leading to confusing debugging sessions where the system appears to freeze.

3.18 Unit 2: Calling Convention and Interrupts

Up to this point, our entire OS relies on **polling**. Our UART driver explicitly reads the hardware `STATE` register in an infinite loop to check if it can send or receive data.

3.18.1 The Limitations of Polling

While polling is simple to implement and works perfectly in restricted environments (like inside a `HardFault` handler where interrupts are strictly disabled), it is highly inefficient for a general-purpose operating system[cite: 127].

- **Wasted CPU Cycles:** A UART transmitting at 115,200 baud requires about 87 microseconds to send a single byte[cite: 127]. At our board's 25 MHz CPU clock speed, that equates to approximately 2,170 clock cycles[cite: 127]. With polling, the CPU spends all 2,170 cycles trapped in a useless `while` loop just waiting for the hardware to finish[cite: 127].

- **Latency:** The system's response time to a hardware event (like a key press) is limited by how often the CPU explicitly decides to check the register, which can cause lag or dropped inputs[cite: 127].

3.18.2 The Interrupt-Driven Alternative

To build a real operating system, we must transition from polling to **interrupts**[cite: 127].

Instead of constantly asking the device if it is ready, the CPU continues executing other useful application code (or goes to sleep to save power)[cite: 127]. When the device requires attention (e.g., a byte arrives on the UART), it raises a physical hardware interrupt signal[cite: 127]. The CPU instantly pauses its current task, jumps to a specific Exception Handler to service the device, and then seamlessly resumes its previous task[cite: 127].

While interrupts solve the CPU waste problem entirely, they introduce significant software complexity[cite: 127]. We must now manage asynchronous handlers, protect shared data variables, configure hardware priorities, and rigorously understand the ARM Architecture Procedure Call Standard (AAPCS) to ensure the CPU can jump between functions without corrupting memory[cite: 127].

3.18.3 Practical Note: Project Navigation and Exam Expectations

As we build these low-level layers, the codebase is becoming increasingly complex. Files are spread across different directories (`drivers/`, `kernel/`, `lib/`, `src/`), each serving a distinct architectural role.

From an academic and practical standpoint, it is critical to understand how to navigate this project structure. During an exam (or in a real-world debugging scenario), you will not be asked to write a full operating system from scratch in 90 minutes. Instead, you will typically be given an existing, structured project and asked to implement a specific function, fix a bug, or integrate a new hardware feature.

Mastering how to read the existing code, locate the correct files, and understand the flow of execution is just as important as understanding the hardware registers themselves.

`Troubleshooting tip`: When copying code or migrating functions between files, always ensure your `#include` directives are correct and that you do not forget initialization steps. For instance, if you forget to call `uart_init()` in `main` before using `kprintf`, the hardware remains disabled and nothing will print, leading to confusing debugging sessions where the system appears to freeze.

3.19 Unit 2: Calling Convention and Interrupts

Up to this point, our entire OS relies on **polling**. Our UART driver explicitly reads the hardware `STATE` register in an infinite loop to check if it can send or receive data.

3.19.1 The Limitations of Polling

While polling is simple to implement and works perfectly in restricted environments (like inside a `HardFault` handler where interrupts are strictly disabled), it is highly inefficient for a general-purpose operating system[cite: 127].

- **Wasted CPU Cycles:** A UART transmitting at 115,200 baud requires about 87 microseconds to send a single byte[cite: 127]. At our board's 25 MHz CPU clock speed, that equates to approximately 2,170 clock cycles[cite: 127]. With polling, the CPU spends all 2,170 cycles trapped in a useless `while` loop just waiting for the hardware to finish[cite: 127].
- **Latency:** The system's response time to a hardware event (like a key press) is limited by how often the CPU explicitly decides to check the register, which can cause lag or dropped inputs[cite: 127].

3.19.2 The Interrupt-Driven Alternative

To build a real operating system, we must transition from polling to **interrupts**[cite: 127].

Instead of constantly asking the device if it is ready, the CPU continues executing other useful application code (or goes to sleep to save power)[cite: 127]. When the

device requires attention (e.g., a byte arrives on the UART), it raises a physical hardware interrupt signal[cite: 127]. The CPU instantly pauses its current task, jumps to a specific Exception Handler to service the device, and then seamlessly resumes its previous task[cite: 127].

While interrupts solve the CPU waste problem entirely, they introduce significant software complexity[cite: 127]. We must now manage asynchronous handlers, protect shared data variables, configure hardware priorities, and rigorously understand the ARM Architecture Procedure Call Standard (AAPCS) to ensure the CPU can jump between functions without corrupting memory[cite: 127].

3.20 Polling vs. Interrupts: A Conceptual Overview

To fully appreciate why we must transition our architecture away from polling, we can use a real-world analogy.

Consider the interaction between a student and a professor regarding the start date of a university project:

- **The Polling Approach:** The student sends the professor an email every single day asking, "When can we start the project?" The professor continuously replies, "You need to wait." The student spends all their time and energy repeatedly asking the same question until, finally, the professor says, "You can start tomorrow." This is exactly how our current UART driver works: the CPU constantly asks the `STATE` register if it is ready, wasting millions of cycles doing nothing else.

- **The Interrupt Approach:** The student asks the professor once: "Can you let us know when the project starts?" The student then relaxes, goes out with friends, and enjoys their life. They do absolutely no waiting. One day, the student receives an email notification (the hardware interrupt) stating the project has started. The student immediately pauses their social life, handles the new task, and gets to work.

For a microprocessor, the interrupt approach means that while a very slow I/O device (like the UART) is transmitting data over a serial line, the CPU is completely free to run other application code or perform background tasks. It is the hardware's responsibility to notify the CPU the exact moment it needs attention.

3.20.1 Hardware Limitations

It is important to note that you cannot simply choose to use interrupts in software if the hardware does not support it. Interrupt capabilities must be physically designed into the silicon of the peripheral device. Fortunately, standard CMSDK devices—including our UARTs and Timers—are explicitly designed with interrupt lines physically connected to the Cortex-M processor.

3.21 Function Calls and the ARM Calling Convention

Before we can write the software to handle these interrupts, we must deeply understand how the C compiler translates function calls into assembly language.

When an interrupt fires, the CPU abruptly pauses the currently executing code to jump to an Exception Handler. To ensure that the interrupted code can resume later exactly as if nothing happened, we must guarantee that its working memory (specifically, the CPU registers) is not corrupted by the interrupt handler.

To manage this, ARM defines a strict set of rules known as the **AAPCS (ARM Architecture Procedure Call Standard)**. Every compiler (like GCC) strictly follows this standard to ensure that C functions can safely call each other without destroying data.

3.21.1 Register Roles in AAPCS

The AAPCS dictates exactly how the 16 core CPU registers must be used during a function call:

Registers	Role defined by AAPCS
R0 - R3	**Arguments and Return Values.** When calling a function, the first four arguments are placed in these registers. If the function returns a value, it is placed in `R0` (or `R0` and `R1` for 64-bit values). Crucially, these are **Caller-Saved** registers. A function is completely free to overwrite them without restoring their original values.
R4 - R11	**Local Variables (Callee-Saved).** These registers are used for long-term local variable storage inside a function. If a function wants to use them, it `must` save their original values to the stack at the beginning of the function and restore them before returning. The caller expects these to be exactly the same before and after the call.
R12	**Intra-Procedure Call Scratch Register.** Used primarily by the linker for complex subroutine branching. It is **Caller-Saved**.
R13 (SP)	**Stack Pointer.** Points to the top of the current stack.
R14 (LR)	**Link Register.** Contains the return address. It is **Caller-Saved**.
R15 (PC)	**Program Counter.** Points to the currently executing instruction.

Understanding the distinction between `Caller-Saved` (`R0-R3`, `R12`, `LR`) and `Callee-Saved` (`R4-R11`) registers is the fundamental key to understanding how Cortex-M processors handle interrupts seamlessly.

3.22 Writing Assembly Callable from C

Why do we need to study the AAPCS calling convention? In an operating system, there are moments when you must go lower than C and write direct assembly instructions.

If you write a function in pure assembly language and want to call it from your C code, you must manually implement it using the exact same standard the C compiler expects. If you break these rules, your system will crash.

3.22.1 Applying the Convention

Let us analyze how you must prepare an assembly function if you want to call it from C:

1. **Receiving Arguments:** If your C prototype defines arguments (e.g., `void my_func(int a, int b);`), you know automatically that `a` will be sitting in `R0` and `b` will be sitting in `R1` the moment your assembly code begins.

2. **Returning Values:** If your function returns an integer, you must physically copy that result into `R0` before executing the return branch (`bx lr`).

3. **Preserving Variables (Callee-Saved):** If your assembly code is complex and needs to use registers `R4` through `R11` to calculate things, you are the "Callee." The standard dictates that you must save these registers to the stack (using `push`) at the very start of your function, and restore them (using `pop`) immediately before returning. The C code that called you expects `R4-R11` to be completely untouched.

4. **Scratch Registers (Caller-Saved):** You are completely free to overwrite `R0-R3` and `R12` inside your assembly function without restoring them. The C compiler (the "Caller") knows these registers might get destroyed, so it saved anything important before calling you.

5. **Stack Alignment:** The AAPCS requires the Stack Pointer (`SP`) to be 8-byte aligned at all public interfaces. If your stack is misaligned when you call another function, you will trigger hardware faults.

To call a C function `from` assembly, the logic is reversed: you must put your arguments into `R0-R3`, save any of your own data that is in `R0-R3/R12` to the stack, and then use the `bl` (Branch with Link) instruction to jump to the C function.

3.23 Managing the CPU's Interrupt State

Before we can use specific hardware interrupts (like the UART), we must write low-level code to manage the CPU's global ability to listen to interrupts.

The Cortex-M CPU has a special hardware register called `PRIMASK`. This is a 1-bit register. When `PRIMASK` is set to 1, all configurable interrupts are globally masked (disabled), meaning the CPU will completely ignore them and continue executing its current code. When `PRIMASK` is set to 0, the CPU will listen to the interrupt lines.

Because PRIMASK is a special CPU core register (not a standard memory-mapped peripheral), it cannot be accessed using standard C variable assignments. It must be accessed using two specific assembly instructions:

- `mrs` (Move to Register from Special register): To read `PRIMASK`.
- `msr` (Move to Special register from Register): To write to `PRIMASK`.

3.23.1 Designing the OS Interrupt API

As operating system designers, we need to create high-level C functions that wrap these assembly instructions. We could simply write two functions: `enable_interrupts()` and `disable_interrupts()`. However, this is dangerous in a complex OS.

Imagine a piece of kernel code that needs to briefly disable interrupts to safely update a shared variable, and then re-enable them. If it blindly calls `disable` and then `enable`, it assumes interrupts were enabled when it started. But what if this code was called from somewhere where interrupts were `already` disabled? By blindly calling `enable` at the end, it breaks the system's expected state.

To solve this, we design a smart API using two functions:

1. `uint32_t irq_save(void);`
   This function reads the current state of `PRIMASK`, globally disables interrupts, and returns the old state.

2. `void irq_restore(uint32_t state);`
   This function takes the state we saved earlier and writes it back to `PRIMASK`, restoring the CPU exactly to whatever state it was in before.

This allows kernel code to safely say: `"I don't know if interrupts are currently enabled or not, but I want to be completely sure they are disabled right now to do my job safely. When I am done, put the system back exactly how it was."`

```
uint32_t state = irq_save(); /* disable interrupts, save old state */
/* safely modify shared variables here */
irq_restore(state);          /* restore old state */
```

3.24 Implementing `irq_save` and `irq_restore` in Assembly

To practice applying the AAPCS calling convention, we will implement these two functions directly in assembly language. We place this code in a file named `arch/irq.s`.

3.24.1 Implementing `irq_save`

The goal of `irq_save` is to read the current `PRIMASK` state, return it to the caller, and disable interrupts.

According to the AAPCS, because this C function (`uint32_t irq_save(void);`) takes no arguments but returns a 32-bit integer, we must leave the resulting `PRIMASK` state strictly inside register `R0` before we branch back to the caller.

```
    .syntax unified
    .thumb
    .text

    .global irq_save
    .type irq_save, %function
irq_save:
    /* Read PRIMASK into R0 (this is our return value) */
    mrs r0, PRIMASK

    /* Disable interrupts (Change Processor State, Interrupt Disable) */
    cpsid i

    /* Return to caller. The return value is safely in R0. */
    bx lr
```

`Code Breakdown:`

- `.global irq_save`: Tells the linker that this symbol should be visible to C code.
- `.type irq_save, %function`: Declares this symbol as executable code.
- `mrs r0, PRIMASK`: The `mrs` instruction copies the special `PRIMASK` register into a standard register (`R0`).
- `cpsid i`: This dedicated instruction explicitly sets `PRIMASK` to 1, immediately masking all configurable interrupts.
- `bx lr`: Branches back to the address stored in the Link Register, returning execution to C.

3.24.2 Implementing `irq_restore`

The goal of `irq_restore` is to take a previously saved state and write it back to `PRIMASK`.

According to the AAPCS, because this C function (`void irq_restore(uint32_t state);`) accepts one argument, the compiler will pass that argument to our assembly code strictly in register `R0`.

```
    .global irq_restore
    .type irq_restore, %function
irq_restore:
    /* Write R0 (our first argument) into PRIMASK */
    msr PRIMASK, r0

    /* Return to caller */
    bx lr
```

`Code Breakdown:`

- `msr PRIMASK, r0`: The `msr` instruction moves the value from `R0` (the saved state) directly into the special `PRIMASK` register. If the saved state was 0, interrupts turn back on; if it was 1, they remain disabled.

While these could technically be implemented using inline assembly inside a C function, putting them in a dedicated `.s` file acts as an excellent exercise to prove that C and Assembly can interface perfectly as long as you respect the AAPCS rules.

3.24.3 Try It: Testing the API in C

To use these newly crafted assembly functions, we define their prototypes in `kernel/irq.h`:

```
#ifndef IRQ_H
#define IRQ_H
#include <stdint.h>

uint32_t irq_save(void);
void irq_restore(uint32_t state);

#endif
```

Now we can navigate to our `main.c` application code, include `kernel/irq.h`, and test if our assembly functions successfully manipulate the hardware state.

```
#include "drivers/uart.h"
#include "kernel/kprintf.h"
#include "kernel/irq.h"

int main(void)
{
    uart_init();

    /* Save current state and disable interrupts */
    uint32_t old_state = irq_save();

    /* Right here, we are absolutely sure interrupts are masked. */

    /* Restore the state back to what it was */
    irq_restore(old_state);

    /* Print the old state to see what it was initially */
    kprintf("PRIMASK before irq_save was: %d\n", old_state);

    for (;;) { }
}
```

When you compile and execute this, the UART should print: `PRIMASK before irq_save was: 0`. This confirms that our boot process successfully started the CPU with interrupts enabled (`PRIMASK = 0`), and that our custom assembly layer accurately captured and restored that state.

3.24.4 The Philosophy of OS Design

When we ran our test in the previous section, the terminal outputted `PRIMASK before irq_save was: 0`. This confirms that the initial state of the interrupt mask was indeed 0 (enabled), our assembly function safely disabled it, and then cleanly restored it.

This pattern reveals the core philosophy of operating system design. We take a complex, highly architecture-specific concept (like manipulating the CPU's `PRIMASK` using inline assembly), wrap it inside a robust, well-tested C function (`irq_save`), and then use that simple wrapper everywhere else. The more you organize your wrappers hierarchically, the easier your job becomes: once the low-level core functions work flawlessly, you build the next layers entirely on top of them without worrying about the underlying hardware mechanics.

3.25 ARM Exception Entry and Return Mechanics

While we can wrap many features in simple C functions, handling exceptions (interrupts) requires a deep understanding of what the Cortex-M hardware does autonomously behind the scenes.

When a hardware interrupt fires and the CPU pauses the current execution to jump to an Exception Handler, it performs several complex operations automatically:

3.25.1 The Automatic Hardware State Save (Stacking)

To respect the AAPCS calling convention and ensure the interrupted code can resume later seamlessly, the CPU hardware automatically pushes **eight words** (32 bytes) onto the current stack before executing the very first instruction of your C handler.

These eight words consist of the Caller-Saved registers: `R0`, `R1`, `R2`, `R3`, `R12`, the Link Register (`LR`), the Program Counter (`PC`), and the Program Status Register (`xPSR`). If you ever write a low-level exception handler in pure assembly, you must account for the fact that the Stack Pointer (`SP`) has already been modified by the hardware by the time your code begins.

3.25.2 The `EXC_RETURN` Value

In standard C functions, the Link Register (`LR`) holds the exact memory address to return to when the function finishes. However, when entering an Exception Handler, the CPU hardware overwrites the `LR` with a special 32-bit code called the `EXC_RETURN` value (which always starts with `0xFFFFFF...`).

This special code is crucial because exceptions can interrupt different execution contexts. For example, an interrupt might happen while the CPU is in User Mode running an application, or it might happen while the CPU is already in Supervisor Mode handling a different, lower-priority exception.

When the handler finishes and executes a return instruction, the CPU catches this `0xFFFFFF...` code. Instead of trying to jump to a literal memory address, the CPU decodes it to determine exactly how to return:

- Which execution mode to return to (Handler mode vs. Thread mode).
- Which stack pointer to restore the eight words from (Main Stack Pointer vs. Process Stack Pointer).

While we will rely on the C compiler to generate the correct return instructions for our basic handlers, you must be aware of this mechanism when debugging or writing advanced context-switching code.

3.26 The Heartbeat of the OS: The SysTick Timer

We are now ready to start using hardware interrupts, and we will begin with the most critical device for any operating system: the **SysTick** timer.

Unlike external peripherals (like the UART or GPIO), the SysTick timer is built directly into the Cortex-M core itself. Every modern operating system—from our minimal OS up to Linux—relies on this specific hardware timer to give a sense of time to the system.

3.26.1 The Necessity of the System Tick

The OS configures the SysTick timer to generate a hardware interrupt at a fixed, strictly periodic frequency (for example, every 30 milliseconds). But why is this so fundamental?

Imagine running a user application that contains an infinite loop:

```
while(1)
{
    /* do nothing forever */
}
```

If an OS relies entirely on cooperative software (where applications voluntarily yield the CPU back to the kernel), this infinite loop would permanently hijack the CPU. The computer would become completely unresponsive, and the OS would be deadlocked.

The SysTick timer solves this through **hardware preemption**. Because the timer is a hardware device wired directly to the CPU's interrupt lines, its periodic interrupt will physically force the CPU to pause the infinite loop and jump into the kernel's SysTick Exception Handler. This guarantees that no matter what user applications do, the operating system kernel is physically guaranteed to regain control of the CPU at a fixed interval to perform scheduling and manage the system.

3.27 Hardware Preemption and the System Tick

If a computer's operating system relied entirely on cooperative software—where applications voluntarily yield the CPU back to the kernel—an infinite loop in a user program would permanently hijack the CPU. The computer would become completely unresponsive, and the OS would be deadlocked.

The SysTick timer solves this through **hardware preemption**. By programming the timer to fire an interrupt periodically (e.g., every 30 milliseconds), the OS ensures

that whatever code is currently running on the CPU is forcefully paused. The CPU is then physically forced to jump into the kernel's Exception Handler.

This guarantees that the operating system kernel regains control of the CPU at fixed intervals. Once inside the kernel, the OS can:

- Perform **scheduling**: give control to different programs fairly so they appear to run in parallel.
- Detect if a program has been running for too long and forcefully terminate it.
- Maintain the system clock by tracking how many ticks have occurred since boot.

A hardware interrupt is the `only` mechanism powerful enough to definitively stop the CPU and guarantee that the operating system can perform its tasks.

3.27.1 The SysTick Hardware Registers

The SysTick timer is a small piece of hardware mapped into memory. Like the UART, its registers are already mapped into a C data structure in our CMSDK headers (`SysTick_Type`).

The primary registers are:

- `0x10 SYST_CSR`: Control and Status Register. Used to enable the timer and its interrupt.
- `0x14 SYST_RVR`: Reload Value Register. This dictates the frequency of the tick.
- `0x18 SYST_CVR`: Current Value Register. The actual counting mechanism.

To configure the timer to fire exactly every T milliseconds, you must consult the manual and calculate the reload value based on your CPU's clock frequency. For our board running at 25 MHz, firing an interrupt every 10 milliseconds requires programming `SYST_RVR` with the value `249,999`. While the exact calculation changes depending on the specific board, the architectural concept remains identical across all Cortex-M devices.

3.28 Building the SysTick Driver

To integrate this hardware into our OS, we write a device driver just like we did for the UART.

3.28.1 The API Definition (`systick.h`)

The driver API exposes initialization and time-tracking functions, as well as the prototype for the hardware interrupt handler itself:

```
void systick_init(unsigned int period_ms);
unsigned int systick_get(void);
void SysTick_Handler(void); /* The actual interrupt handler */
```

3.28.2 The Implementation (`systick.c`)

Inside the C file, we need a global variable to track the number of ticks that have occurred since the system booted.

```
#include "drivers/systick.h"
#include "cmsdk.h"

#define SYSTEM_CORE_CLOCK 25000000

/* Global counter to track ticks since boot */
static volatile unsigned int systick_counter = 0;
```

The initialization function computes the correct reload value mathematically and configures the hardware:

```
void systick_init(unsigned int period_ms)
{
    /* Compute reload value based on CPU clock speed */
    unsigned int reload = (SYSTEM_CORE_CLOCK / 1000) * period_ms - 1;

    SysTick->LOAD = reload;
    SysTick->VAL  = 0;

    /* Enable the timer and its interrupt line */
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                    SysTick_CTRL_TICKINT_Msk |
                    SysTick_CTRL_ENABLE_Msk;
}
```

We then provide a simple getter function so the rest of the kernel can ask what time it is (in ticks):

```
unsigned int systick_get(void)
{
    return systick_counter;
}
```

3.28.3 The Exception Handler

Finally, we must implement the code that actually runs when the hardware interrupt fires. The `SysTick_Handler` function increments our global counter. Because this is a hardware exception, it also serves as an excellent place to prove the system is working by printing a message:

```
#include "kernel/kprintf.h"

void SysTick_Handler(void)
{
    /* Increment the global tracker */
    systick_counter++;
```

```
    /* Print a heartbeat message to the terminal */
    kprintf("Tick: %u\n", systick_counter);
}
```

`Crucial Step`: Defining this function is not enough. For the CPU to actually jump to this code, the address of `SysTick_Handler` must be correctly placed into the 15th slot of the CPU's Vector Table in our `boot.s` startup code. If the table is missing this entry, the CPU will jump to the default `while(1)` loop and the system will freeze on the very first tick.

3.28.4 The `volatile` Variable and the Uptime Calculation

In our implementation, the variable tracking the number of ticks must absolutely be declared as `volatile`.

Because the main CPU thread never writes to this variable (it only reads it via the getter function), the C compiler's optimizer will assume its value never changes. Without `volatile`, a loop waiting for the time to reach a certain value would be optimized into an infinite spin lock. The `volatile` keyword informs the compiler that this variable can be modified at any time by the hardware interrupt handler.

Additionally, in our final implementation, we will track the system `uptime` in milliseconds rather than raw ticks.

```
static volatile unsigned int systick_uptime_ms = 0;
static unsigned int systick_period_ms = 0;

void systick_init(unsigned int period_ms)
{
    systick_period_ms = period_ms;

    unsigned int reload = (SYSTEM_CORE_CLOCK / 1000) * period_ms - 1;
    SysTick->LOAD = reload;
    SysTick->VAL  = 0;
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                    SysTick_CTRL_TICKINT_Msk |
                    SysTick_CTRL_ENABLE_Msk;
}

unsigned int systick_get_ms(void)
{
    return systick_uptime_ms;
}

void SysTick_Handler(void)
{
    systick_uptime_ms += systick_period_ms;
}
```

`Implementation Detail`: The hardware manual specifies a skew of one clock cycle when configuring the reload register. Therefore, the mathematical formula strictly requires subtracting 1 after multiplying by the frequency. Furthermore, the

`SysTick_CTRL_CLKSOURCE_Msk` flag explicitly tells the timer to use the internal CPU clock as its timing source, rather than an external crystal oscillator.

3.28.5 Try It: Proving the Hardware Interrupt Works

To prove that our SysTick timer is functioning and successfully firing interrupts in the background, we can test it in our `main` application loop.

First, we must include the proper driver headers (`drivers/systick.h` and `drivers/uart.h`). Then, we initialize the UART and initialize the SysTick timer to fire an interrupt every 10 milliseconds.

We then enter an infinite loop that simply reads the current uptime and prints it via `kprintf`.

```
#include "drivers/uart.h"
#include "drivers/systick.h"
#include "kernel/kprintf.h"

int main(void)
{
    uart_init();
    systick_init(10); /* Fire interrupt every 10 ms */

    for (;;)
    {
        kprintf("Uptime: %u ms\n", systick_get_ms());
    }
}
```

When you compile and run this code (`make run`), your terminal will output the uptime value repeatedly. Because `kprintf` relies on our polled UART driver—which is incredibly slow at transmitting characters over the serial line—the CPU will spend a significant amount of time trapped inside the `kprintf` logic.

Despite the CPU being stuck inside an infinite loop continuously running `kprintf`, the numbers printed on the screen will actively increment (e.g., `Uptime:  0 ms`, ..., `Uptime:  10 ms`, ..., `Uptime:  20 ms`).

This proves that every 10 milliseconds, the hardware SysTick timer is forcefully pre-empting the CPU, pausing the `kprintf` execution, jumping to `SysTick_Handler` to increment the `systick_uptime_ms` variable, and then smoothly returning the CPU to exactly where it left off inside `kprintf`.

You have successfully built the fundamental hardware beating heart of an operating system. This periodic interrupt will become the foundation of our multitasking scheduler in the upcoming units.

3.28.6 A Remark on Emulated Time vs. Real Time

Before moving on, it is important to note a caveat about running an operating system inside an emulator like QEMU.

When you run the SysTick test on physical hardware, setting the timer to 10 milliseconds guarantees a hardware interrupt exactly every 10 milliseconds of real-

world wall-clock time. In QEMU, the execution is functionally and architecturally perfect: the emulator strictly counts the exact number of Cortex-M3 clock cycles.

However, QEMU is just a software application running on a host operating system (like Linux or Windows). If your host CPU temporarily pauses QEMU to refresh a web browser or update a presentation, the emulator loses real-world time. Therefore, while the internal logic and register states are perfectly accurate, you cannot rely on QEMU to measure precise real-time hardware performance.

3.29 The Nested Vectored Interrupt Controller (NVIC)

Until now, we have handled the SysTick timer, which is an internal core exception. But what about external peripheral interrupts, like the UART or the timers on the APB bus?

All external hardware interrupts in an ARM Cortex-M processor are centrally managed by a dedicated, memory-mapped I/O device called the **NVIC (Nested Vectored Interrupt Controller)**.

The NVIC sits between the peripheral devices and the CPU core. It acts as a gatekeeper and priority manager. You control the NVIC by reading and writing to its specific memory-mapped registers:

- `ISER` **(Interrupt Set-Enable Register):** Used to enable specific interrupt lines. Writing a 1 to bit N enables IRQ line N.
- `ICER` **(Interrupt Clear-Enable Register):** Used to disable specific interrupt lines.
- `IPR` **(Interrupt Priority Registers):** Used to assign a priority level to each interrupt. If two interrupts fire simultaneously, or if an interrupt fires while the CPU is already handling another one, the NVIC uses these registers to determine which one preempts the other (hence "Nested").

3.29.1 The Golden Rule of Interrupt Handlers

When a hardware device requires attention, it asserts its interrupt line. The NVIC detects this, records that the interrupt is `pending`, and forces the CPU to jump to your handler.

This introduces a critical complexity: **your software must acknowledge and clear the interrupt reason before returning**. Usually, this means reading the device's data register or writing a 1 to a specific "Interrupt Clear" bit in the device's status register.

If your handler finishes and executes its return instruction `without` clearing the hardware flag, the NVIC will see that the line is still asserted. The very microsecond your CPU attempts to return to the main program, the NVIC will instantly fire the interrupt again. Your system will become permanently trapped in an infinite loop of re-entering the exact same interrupt handler, completely starving the rest of the OS.

3.30 The Three Gates of Interrupts

To successfully transition a device like our UART from polling to interrupts, we must coordinate multiple layers of the architecture. An external device's interrupt will only execute its handler if a strict set of conditions—the three gates—are all simultaneously true:

1. **The Device Gate:** The peripheral itself must be configured to generate interrupts. For example, in the UART's `CTRL` register, you must explicitly set the `TXINTEN` (Transmit Interrupt Enable) or `RXINTEN` (Receive Interrupt Enable) bits.

2. **The NVIC Gate:** The specific IRQ line connecting that device to the CPU must be unmasked in the NVIC. For UART0 (which lives on IRQ 0 and 1), you must write the corresponding bits in the NVIC's `ISER` register.

3. **The CPU Gate:** The CPU's global interrupt mask (`PRIMASK`) must be set to 0 (enabled), meaning the core is actively listening to the NVIC.

By mastering these three layers, we unlock the ability to write highly efficient, asynchronous device drivers that let the CPU sleep or run applications while the hardware does the heavy lifting in the background.

3.30.1 Shared Interrupt Lines (Q&A)

During the lecture, a common question arises: `Is there strictly one device per interrupt line?`

From the CPU's perspective, an interrupt line is just a single physical wire (a bit). In most commercial Cortex-M boards, hardware designers map exactly one device to one line to keep software simple. However, if a complex board has more peripherals than available NVIC lines, hardware designers might physically wire multiple devices to the same IRQ line.

If this happens, the CPU hardware has no way of knowing which specific device triggered the line. The software interrupt handler must act as a dispatcher: it must manually read the status registers (poll) of every device sharing that line to figure out who raised the flag, service the correct one, and clear it. While perfectly legal, it reintroduces a layer of polling inside the handler, so it is usually avoided unless hardware pin constraints demand it.

3.31 Expanding the Vector Table for External Devices

To finally make our UART interrupts work, we must connect the hardware signals to our software functions. This requires expanding the CPU's Interrupt Vector Table that we initially created in Week 2.

Recall that the first 16 slots (indices 0 to 15) of the vector table are strictly reserved for the ARM core's internal exceptions (e.g., Initial SP, Reset, HardFault, SysTick). External peripheral interrupts, which are managed by the NVIC, begin immediately after this reserved block.

Therefore, a device connected to NVIC IRQ line N must have its handler address placed at slot $16 + N$ in the vector table.

Vector Slot	IRQ Number	Handler / Device
15	-1	`SysTick_Handler` (Core Exception)
16	IRQ 0	`UART0_RX_Handler` (External)
17	IRQ 1	`UART0_TX_Handler` (External)
18	IRQ 2	`UART1_RX_Handler` (External)

To support our UART, we must update our startup code (whether in `boot.s` or a C-based vector table array) to include the specific addresses of our new handlers:

```
/* Declare the handler prototypes */
void SysTick_Handler(void);
void UART0_RX_Handler(void);
void UART0_TX_Handler(void);

/* Inside the vector table array structure: */
(void *)SysTick_Handler,     /* Slot 15: Core Timer */
(void *)UART0_RX_Handler,    /* Slot 16: IRQ 0 */
(void *)UART0_TX_Handler,    /* Slot 17: IRQ 1 */
```

With the vector table fully populated, the NVIC configured, the device interrupts enabled, and the CPU's `PRIMASK` cleared, the hardware is fully armed. The moment a character arrives over the serial line, the CPU will autonomously pause its current task and jump directly to `UART0_RX_Handler`.

3.32 Looking Ahead: System Calls and User Mode

By successfully configuring the SysTick timer and mastering the interrupt controller, we have built the foundational hardware layer of our operating system. However, at this point in our project, all our software—including our test applications—is running entirely in privileged kernel mode (Supervisor Mode).

In the next unit, we will finalize our asynchronous interrupt architectures and establish the boundary between Kernel Space and User Space. We will wrap our powerful hardware capabilities (like our UART printing) behind secure **System Calls**, ensuring that user-level programs can safely request OS services without ever being allowed to touch the raw hardware registers directly.

Chapter 4

Exercises

4.1 week-01-Lab

4.1.1 Exercise A2: Construct the Makefile

The starter directory contains no `Makefile`. The goal of this exercise is to construct one from scratch. Instead of listing generic Make concepts, we will dissect the three most critical and bare-metal-specific mechanisms used in this configuration.

Deep Dive: LDFLAGS, the Map File, and QEMU

1. LDFLAGS and Their Importance `LDFLAGS` governs the memory layout and the final link[cite: 17]. In a bare-metal environment, the linker cannot simply guess where to put things. We use these flags to sever all ties with the host OS and enforce our own rules:

- `-T scripts/mps2_m3.ld`: Tells the linker to use our custom script to map the physical Flash and RAM regions.
- `-nostartfiles`: A crucial flag that prevents the toolchain's default C startup objects from taking control, forcing the CPU to use our custom `Reset_Handler`[cite: 17].
- `-nostdlib`: Completely strips out standard C libraries (no `printf`, no `malloc`).
- `-lgcc`: Explicitly links back GCC's internal helper library, which provides low-level math routines (like 64-bit division) that the Cortex-M3 lacks in hardware.
- `-Wl,-Map=...`: The `-Wl,` prefix passes the subsequent command directly to the underlying linker (`ld`), instructing it to generate the `output.map` file.

2. What the output.map File Actually Does The map file is the definitive "receipt" of your compilation. It is an invaluable document that explains exactly where the linker physically placed every single input contribution and symbol in the hardware's memory[cite: 14]. If your program crashes, the `output.map` tells you exactly which function or variable lives at the physical address that caused the fault. For example, it proves that your vector table was placed exactly at Flash address zero, and your variables were mapped to SRAM[cite: 14].

3. The QEMU Command and Its Parameters Running a bare-metal image requires setting up a highly specific software emulator environment. Here is every parameter of the `run` command explained:

- `qemu-system-arm`: The emulator executable for ARM architectures.
- `-machine mps2-an385`: Tells QEMU which specific hardware board to emulate, ensuring the CPU and memory map exactly match our Cortex-M3 target[cite: 21].
- `-kernel $(BIN)`: Passes our generated ELF executable image to the emulator to be loaded into memory[cite: 21].
- `-nographic`: Disables the graphical emulation window since we only need the terminal output[cite: 22].
- `-monitor null -serial stdio`: Detaches QEMU's default internal monitor and routes the emulated serial port directly to the host's terminal. This specific routing is why hitting `Ctrl-C` successfully terminates the emulator, rather than being intercepted by QEMU's multiplexer.
- `-semihosting` and `-semihosting-config enable=on,target=native`: Enables the semihosting interface. This allows QEMU to intercept our `bkpt 0xab` instructions and route the text payload directly to our laptop's console[cite: 22].

Here is the complete and correct `Makefile` solution for this project:

```
CC := arm-none-eabi-gcc
BUILD := build
BIN := $(BUILD)/scratch-os.axf

# Compilation Flags
CFLAGS := -mthumb -mcpu=cortex-m3 -nostartfiles -std=gnu17
CFLAGS += -Wall -Wextra -O0 -g3

# Linker Flags: script, memory map, and standard library removal
LDFLAGS := -T scripts/mps2_m3.ld -nostdlib -nostartfiles -lgcc
LDFLAGS += -Wl,-Map=$(BUILD)/output.map

# Source files (C and Assembly)
C_SOURCES   := src/main.c
ASM_SOURCES := boot/startup.s

# Automatic generation of object file paths (.o)
C_OBJECTS   := $(C_SOURCES:%.c=$(BUILD)/%.o)
ASM_OBJECTS := $(ASM_SOURCES:%.s=$(BUILD)/%.o)
OBJECTS     := $(C_OBJECTS) $(ASM_OBJECTS)

.PHONY: all clean run debug

all: $(BIN)
```

```
# Pattern rule to compile C files
$(BUILD)/%.o: %.c
        @mkdir -p $(dir $@)
        $(CC) $(CFLAGS) -c $< -o $@

# Pattern rule to compile ASM files
$(BUILD)/%.o: %.s
        @mkdir -p $(dir $@)
        $(CC) $(CFLAGS) -c $< -o $@

# Rule to link everything into the final executable (.axf) and create the
$(BIN): $(OBJECTS)
        $(CC) $(CFLAGS) $(LDFLAGS) $(OBJECTS) -o $(BIN)

# Rule to start the emulator
run: $(BIN)
        qemu-system-arm -machine mps2-an385 -monitor null -semihosting \
                --semihosting-config enable=on,target=native \
                -kernel $(BIN) -serial stdio -nographic

clean:
        rm -rf $(BUILD)
```

Checkpoint: Running `make run` successfully prints `toolchain OK` from the newly constructed `Makefile`.

4.1.2 Exercise A3: Hosted Environment and Inline Assembly

Our main operating system is being built in a **freestanding** (bare-metal) environment. As established, this means there is no standard C library (`libc`) and no `printf` available to print to a console[cite: 9, 17].

However, to develop and test our first assembly routine, this specific exercise is linked as a **hosted** program. This temporarily provides access to `libc` and `printf`, allowing us to print the variables and verify that our assembly code actually works before injecting it into the blind bare-metal firmware.

Deep Dive: Semihosting Initialization

When running the starter code, the program executes completely but prints absolutely nothing. This is not a crash. While the C library is present in this hosted build, its standard I/O file descriptors (`stdout`, `stdin`) are disconnected by default. When `printf` attempts to output text, the data is silently discarded.

To fix this, we must explicitly instruct Newlib (the embedded C library) to connect its file handles to the QEMU semihosting interface. This is achieved by declaring and calling `initialise_monitor_handles()` at the very beginning of `main`. Once connected, QEMU can intercept the semihosting trap and write the requested string directly to the host console[cite: 21].

Deep Dive: Inline Assembly Mechanics

The exercise requires writing a function to swap two integers directly in ARM Assembly. This exposes several low-level mechanics:

- **Load/Store Architecture:** ARM processors cannot directly manipulate values in RAM. You must load the values from memory into CPU registers (using `ldr`), manipulate them, and store them back (using `str`).
- **Calling Convention:** When C calls our assembly function, it passes the arguments via registers. The pointer `a` arrives in `r0`, and the pointer `b` arrives in `r1`. We can use `r2` and `r3` as temporary scratch registers.
- **The `naked` Attribute:** The GCC `__attribute__( ( naked ) )` directive strips away the standard C function prologue and epilogue. The compiler will not generate the hidden instructions that usually save and restore the stack. Because the epilogue is missing, we must manually return to the caller using the `bx lr` (Branch and eXchange to Link Register) instruction.
- **Explicit Newlines (`\n`):** C concatenates adjacent string literals at compile time (`"a" "b"` becomes `"ab"`). If we omit the `\n` inside the `__asm__` block, the assembler receives a single, run-together line (e.g., `ldr r2,[r0]ldr r3,[r1]`) and throws a garbage error. The newline tells the assembler where one instruction ends and the next begins.
- **Unused Parameter Warnings:** When compiling the assembly function, GCC warns that parameters `a` and `b` are unused. This is expected. The C compiler treats the inline assembly block as an opaque string. It does not parse the assembly to see that we are manually reading `r0` and `r1`, so it assumes the C variables were ignored.

The Complete Implementation

Here is the completed `swap.c` file demonstrating both the C and Assembly implementations:

```
#include <stdio.h>

/* Establish the file descriptors that stdio writes through for semihosti
extern void initialise_monitor_handles( void );

/* Exchange the values *a and *b, in C. */
void swap_c( int *a, int *b )
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

/* Exchange the values *a and *b, in ARM assembly. */
__attribute__( ( naked ) ) void swap_asm( int *a, int *b )
```

```
{
    __asm__ volatile (
        "ldr r2, [r0]\n"     /* r2 = *a (Load value from address r0) */
        "ldr r3, [r1]\n"     /* r3 = *b (Load value from address r1) */
        "str r3, [r0]\n"     /* *a = r3 (Store r3 into address r0) */
        "str r2, [r1]\n"     /* *b = r2 (Store r2 into address r1) */
        "bx lr\n"            /* Return to caller */
    );
}

int main( void )
{
    int x, y;

    /* Initialize semihosting to make printf work */
    initialise_monitor_handles();

    x = 1; y = 2;
    printf( "C   before: x=%d y=%d\n", x, y );
    swap_c( &x, &y );
    printf( "C   after:  x=%d y=%d\n", x, y );

    x = 1; y = 2;
    printf( "asm before: x=%d y=%d\n", x, y );
    swap_asm( &x, &y );
    printf( "asm after:  x=%d y=%d\n", x, y );

    return 0;
}
```

Checkpoint: Running `make run` successfully prints four lines to the terminal, proving that both the C and Assembly implementations successfully swap the values using the exact same underlying logic.

4.1.3 Exercise A4: Transfer to the Freestanding Image

The final step is to transfer our verified `swap_asm` function from the hosted environment back into our bare-metal firmware image (`starter/src/main.c`).

If we blindly copy the assembly function, include `<stdio.h>`, and try to print the result using `printf` just like we did in Exercise A3, the build will abruptly fail. This failure is intentional and highly educational.

Deep Dive: Diagnosing the Linker Error

The compiler successfully translates the C code, but the build crashes during the **link** phase with the following error:

```
src/main.c:129:(.text+0x6c): undefined reference to `printf'
collect2: error: ld returned 1 exit status
```

Why does this happen? The two programs contain the exact same function, compiled for the same processor, by the same compiler, and executed by the same QEMU command. The critical difference lies entirely in **how they were linked**, which dictates their execution environment:

- **Hosted Environment (Exercise A3):** Linked with `-specs=rdimon.specs`. This explicitly supplies Newlib's semihosting C library and its startup code.
- **Freestanding Environment (Exercise A4):** Compiled with `-ffreestanding` and linked with `-nostartfiles -nostdlib`[cite: 9, 11]. Because of `-nostdlib`, no standard C library is searched at all, leaving `printf` with no definition to resolve to[cite: 9].

Even if we temporarily removed `-nostdlib` to force the linker to find `printf`, the link would still fail later on missing system call stubs like `_write`, `_read`, and `_sbrk`. Our bare-metal `Reset_Handler` simply sets up RAM and calls `main`[cite: 10]; it performs none of the complex run-time initializations that standard I/O strictly requires.

ISO C Standard: Hosted vs. Freestanding

The ISO C Standard (Section 4) formally names these two states:

- **Hosted Implementation:** Provides the full standard library and relies on an underlying operating system for startup and system calls.
- **Freestanding Implementation:** Provides no standard library[cite: 9]. The program must supply its own startup code and can only rely on a few hardware-agnostic headers (like `stdint.h`)[cite: 9].

We can visually prove the massive overhead of a hosted environment by checking the file sizes. Running `arm-none-eabi-size` on both images will reveal that the hosted `swap.axf` is significantly larger than our custom, freestanding `scratch-os.axf`, purely because of the bloated `libc` baggage.

Empirical Proof: The Size Cost of `libc`

By running the size profiling tool on both executable images, we can visually prove the massive overhead of a hosted environment[cite: 17]:

```
$ arm-none-eabi-size hosted/build/swap.axf
   text    data     bss     dec     hex filename
  45180    2328    5568   53076    cf54 swap.axf

$ arm-none-eabi-size build/scratch-os.axf
   text    data     bss     dec     hex filename
    224       0    4096    4320    10e0 scratch-os.axf
```

The numbers speak for themselves. The hosted program occupies over 45,000 bytes of instruction memory (`text`). In stark contrast, our custom bare-metal OS occupies merely 224 bytes[cite: 17]. That massive difference is the precise "tax" of the standard C library (`libc`) and the `printf` machinery.

The Freestanding Workaround and Final Code

Because we cannot easily use `printf` to format dynamic strings in our freestanding OS, we must adapt our verification strategy. We use standard C logic to verify the swap internally, and then use our custom `semihost_write0` function to print a hardcoded, NUL-terminated status string.

Troubleshooting Note: If the compiler throws an `implicit declaration of function 'swap_asm'` error, it means the function was called inside `main` before the compiler ever saw its definition. In C, the function body (or its prototype) must be placed *above* the `main` function to be recognized.

Here is the complete, integrated `src/main.c` file for our freestanding OS:

```
#include <stdint.h>

/* Emit a NUL-terminated string over the ARM semihosting debug channel. *
static void semihost_write0( const char *message )
{
    register uint32_t r0 __asm__( "r0" ) = 0x04;
    register const char *r1 __asm__( "r1" ) = message;
    __asm__ volatile ( "bkpt 0xab" : : "r"( r0 ), "r"( r1 ) : "memory" );
}

/* Custom inline assembly routine to swap two values */
__attribute__( ( naked ) ) void swap_asm( int *a, int *b )
{
    __asm__ volatile (
        "ldr r2, [r0]\n"
        "ldr r3, [r1]\n"
        "str r3, [r0]\n"
        "str r2, [r1]\n"
        "bx lr\n"
    );
}

int main( void )
{
    semihost_write0( "toolchain OK\n" );

    int x = 1 , y = 2 ;
    swap_asm ( &x , &y ) ;

    /* Verify logic: if they started as 1 and 2, they must now be 2 and 1
    semihost_write0((x == 2 && y == 1) ? "swap OK\n" : "swap FAILED\n");

    return 0;
}
```

Checkpoint: Running `make run` successfully prints `toolchain OK` followed by `swap OK`, proving that our bare-metal system successfully executed custom ARM assembly without needing any underlying C library.

4.1.4 Exercise B1: Extending the Makefile and the Dependency Graph

Extending Targets: `size` and `disasm`

To deeply analyze the compiled image, two new action targets are added to the Makefile. Because they execute analysis commands (`size` and `objdump`) rather than creating literal output files, they must be strictly appended to the `.PHONY` declaration.

```
.PHONY: all clean run debug size disasm

size: $(BIN)
	arm-none-eabi-size $(BIN)

disasm: $(BIN)
	arm-none-eabi-objdump -d $(BIN)
```

Rebuild Behavior and Timestamp Dependencies

Make uses the last modified timestamp of files to decide whether a target requires rebuilding. Initial logic and parameters are validated through the following terminal observations:

1. **Invoke `make` with nothing modified:** Make reports `Nothing to be done for 'all'`. The `.axf` target is already newer than all its registered prerequisites, so the build is skipped entirely.

2. **`touch src/main.c` then `make`:** Make compiles only `build/src/main.o` and executes the link phase.

3. **`touch boot/startup.c` then `make`:** Make compiles only `build/boot/startup.o` and executes the link phase. Standard processing applied: Make identifies that only one source file has changed, recompiles that specific file, and re-runs the linker.

4. **`touch scripts/mps2_m3.ld` then `make`:** Make falsely reports `Nothing to be done for 'all'` despite the updated timestamp.

The Linker Script Anomaly: The final transformation skips rebuilding despite the linker script modification. This occurs because `mps2_m3.ld` is an input argument passed via `LDFLAGS`, but it was never explicitly declared as a formal prerequisite in the dependency graph for `$(BIN)`. Consequently, Make remains completely blind to its timestamp updates.

Understanding the Disassembly Output (`objdump`)

Executing `arm-none-eabi-objdump -d` on the final ELF image. This tool reverse-engineers the raw machine code inside the `.text` section back into human-readable ARM Thumb assembly.

Unlike inspecting intermediate object (`.o`) files where memory addresses are left blank (unresolved), running `objdump` on the final linked `.axf` file provides the absolute truth of the physical memory layout. The output is structured in three columns:

- **Physical Address (Left):** Shows the exact Flash memory address assigned by the linker. For example, `00000040 <semihost_write0>` confirms that the code section correctly begins at `0x40`, immediately following the 64-byte hardware vector table.
- **Machine Code (Middle):** The raw hexadecimal representation of the instruction (e.g., `b480`) that the CPU physically reads.
- **Assembly Mnemonics (Right):** The human-readable instruction (e.g., `push {r7}`).

This view is critical for bare-metal debugging. It allows developers to verify exactly how the C compiler translated high-level logic into low-level stack manipulations, branches (`bl`), and comparisons (`cmp`). Furthermore, inspecting the `00000058 <swap_asm>` block proves that our custom inline assembly was injected into the firmware byte-for-byte exactly as we wrote it, with no compiler interference.

4.1.5 Exercise B2: Comparing Generated vs. Hand-Written Code

With both the C implementation (`swap_c`) and the Assembly implementation (`swap_asm`) functional, we can analyze how the C compiler translates high-level code into ARM instructions and compare it against our manual logic.

Unoptimized Translation (`-O0`)

First, we compile the C implementation without optimizations (`-O0`) and examine the resulting object file (`build/swap-c.o`).

Looking at the disassembly of our hand-written `swap_asm`, it remains perfectly pristine, containing only the 5 instructions we provided:

```
00000028 <swap_asm>:
  28:   6802            ldr     r2, [r0, #0]
  2a:   680b            ldr     r3, [r1, #0]
  2c:   6003            str     r3, [r0, #0]
  2e:   600a            str     r2, [r1, #0]
  30:   4770            bx      lr
```

However, the C compiler's unoptimized output for `swap_c` generates 20 instructions to achieve the exact same logic:

```
00000000 <swap_c>:
   0:   b480            push    {r7}
   2:   b085            sub     sp, #20
   4:   af00            add     r7, sp, #0
   6:   6078            str     r0, [r7, #4]
   8:   6039            str     r1, [r7, #0]
```

```
        ... [Memory shuffling omitted] ...
24:     bc80                pop     {r7}
26:     4770                bx      lr
```

Analysis of the Differences:

- **Prologue and Epilogue:** The generated `swap_c` contains instructions like `push {r7}`, `sub sp, #20`, and `pop {r7}`. These are standard C function prologues and epilogues that create a stack frame for local variables (like our `temp` variable).

- **The Effect of `naked`:** These stack instructions are completely absent from `swap_asm`. This proves that the `__attribute__( ( naked ) )` directive successfully prevented the compiler from wrapping our assembly in stack management code.

- **Memory Overhead:** Because optimization is disabled (`-O0`), the compiler rigidly translates the C code by aggressively shuffling data between the CPU registers and the RAM stack, resulting in numerous redundant `ldr` and `str` instructions.

Size-Optimized Translation (`-Os`)

To see the compiler's true capability, we recompile the C file using the size optimization flag (`-Os`).

The generated code for `swap_c` drastically changes:

```
00000000 <swap_c>:
   0:   6803            ldr     r3, [r0, #0]
   2:   680a            ldr     r2, [r1, #0]
   4:   6002            str     r2, [r0, #0]
   6:   600b            str     r3, [r1, #0]
   8:   4770            bx      lr
```

Conclusion: Once optimized for size, the compiler recognizes that it can perform the swap entirely within the CPU scratch registers (`r2` and `r3`) without needing a local variable on the stack. The resulting C-generated machine code is now virtually identical to our hand-written `swap_asm`. This confirms that while writing bare assembly offers absolute control, modern C compilers are exceptionally capable of generating hardware-optimal instructions when permitted.

4.1.6 Exercise B3: Extended Inline Assembly and the Clobber List

The `__attribute__((naked))` directive bypasses the compiler completely, which can be dangerous. A safer, more standard approach in C is to use **Extended Inline Assembly**. This allows us to write assembly logic while letting the C compiler automatically manage the stack, the function return, and the register allocation.

The Implementation

Instead of hardcoding physical registers (like `r0` or `r2`), we use symbolic placeholders (like `%[t1]`). We then map these placeholders to C variables in the input and output sections.

```
static void swap_ext( int *a, int *b )
{
    int temp1, temp2;
    __asm__ volatile (
        "ldr %[t1], [%[pa]]\n"
        "ldr %[t2], [%[pb]]\n"
        "str %[t2], [%[pa]]\n"
        "str %[t1], [%[pb]]\n"

        : [t1] "=&r" (temp1), [t2] "=&r" (temp2) /* Outputs */
        : [pa] "r" (a),       [pb] "r" (b)       /* Inputs */
        : "memory"                               /* Clobbers */
    );
}
```

Why the Clobber List Needs "memory"

The clobber list (the third section after the colons) tells the compiler's optimizer which parts of the system our assembly code destroys or alters.

Including `"memory"` in the clobber list is strictly required here. It warns the compiler that our assembly block reads from and writes to arbitrary RAM addresses. Without this warning, the compiler might falsely assume that global variables in RAM remained untouched during the `__asm__` block. To optimize speed, it might reuse old, cached values from registers instead of fetching the newly swapped values from RAM, leading to catastrophic logical failures.

Additionally, we **must not** include the `bx lr` instruction. Because this is a standard C function, the compiler automatically generates the prologue and epilogue. Forcing a manual return inside the assembly block would corrupt the stack and crash the processor.

4.1.7 Exercise B4: Memory Layout and Symbol Location

To guarantee our firmware complies with the Cortex-M3 physical constraints, we must verify the final memory layout. This is done by analyzing the output of `arm-none-eabi-nm -n` and cross-referencing it with the Linker's Map file (`build/output.map`)[cite: 13, 14].

Symbol Mapping Verification

By parsing the generated evidence, we locate four critical symbols and confirm their placement within the hardware memory regions defined by `mps2_m3.ld`:

1. `isr_vector` at address `0x00000000` (Region: **FLASH**). This confirms the vector table is placed at the exact absolute zero address, satisfying the hardware boot requirement.

2. `main` at address `0x0000007c` (Region: **FLASH**). As part of the executable `.text` section, the application logic is permanently stored in read-only Flash memory.

3. `_sdata` at address `0x20000000` (Region: **RAM**). This pointer marks the start of initialized global variables. As expected, it is mapped to the absolute beginning of the volatile SRAM.

4. `_ebss` at address `0x20000000` (Region: **RAM**). This pointer marks the end of the zero-initialized data section. Because the current application has no global variables, the start of data and the end of BSS resolve to the exact same physical address.

Tool Preference: `nm` vs. `output.map`

Both tools provide address resolution, but they serve different diagnostic purposes[cite: 14]:

- `arm-none-eabi-nm -n`: This tool is highly preferable for quick, direct lookups. The `-n` flag sorts the output numerically, instantly revealing the linear progression of symbols in memory without extraneous clutter.

- `output.map`: The map file is essential for deep debugging. While verbose, it reveals the *derivation* of the addresses, showing exactly which object file contributed a section and how linker mathematics (like `ALIGN`) calculated the final boundaries.

4.1.8 Exercise B5: Provoking Link-Time Failures

In a bare-metal environment lacking a Memory Management Unit (MMU)[cite: 5, 8], there is no hardware isolation to protect running tasks from overwriting each other's memory[cite: 7]. The heap grows upwards, and the stack grows downwards from the very top of the RAM (`0x20400000`)[cite: 8]. If they collide during execution, the system will silently corrupt data and crash[cite: 7].

To prevent this runtime catastrophe, we enforce strict boundary checks at *build time* directly within the linker script (`mps2_m3.ld`)[cite: 14].

Failure 1: Stack and Heap Collision

Action: We artificially increased `_Min_Stack_Size` (e.g., to `0x00400000`) until the reserved stack mathematically overlapped with the heap region.

Diagnostic Observed:

```
ld: scripts/mps2_m3.ld:125: region RAM overflowed: heap collides with sta
collect2: error: ld returned 1 exit status
```

Mechanism: This failure is triggered by the custom `ASSERT(__StackLimit >= _heap_top, "...")` statement at the end of the linker script. The linker calculates the boundaries before generating the ELF image; seeing that the limits cross, it intentionally aborts the build and prints our custom string.

Failure 2: Flash Memory Overflow

Action: We reverted the stack size and instead reduced the `LENGTH` parameter of the `FLASH` memory region (e.g., from `0x00400000` down to `0x40`) so that the `.text` section could no longer fit.

Diagnostic Observed:

```
ld: build/scratch-os.axf section `.text' will not fit in region `FLASH'
ld: region `FLASH' overflowed by 160 bytes
collect2: error: ld returned 1 exit status
```

Mechanism: This failure is triggered natively by the linker. It mathematically checks the size of the compiled `.text` and `.rodata` sections against the strict physical bounds of the hardware mapped in the script[cite: 8, 14].

Statement of the Difference

While both are build-time failures protecting the firmware, their fundamental nature differs:

- **The Flash Overflow is a physical, native constraint:** The compiled binary instructions are simply too large for the physical ROM chip defined in the hardware specifications. The GNU linker blocks this natively.
- **The RAM Overflow is a logical, custom constraint:** The physical RAM chip is actually large enough to store the variable sections, but the *dynamic runtime layout* predicted by the developer creates a severe software safety hazard. The linker only blocks this because we explicitly programmed it to do so via the `ASSERT` directive.

4.1.9 Exercise B6: Recovering the Vector Table from the Image

The Hardware Vector Table is located at the absolute beginning of the memory (`0x00000000`)[cite: 4]. By dumping the raw contents of the `.isr_vector` section using `objdump -s`, we can empirically verify the exact "boot contract" the Cortex-M3 processor executes upon power-on.

1. The First Word: Initial Main Stack Pointer

The first four bytes represent the starting address of the stack[cite: 4]. Because ARM is little-endian, the raw bytes `00 00 40 20` are assembled by the CPU into the 32-bit word `0x20400000`. Cross-referencing this with our `nm` output confirms this exactly matches the `_estack` symbol (the top of our 4 MiB SRAM).

2. The Second Word: The Reset Handler and the Thumb Bit

The second set of four bytes points to the first instruction to execute[cite: 4]. The raw bytes `d5 00 00 00` assemble into `0x000000d5`. However, checking `nm` reveals that `Reset_Handler` is actually located at `0x000000d4`.

This difference of 1 is not a defect, but a strict architectural rule of the Cortex-M3[cite: 15]. The CPU executes exclusively in Thumb mode. Setting the lowest bit

(LSB) of a function pointer to 1 is the hardware flag that explicitly commands the processor to execute the destination code in the Thumb-2 instruction state[cite: 15].

3. The Remainder of the Table: The Default Handler

Scanning the rest of the vector table reveals a recurring sequence of bytes (`cd 00 00 00`, resolving to `0x000000cd`). Removing the Thumb bit gives `0x000000cc`. Consulting `nm`, this address strictly belongs to `Default_Handler`. It recurs frequently because the startup code statically assigns this safe, infinite spin-loop to every single unimplemented hardware interrupt or exception, preventing the CPU from crashing into uninitialized memory if an unexpected fault occurs[cite: 10].

4. The Evidence-Based Boot Sequence

Based purely on the empirical evidence gathered from the ELF image, map file, and disassembly in this laboratory, the bare-metal boot sequence is as follows:

1. **Power-On:** The CPU hardware automatically reads address `0x00000000` and sets its Stack Pointer to `0x20400000` (top of SRAM).
2. **Vector Fetch:** The CPU reads address `0x00000004`, finds `0x000000d5`, drops the Thumb bit, and forces the Program Counter to jump to `0x000000d4` (`Reset_Handler`).
3. **Memory Initialization:** The disassembly of `Reset_Handler` shows it copying data from Flash to RAM and zeroing out the `.bss` section using the boundaries (`_sdata`, `_ebss`) defined in the `output.map`.
4. **Transition to Application:** The final instruction of the `Reset_Handler` disassembly is a branch with link (`bl <main>`), which transfers control to our C application to print the output via semihosting.